NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

SENTRI: A SEARCHABLE ENCRYPTED TRIPLESTORE FOR SEMANTIC DATA

DEVELOPING PROTOCOLS TO SECURE EXPRESSIVE SPARQL QUERIES OVER ENCRYPTED SEMANTIC DATA

MASTER IN COMPUTER SCIENCE

NOVA University Lisbon

Draft: *September 4, 2024*

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTMENT OF
COMPUTER SCIENCE

SENTRI: A SEARCHABLE ENCRYPTED TRIPLESTORE FOR SEMANTIC DATA

DEVELOPING PROTOCOLS TO SECURE EXPRESSIVE SPARQL QUERIES OVER ENCRYPTED SEMANTIC DATA

DIOGO HENRIQUE SILVESTRE MATOS CEBOLA

Advisers: Matthias Knorr
Assistant Professor, NOVA University Lisbon
João Leitão
Assistant Professor, NOVA University Lisbon
Henrique Domingos
Associate Professor, NOVA University Lisbon

MASTER IN COMPUTER SCIENCE

NOVA University Lisbon
Draft: *September 4, 2024*

Abstract

Ontologies provide a shared understanding of a domain and, in general, only contain public, and non-sensitive data, but sometimes they are used alongside electronic health records and clinical decision systems. These systems heavily manipulate personal data, which is sensitive. Additionally, ontologies themselves have an inherent private value, due to the investment of time and money in their development.

With ontologies gradually growing in dimension, the industry is currently evolving towards the use of collaborative solutions and the outsourcing of heavy computations, relying on cloud infrastructures or other shared computing platforms. These approaches, albeit inevitable, present new security concerns.

In this thesis, we investigate existing secure computation techniques, encryption protocols and storage systems with the finality of developing a practical and secure solution for the collaborative development, distribution and usage of ontologies.

Our main contributions from this research are: (i) SEnTri, a searchable encrypted triplestore that can evaluate secure SPARQL read and write queries, possesses reasoning capabilities and supports access control; (ii) Two secure protocols for the evaluation of SPARQL queries, based on Searchable Symmetric Encryption techniques and secure equality joins; (iii) A novel secure equality join, inspired by the DGK cryptosystem, that operates over homomorphically encrypted values.

The system has been evaluated using the LUBM benchmark under different configurations with varying security and performance trade-offs. T[1]

1) *TODO:* Summarize experimental evaluation results

Keywords: Ontologies, Searchable Encryption, Secure Computing, SPARQL, Secure Equality Join

Resumo

As ontologias fornecem conhecimento partilhado referente a um domínio. Fazem-no através da especificação dos conceitos relevantes, e suas relações, num dado domínio.

Em geral, contêm apenas dados públicos e não sensíveis mas, por vezes, podem ser usadas em conjunto de sistemas de apoio à decisão clínica e repositórios de dados médicos. Nestes sistemas existe manipulação frequente de dados pessoais, de carácter sensível.

Atualmente, com o gradual aumento da dimensão das ontologia, a indústria está evoluir no sentido da utilização de ferramentas colaborativas e *outsourcing* de computações pesadas, recorrendo a infrastruturas *cloud* e outras plataformas de computação partilhada. Estas soluções, embora inevitáveis, apresentam novas questões de segurança.

Nesta tese, investigamos técnicas existentes de computação seguras e propomos a definição de um novo protocolo de Encriptação Simétrica Pesquisável, que suporta *queries* SPARQL expressivas sobre ontologias encriptadas.

Palavras-chave: Ontologias, Encriptação Pesquisável, Computação Segura, SPARQL

Contents

List of Figures

List of Tables

List of Listings

Acronyms

ACL Access Control List 33, 69
ACP Access Control Policy 35, 37
AES Advanced Encryption Standard 26
AES-CBC Advanced Encryption Standard in cipher-block chaining mode 29
API Application Programming Interface 19, 33

BGP Basic Graph Pattern 15, 52, 59
BSBM Berlin SPARQL Benchmark 19

DBMS Database Management Systems 27
DGK Damgård, Geisler and Krøigaard 39, 40

FHE Fully Homomorphic Encryption 3, 20

GO Gene Ontology 1
GOOSE Graph OutsOurcing and SPARQL Evaluation 4, 28, 29, 30

HE Homomorphic Encryption 2, 3, 6, 20, 21, 30
HTML HyperText Markup Language 8
HTTPS Hyper Text Transfer Protocol Secure 33

IAM Identity and Access Management 33, 71
IND-CKA2 Indistinguishability Against Adaptive Chosen Keyword Attack 24
IND-CKA1 Indistinguishability Against Non-Adaptive Chosen Keyword Attack 24
IND-CPA Indistinguishability Against Chosen Plaintext Attacks 23
IRI Internationalized Resource Identifier 6, 7, 8, 12, 15

LFHE Leveled-Fully Homomorphic Encryption 3, 20
LUBM Lehigh University Benchmark 19

M/M Multiple Writer/Multiple Reader 23, 32, 35
M/S Multiple Writer/Single Reader 23

NCI National Cancer Institute 1

OPE Order-Preserving Encryption 22, 28
ORAM Oblivious RAM 3, 20, 30
OWL2 Web Ontology Language viii, x, 1, 7, 11, 12, 13, 14, 15, 18, 19, 26, 30

PEKS Public Key Encryption with Keyword Searchable 3, 22, 30
PHE Partially Homomorphic Encryption 3, 20
PRF Pseudo-Random Function 24
PRP Pseudo-Random Permutation 25

RDF Resource Description Language x, xii, 1, 7, 8, 9, 10, 11, 15, 16, 17, 18, 19, 29, 30, 32, 35
RDFS Resource Description Framework Schema ix, 1, 7, 9, 10, 11, 12, 18, 30
REST Representation State Transfer 33
RSA Rivest-Shamir-Adleman 39

S/M Single Writer/Multiple Reader 23
S/S Single Writer/Single Reader 23, 31
SE Searchable Encryption 2, 3, 6, 20, 22, 23, 30, 31, 32
SEnTri Searchable Encrypted Triplestore x, 4, 32, 33, 34, 35, 36, 37, 38, 39, 41, 42, 47, 48, 52, 53, 54, 55, 56, 58, 59, 62, 63, 66, 67, 69, 71, 74, 75, 76, 77, 78
SHE Somewhat Homomorphic Encryption 3, 20
SLA Service Level Agreement 2
SPARQL SPARQL Protocol and RDF Query Language x, 1, 4, 5, 7, 15, 16, 17, 18, 19, 28, 29, 30, 32, 35, 36, 37, 38, 39, 41, 47, 48, 58, 69, 76, 77, 78
SQL Structured Query Language 16, 27, 28
SSE Symmetric Searchable Encryption 3, 4, 22, 23, 24, 25, 30, 31, 35, 62
STE Structured Symmetric Encryption 3, 25
SWRL Semantic Web Rule Language 7

TLS Transport Layer Security 33
TURTLE Terse RDF Triple Language x, 8, 12, 17

UCRPQ Unions of Conjunctions of Regular Path Queries 4, 28, 29
UUID Universally Unique Identifier 35, 36, 38, 76

W3C World Wide Web Consortium 7
WWW World Wide Web 1

XML Extensible Markup Language 7, 8, 11

1

Introduction

1.1 Context

The Semantic Web In 2001, the term *Semantic Web* [7] was first used to define what would be the evolution of the World Wide Web (WWW). The traditional *Web* solved the challenge of distributing information at scale and globally, however it was designed for human use. Documents are easily accessible and readable by humans, but for automated agents little more than document retrieval is possible. Contrary to humans, machines cannot reason over the information they access, if they do not have access to the *semantics of the data*.

The *Semantic Web* addresses this issue, by redesigning the representation of information, in a way that not only preserves meaning and relationships but is also machine-readable. A concept is mapped to a unique identifier, and its meaning is encoded in sets of triples, each triple consisting of a subject, predicate and object (similar to an elementary sentence). *Ontologies* provide a shared understanding of a domain, by formalizing the specification of relevant concepts, and their relations, in such domain.

This novel method of knowledge representation differs from the previous, centralized way, and greatly simplifies the process of interconnecting different knowledge bases. Centralized representations tend to be more expensive to develop, usually needing to be built from scratch [61]. Furthermore, for sharing, both parties need to agree on the semantics. In contrast, using ontologies, information is semantically linked by default. In the *Semantic Web*, machines can truly communicate, share and reason about distinct knowledge bases.

Semantic data is specified using a stack of different language specifications. The basic data model is specified using RDF [106], these resources can be organized in hierarchies with RDFS [110] and to define ontologies we use OWL2 [99]. To query semantic data, the standard language is SPARQL [114, 115, 116].

Industry Landscape Knowledge representation is essential in the industry [64] and ontologies help remove ambiguity in terminology. Nowadays, they are being developed and applied in a variety of domains [112] ranging from the health sector[1], to linguistics [2]

[1] The NCI Thesaurus [83], developed by the National Cancer Institute (NCI) or the Gene Ontology (GO) [39], the world's largest source of information on the functions of genes, are some existing applications.

[2] The WordNet [57] is a lexical database for English where related words and concepts are semantically linked.

and even being used by companies like NASA[3] [6].

To developed and maintain ontologies, advanced semantic editors are the standard tool for professionals, with Protégé [60] being a very popular example. These provide numerous functionalities, usually based on plugins (e.g. automatic verification of errors and consistency in the design; debugging tools; query answering; graphic visualization of data and knowledge).

With datasets gradually growing in dimension, the industry is currently evolving towards the use of collaborative solutions [95, 94] and outsourcing heavy computations, relying on cloud infrastructures or other shared computing platforms. These approaches, albeit inevitable, present new security concerns and engineering challenges.

By 2021, 50 percent of all enterprise workloads were already hosted in *public clouds* [81]. As of 2023, worldwide end-user spending on public cloud services is forecast to grow 20.7% more than the growth forecast for 2022 of 18.8% [33].

Research on secure cloud computing [14] studies such adoption but also discusses security concerns. Adoption can be explained with the cost efficiency[4], high scalability, flexibility and availability that cloud provides. Additionally, the pay-per-use model is an easy entry point to the ecosystem.

Cloud Security Concerns While delegation of responsibilities is beneficial for costs, it is prejudicial to control. Using cloud services implies trusting its owner. A Service Level Agreement (SLA) defines, in accordance to metrics, the service levels of the system that the vendor should provide (e.g. percentage of availability), while also defining penalties in case of not achieving what was agreed-on. However, the risk of loss of privacy and data breaches still exists. A problem that is especially concerning when sensitive data is being stored by such services. This is much more relevant in *public clouds* where, compared to *private clouds*, trust is distributed between various actors and not on a single organization, namely the cloud operator [14].

In cloud storage services, administrators, or hacker with root privileges, might have access to the data [10]. Additionally, these cloud services might outsource computations to third parties whom we do not trust. In this setting, we usually consider the *honest-but-curious*, or *passive*, adversary model. This adversary will follow the protocol properly but keep a record of all its intermediate computations [35]. Furthermore, it cannot do any other action except attempting to learn private information while observing views of the protocol execution [27].

Security Approaches A first approach to secure data in the cloud would be to use *Encryption at Rest*, which naively encrypts all data stored in the untrusted servers. If a user wants to update or query the data, they will have to download all the data, and decrypt it. This incurs in high network requirements and is computationally expensive on the client-side, hindering all the benefits of using a cloud service to store the data.

Although we can observe some major database services, like MongoDB, starting to adopt Searchable Encryption [59], the majority of cloud and storage providers only support the former solution.

Currently, active research is being done in the fields of Searchable Encryption (SE) and Homomorphic Encryption (HE).

[3] Observing the visual representation provided by the Linked Open Data Cloud [55] is a good exercise to perceive the dimension of adoption of this new paradigm.

[4] Cloud services reduce implementation, maintenance, power, cooling, floor space, storage and operational costs.

Homomorphic Encryption is based on the concept of algebra homomorphisms, where both decryption and encryption can be seen as an homomorphism of the relations between the plaintext and the ciphertext [3, 31]. Homomorphic Encryption, based on the type and number of operations supported (e.g. addition or multiplication of encrypted values), can be classified as: Fully Homomorphic Encryption (FHE) supporting any number of operations with no restriction on the number of times; Somewhat Homomorphic Encryption (SHE) permitting some types of operations, a limited number of times; Partially Homomorphic Encryption (PHE) allowing one type of operation with no restriction on the number of times; and Leveled-Fully Homomorphic Encryption (LFHE) [11] supporting any type of operations, but with a pre-determined expected depth-L (can be executed up to L times).

Searchable Encryption uses classical encryption schemes to perform encrypted search queries, sequentially over the ciphertext [85], or through the generation of searchable *encrypted indexes* accessible with *tokens*, generated via *trapdoor* functions. Searchable Encryption can be divided in Symmetric Searchable Encryption (SSE), when it is based on symmetric key cryptography, and Public Key Encryption with Keyword Searchable (PEKS), when asymmetric key algorithms are used. These techniques may leak some information about the *indexes*, *search pattern*, and *access pattern* during execution. Most approaches leak, at least, the *search* and *access patterns* [10]. Recently, to mitigate information leakage, *hybrid* solutions combining the software approach of SE with *trusted hardware* have been proposed [29].

Besides HE and SE, Oblivious RAM (ORAM) [36] could theoretically be used as an alternative. However, ORAM and HE continue to be too inefficient for real world applications, when considered as an exclusive solution [10]. Furthermore, SSE is often more practical than PEKS, as asymmetric cryptographic schemes tend to be more efficient than their symmetric counterparts [70].

1.2 Problem

Usually, ontologies only contain public, and non-sensitive data, but sometimes they are used alongside electronic health records and clinical decision systems. These systems heavily manipulate personal data, which is sensitive. Additionally, ontologies themselves have an inherent private value, due to the investment of time and money in their development. With all these potential security needs, a solution that supports the secure development, sharing and usage of ontologies alongside sensitive data is essential.

Taking into consideration the efficiency, security trade-offs and ease of implementation from the various alternative techniques, a pure software SSE approach seems to be the most practical and easier to adopt solution, not requiring excessive computational power or special trusted hardware. Recently, Chase and Kamara, on their work on Structured Symmetric Encryption (STE), generalized index-based SSE to support queries on *structured* data [15].

Semantic data can be represented as a graph and STE supports various types of query operations in graph-structured data. Some existing schemes, adapted to conceptual graphs, need to pre-compute all possible query answers *a priori*, which is not practical for large datasets [73]. Furthermore, there can be leakage of information about the topology of the data while trying to pre-compute all query answers.

Recent developments do not require pre-computation of all query answers. The *CryptGraphDB* [2] system is an adaptation of CryptDB [76, 75, 74] capable of answering queries over conceptual graphs, but it was developed for the cypher [63] language

and not SPARQL.

Another work, focused on access control for RDF datasets, did also implement a solution for retrieving simple triple patterns over an encrypted dataset, but, due to performance needs, hashes were used for indexing the encrypted triples. This leaks the structure of the dataset, which they tried to mitigate by creating dummy triples [28].

The Graph OutsOurcing and SPARQL Evaluation (GOOSE) [17] framework, demonstrates how to use multiparty computation techniques to answer Unions of Conjunctions of Regular Path Queries (UCRPQ) over encrypted graphs, only relying on a standard SPARQL engine. But deterministic data obfuscation techniques are used, and therefore the structure of the graph is also leaked. This framework only focuses on SPARQL read queries.

Although, a lot of progress has been made in this field of research there are still various open questions. In this thesis, we will focus on answering the following questions:

1. *How to support secure and expressive read and write SPARQL queries over encrypted data?*

   A practical solution ought to support more than basic triple pattern retrieval, it must be capable of querying complex patterns. As such it should be able to execute secure set operations without the client needing to decrypt the intermediate values (obtained from simple triple patterns). Furthermore, it must be able to execute write queries, so that datasets can be updated.

2. *How can we add reasoning capabilities to the solution?*

   A simple SPARQL engine does not possess reasoning capabilities. The query results may or may not possess inferred values, depending on the underlying storage solution. When working with ontologies, a system should have reasoning capabilities to take full advantage of the semantics preserved within the data.

3. *How can we minimize the information leakage and enforce access control?*

   A data outsourcing solution should support granular access control over the shared datasets. SSE solutions always leak some information about the data, we will investigate how to minimize such leakage during query execution.

1.3 Contributions

In this thesis we aim to develop a practical and secure solution for the collaborative development, distribution and usage of ontologies.

SEnTri SEnTri is a searchable encrypted triplestore that can evaluate secure SPARQL read and write queries, possesses reasoning capabilities and supports role based access control.

P1, P2 - Two secure SPARQL evaluation protocols We developed two secure SSE protocols for the evaluation of SPARQL queries, in a dynamic and multi-user setting. The main distinction between the two protocols is in the equality function used in join operations. The first protocol relies on deterministic encryption whereas the second protocol uses additive homomorphic encryption. The use of non-deterministic encryption mitigates information leakage.

A secure equality join over non-deterministic values For the second protocol we developed a novel equality join, which operates over non-deterministic values, which we denominate Eq_{Tags}. These values are encrypted using the DGK cryptosystem, an additive homomorphic scheme developed for secure integer comparison.

The prototype will be available as open-source code in a public repository, alongside the benchmark results.

1.4 Document Organization

The remaining of the document will be organized as follows:

Chapter 2 discusses related work on *Semantic Web* and *Secure Computations* and compares existing solution for secure semantic data outsourcing and SPARQL evaluation.

Chapter 3 presents the solution.

Chapter 4 describes the implementation details of each of the system's components.

Chapter 5 presents the experimental evaluation procedures, describes and discusses the obtained results.

Chapter 6 summarizes main conclusions from the research and suggests future work.

2

Related Work

In this chapter, we will discuss the architecture and design of the *Semantic Web* as well as analyzing work already done in the fields of *Secure Computations*. The remaining of the chapter is organized as follows: on Section 2.1 we discuss the *Semantic Web* stack, languages and tools; Section 2.2 describes work on *Secure Computation*, with particular attention regarding HE and SE; finally, on Section 2.4 we summarize and discuss which work is relevant.

2.1 Semantic Web: A layered architecture

In Section 1.1, we have contextualized the main philosophy and design principles of the *Semantic Web*:

1. Usage of *labeled graphs* as the data model where concepts are represented as triples consisting of a subject, predicate and object (with nodes being objects and relations between such objects modelled as the edges);

2. All data items are identified by Internationalized Resource Identifier (IRI) [25];

3. Usage of ontologies to model semantics within a domain.

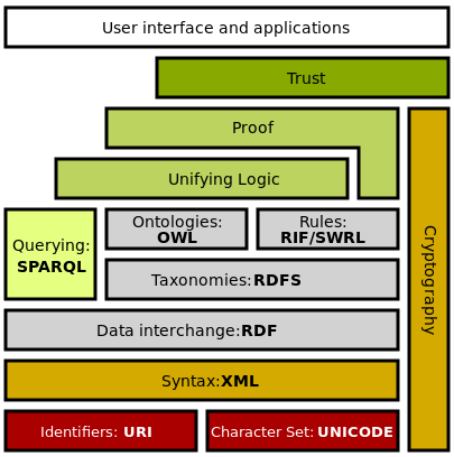

Figure 2.1: The *Semantic Web* stack [118].

Additionally, the *Semantic Web* can also be seen as a stack of various layers [5] (figure 2.1). Most of these layers are standardized due to the collective work of the World Wide Web Consortium (W3C).

Briefly describing this layered architecture, at the root level, we have the IRIs and a character set providing the baseline of communication, with distinguishable identification of resources and an agreed encoding.

At the syntax level we find the Extensible Markup Language (XML) [97]. XML supports the definition of unlimited *tags*, that can be placed in the text of a document. These *tags* facilitate the creation of hierarchy and give *structure* to the information.

From structured data we then define the basic data model using RDF [106]. With a XML like syntax, we can write statements about resources, thus embedding *semantics* within the information, as *metadata*.

The RDFS [110] is a vocabulary description language to organize RDF resources into hierarchies, based on primitives such as *classes* and *properties*. It can be seen as a rudimentary *ontology language* [5].

Ontologies are defined with OWL2 [99]. Like RDFS, it is also a vocabulary description language, but provides more expressiveness (e.g. relations between classes, *cardinality*, *equality*, richer properties with characteristics).

RDF triples can be interpreted as logical binary predicates, making reasoning possible over RDF-based data. This allows us to infer and validate ontologies, and also provide explanations on operations' results. The rules layer is a reference to this specific part of the architecture stack. There exist various languages, such as the Semantic Web Rule Language (SWRL) [117], that allow us to express rules as well as logic.

To query and retrieve any data based on RDF we use SPARQL.

The Proof and Unifying Logic layers include the notion of interconnectivity of terminologies between *knowledge bases*, with explainable proofs to operations.

Only with secure cryptographic schemes can we reach the trust layer. The security layer is still not standardized.

The top layer, references all the applications and interfaces that use this new paradigm of data representation.

In the following of this Section we will describe, in more detail, some languages and tools. We will focus on RDF in Section 2.1.1; on RDFS, in Section 2.1.2; on OWL2, in Section 2.1.3; on SPARQL, in Section 2.1.4; on common tools and software, in Section 2.1.5.

2.1.1 RDF: Resource Description Framework

2.1.1.1 Overview

RDF [106] is a flexible *domain-independent* data model. The RDF basic primitives are *resources*, *properties* and *literals*. We can construct *statements* about resources by organizing these basic primitives in sets of *triples*. A triple is composed of a *subject*, a *predicate* and an *object*. In a statement (triple) about a subject (always a resource) we define the value (object) of a property (predicate), which can be another resource or a literal.

Literals are atomic values (e.g. strings, numbers). Resources and properties, respectively, model the "things"and relationships between such "things". Both are identified using IRIs. Resources without an IRI are called *blank nodes* (or *bnodes*).

To make a statement B about a statement A we can use a technique called *reification*. Reification consists in: firstly, defining a statement B where the object $objA$ represents the statement A; secondly, creating the properties *subject*, *predicate* and *object* mapped from

objA to the original subject, predicate (now a resource), and original object of statement *A* [5].

2.1.1.2 Syntax

Graphical RDF being a data model, can be written using different syntaxes. Graphically we can represent statements as a labeled and directed graph. The *nodes* are labelled with the resources IRIs, a literal value or left blank, whereas *arcs* are labelled with the properties IRIs. The arcs are directed from subject to object of a statement. Figure 2.2 provides an example of a RDF graph.

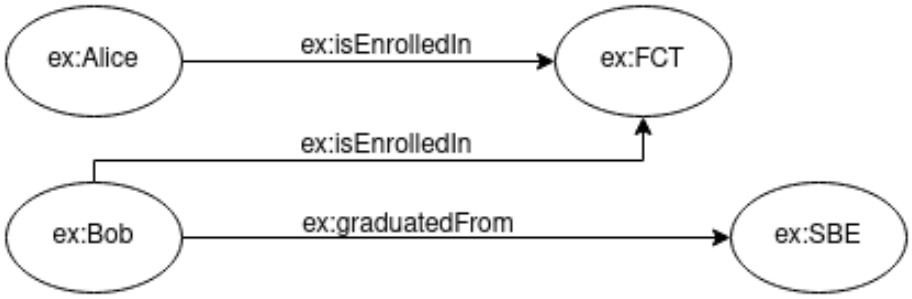

Figure 2.2: A simple RDF graphical example.

Text-Based There are multiple text-based syntaxes for RDF. Some of the most commonly used are TURTLE [108] (listing 2.1), and its extension for named graphs *TriG* [107]. In TURTLE, IRIs are enclosed by angle brackets. The subject, property, and object are written in this order. Statements are finished by a period. Literals are surrounded in quotes with the type of data specified with an IRI. The data value and type are separated by ^^ (if no data type is mentioned, the value is inferred as being a string). We can also define IRI abbreviations (using @prefix, followed by the prefix to substitute the IRI, a colon, the IRI and a period) and simplify statements, for easier readability (e.g.using a semicolon to make more than one statement about the same subject, defining statements with nested blank nodes using square brackets). Comments start with #.

```
1 # FCT is located in Almada
2 <http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/ontology/
      ↪ location> <http://www.my-custom-domain/resource/Almada> .
3
4 # FCT has 8.000 students
5 <http://www.my-custom-domain/faculties.ttl#FCT> <http://www.my-custom-domain/faculties.
      ↪ ttl#asNumberOfStudents> "8000"^^<http://www.w3.org/2001/XMLSchema#integer> .
```

Listing 2.1: Simple RDF example, written in TURTLE syntax.

XML/RDF [109] is a standardized XML-based syntax and *RDFa* [111] permits RDF to be embedded in the HyperText Markup Language (HTML)[1].

[1]We recommend the curious reader to read the W3C recommendations to better learn the syntaxes.

2.1.2 RDFS: RDF Schema

2.1.2.1 Overview

RDF is a flexible data-model. However, for interconnectivity of different RDF knowledge bases, semantics for restricting meaning in a specific domain are needed.

RDFS [110] does that by providing vocabulary to describe groups of related resources and the relationships between these resources. This vocabulary is a set of RDF resources identified under the `http://www.w3.org/2000/01/rdf-schema#`.

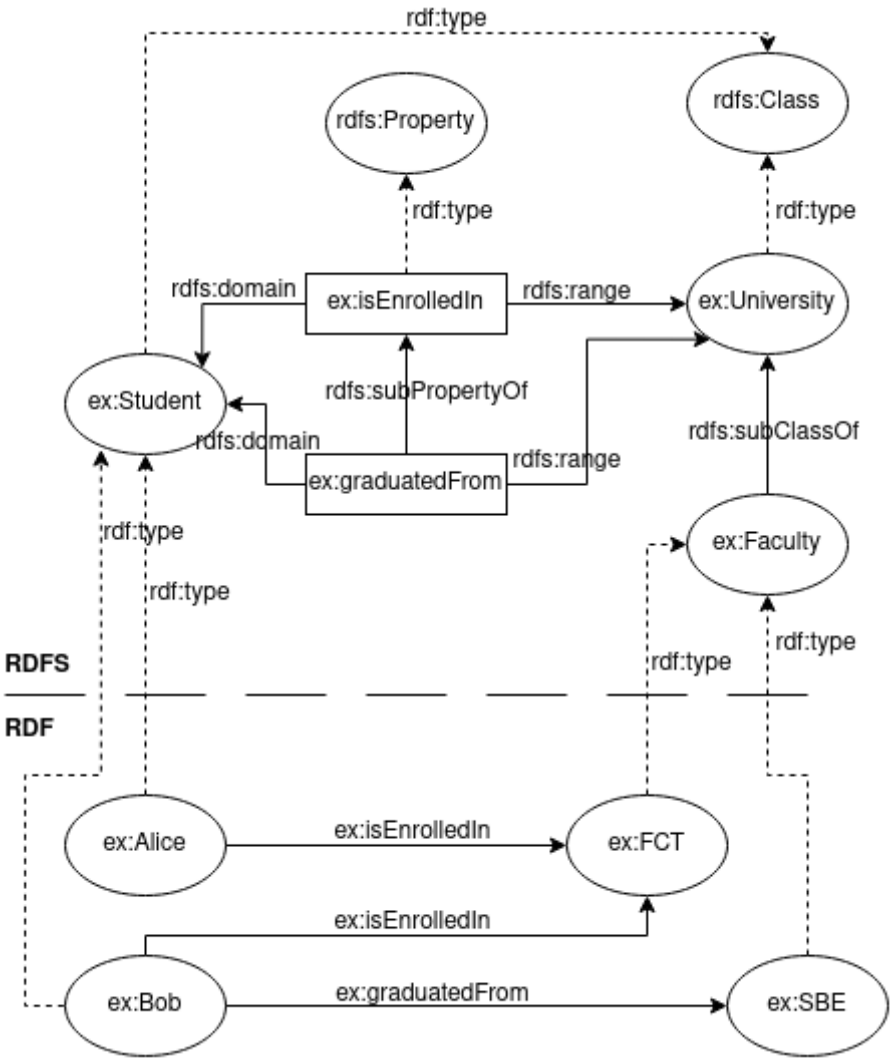

Figure 2.3: A simple RDFS graphical example.

2.1.2.2 RDFS primitives

Classes group resources together, the *instances* of the class. *Properties* are defined by specifying to which classes of resource they may apply, using the mechanism of *domain* and *range*. This allows for easy extension, by not having to redefine classes wherever a new property, that applies to an existing class, is needed.

To state that a resource is an instance of a class we use the *rdf:type* property.

The *rdfs:subClassOf* is used to define that all the instances of a class are instances of its *superclasses*: if *A* rdfs:subClassOf *B*, then instances of *A* are also instances of *B*.

The *rdfs:subPropertyOf* states that instances related by a property are also related by its *superproperties*: if *A* rdfs:subPropertyOf *B*, then instances related to *A* are also related to *B*.The properties rdfs:subClassOf and rdfs:subPropertyOf are *transitive*, and there are no restrictions on the number of superclasses or superproperties that, respectively, a class or a property can have.

The *rdfs:domain* property defines that any resource with a given property is an instance of the domain classes that the property applies to. The *rdfs:range* property specifies that the values of the property are instances its range classes.

In tables 2.1 and 2.2, we summarize the core RDFS classes and properties. In figure 2.3, we extend the RDF graphical example 2.2 with RDFS.

RDFS Core Classes	
Class	**Description**
rdfs:Resource	The class resource, everything.
rdfs:Class	The class of classes.
rdfs:Literal	The class of literal values, e.g. textual strings and integers.
rdf:Property	The class of RDF properties.
rdfs:Statement	The class of RDF statements.

Table 2.1: RDFS Core Classes and respective description.

Core RDFS Properties			
Property	**Description**	**Domain**	**Range**
Relationship Definition			
rdf:type	The subject is an instance of a class.	rdfs:Resource	rdfs:Class
rdfs:subClassOf	The subject is a subclass of a class.	rdfs:Class	rdfs:Class
rdfs:subPropertyOf	The subject is a subproperty of a property.	rdf:Property	rdf:Property
Property Restriction			
rdfs:domain	A domain of the subject property.	rdf:Property	rdfs:Class
rdfs:range	A range of the subject property.	rdf:Property	rdfs:Class
Reification			
rdf:subject	The subject of the subject RDF statement.	rdf:Statement	rdfs:Resource
rdf:predicate	The predicate of RDF statements.	rdf:Statement	rdfs:Resource
rdf:object	The object of RDF statements.	rdf:Statement	rdfs:Resource
Utility			
rdfs:seeAlso	Further information about the subject resource.	rdfs:Resource	rdfs:Resource
rdfs:isDefinedBy	The definition of the subject resource.	rdf:Resource	rdfs:Resource
rdfs:comment	A description of the subject resource.	rdf:Resource	rdfs:Literal
rdfs:label	A human-readable name for the subject.	rdf:Resource	rdfs:Literal

Table 2.2: RDFS Core Properties and respective description.

2.1.3 OWL2: Web Ontology Language

2.1.3.1 Overview

An *ontology* was defined by Studer et al. as "a formal, explicit specification of a shared conceptualisation"[87]. An ontology should specify consensual and machine-readable knowledge by means of a formal definition of concepts, where concepts have explicit types and restrictions. Therefore, an *ontology language* should provide a well-defined

syntax, formal semantics, convenient and sufficient expressiveness and efficient reasoning support.

RDFS can be considered a very limited *ontology language*. OWL2 [99] provides a number of additional features (e.g. cardinality constraints, class equivalence, intersection and disjunction). OWL2 does not simply extend RDFS, because RDFS primitives are too expressive. This would make reasoning undecidable [5]. As such, OWL2 is divided in two sublanguages: *OWL2 Full* [103], a direct extension of RDFS with all OWL2 semantics; *OWL2 DL* [98], a language with some restrictions on the way OWL2, RDF and RDFS primitives may be used.

The correspondence theorem of [98] states that: given an *OWL2 DL* ontology, inferences drawn using its *Direct Semantics* will still be valid if the ontology is mapped into an RDF graph and interpreted using the *RDF-Based Semantics*, the semantics of *OWL2 Full*.

OWL2 can be expressed with any valid RDF syntax, however there are various specific syntaxes: the Functional-Style Syntax [104], which is very compact and was the language used in the formal specification of the language; the Manchester Syntax [100], designed to be human-readable (it is very popular in user interfaces of semantic editors); and the OWL/XML Syntax [105], which allows easier processing with XML tools.

Figure 2.4 details the structure of OWL2. Finally, *OWL2* can have different specification, called profiles [101], which provide reasoning in polynomial time. The coverage of OWL2 profiles is not on the scope of this work.

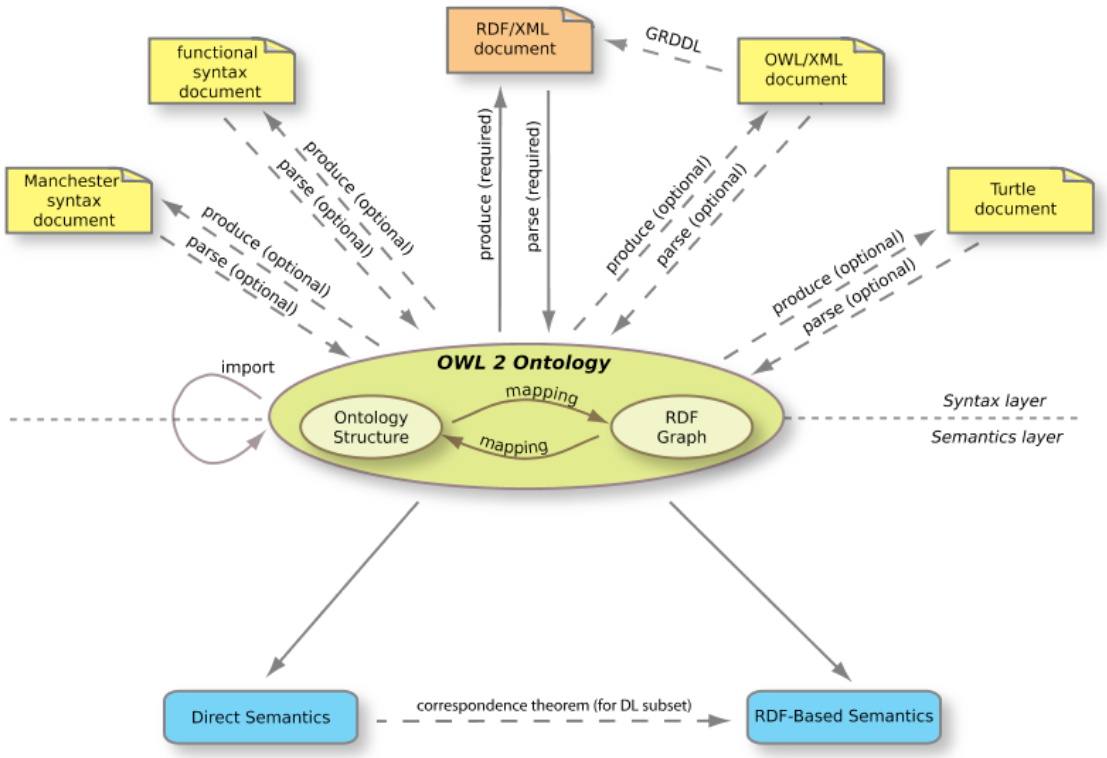

Figure 2.4: The OWL2 Structure [99].

2.1.3.2 OWL Language

In OWL2 the term individual is equivalent to instance in RDFS. An *assertion* is a statement about a resource. *Expressions* are a combination of classes, properties and instances. An

axiom relates such expressions to classes[2]. Listing 2.2 provides a simple OWL2 example.

Individual Assertions In OWL2 *assertions* about individual resources can be used to define their type, class membership, properties and even provide ways to define identity and negative statements. More specifically on the identity statements, because OWL2 has an open world assumption, we can never differentiate two individuals just because they have different IRIs. To do so we use owl:differentFrom. To explicitly state negative facts we use owl:NegativePropertyAssertion [5].

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

# The base namespace of the FCT ontology
@base<http://www.my-custom-domain/fct.ttl> .

<http://www.my-custom-domain/fct.ttl>
    rdf:typeowl:Ontology ;
    rdfs:label"FCT Ontology"^^xsd:string ;
    rdfs:comment "An example OWL2 ontology"^^xsd:string ;
    owl:versionIRI <http://www.my-custom-domain/fct.ttl#1.0> .

# Class Expression
_:x rdf:type owl:Class ;
    owl:unionOf ( :Mandatory :Optional ) .

# Class Axiom
:Course owl:equivalentClass _:x .

# Class Assertion
:class101 rdf:type :Course .
```

Listing 2.2: Simple OWL2 example, written in TURTLE syntax.

Classes In OWL2 every resource that is of type owl:Class is a class. The two classes defined by default are *owl:Thing* and *owl:Nothing*. Respectively, these two specify the superclass of all classes and the subclass of all classes: every individual is a member of owl:Thing and no individual can be member of owl:Nothing. This is very useful for reasoning, because inconsistency is detected when classes are equivalent to owl:Nothing (have no individuals in their membership).

Class Axioms In OWL2 we can relate *expressions* to classes using *axioms*. These can define:

- *Subclass Relations*, with equivalent usage like in RDFS;
- *Class Equivalence*, in equivalent classes every individual must be a member of each class;
- *Enumerations*, a type of class definition via explicitly stating all individuals who belong to the class membership;

[2]For an overview of the language we recommend the reader to consult the reference guide [102].

- *Intersection*, in a class described as the intersection of various classes, all of its members must be also individuals belonging to the membership of each of those classes;
- *Union and Disjoint Union*, in a class described as the union of various classes, every individual belonging to its membership must also belong to the membership of at least one of those classes. In the particular case of disjoint union, the intersection of the classes is equivalent to owl:Nothing;
- *Disjoint Classes*, in disjoint classes no individual can be a member of both class (the intersection of the classes is equivalent to owl:Nothing);
- *Complement*, complement classes contain all the individual such that the union of both classes is equivalent to owl:Thing.

Properties OWL2 has three types of properties: *object* properties, which relate individuals to other individuals; *datatype* properties, which relate individuals to literals; and *annotation* properties; which are used to specify human-readable labels, comments and explanations.

The *top* and *bottom* properties specify, respectively, properties that relate to all and no individuals: owl:topObjectProperty is the superproperty of all owl:ObjectProperties and owl:bottomObjectProperty relates to no individual; owl:topDataProperty is the superproperty of all owl:DatatypeProperties, relating an individual to any possible literal value, and owl:bottomDataProperty relates no individual to any literal value.

Properties can have different characteristics:

- *Transitive Properties*: if a relates to b via property p and c relates to a via property p, then c relates to b via property p;
- *Symmetric Properties*: if a relates to b via property p, then inverse is true;
- *Asymmetric Properties*: if a relates to b via property p, then the inverse is $\bot$;
- *Functional Properties*: if for each instance the property can only have one unique value;
- *Inverse-functional Properties*: if the object of a property statement relates to a single subject;
- *Reflexive Properties*: if every individual relates to itself via the property;
- *Irreflexive Properties*: if no individual relates to itself via the property.

Property Axioms Just like class axioms, we can specify *property axioms* to describe the way properties relate to classes and other properties. These allow us to define:

- *Domain and Range*: used to determine class membership of individuals;
- *Inverse Relationship*: used to specify the inverse of the property;
- *Equivalence*: used to specify equivalent properties. Individuals related by one property will always relate by all equivalent properties;

- *Disjointness*: used to specify disjoint properties, where individuals related by one property can never be related by the other;
- *Property Chains*: used to aggregate connected properties under another property.

More Granular Class Axioms We can write more granular class definitions in OWL2 than by simply defining class axioms. This is achieved by combining class and property axioms. It is done by defining a class axiom that relates to an anonymous class (owl:Restriction) which contains axioms on its properties. The anonymous class definition captures all individuals that satisfy such restrictions. As such we can define:

- *Universal and Existential Restrictions*: which the members of the class must satisfy. The class is defined as a subclass of an anonymous class with property restrictions on, respectively, owl:allValuesFrom and owl:someValuesFrom;
- *Necessary and Sufficient Conditions*: relating a class to a property, using rdfs:subClassOf, does not provide sufficient information to conclude whether an individual belongs to the class membership or not (it only states necessary, not sufficient, conditions for class membership). With owl:equivalentClass, it becomes clear that an individual belongs to the class membership, if it satisfies the restriction (the conditions are necessary and sufficient).
- *Value, Cardinality, Data Range and Datatype Restrictions* on the anonymous class: limiting the value, cardinality of the value, range of possible values and the datatype of the property value relating to members of the superclass;
- *Self Restrictions*: on the individuals captured by the anonymous class.

2.1.3.3 OWL2 DL

Contrary to *OWL2 Full*, *OWL2 DL*, being less expressive, retains *decidability*. This is achieved by imposition of various restrictions [5]:

- The application of OWL2 primitives to each other is not allowed; only classes and non-literals resources may be defined, with all classes being individuals of owl:Class;
- Properties are disjoint individuals of either owl:ObjectProperty or owl:DatatypeProperty;
- Annotation properties are ignored by reasoners;
- Properties, for which range includes non-literal resources, are separated from those which relate to literal values;
- A resource can only be a class, property, or instance at the same time. Classes, properties or instances with the equal names will be treated as distinct (a mechanism called punning);
- Only functional properties can be assigned to datatype properties;
- Only object properties can have an inverse;
- Property chains can only involve object properties;
- Self restrictions on datatype properties are not allowed.

2.1.4 SPARQL

2.1.4.1 Overview

In the previous Section we've discussed the structure of OWL2. OWL2 documents have a direct mapping to RDF documents. As such, we can publish and query OWL2 ontologies using the standard approach for RDF data.

RDF triples are stored in *triple stores*, specialized databases for this type of data. The underlying architecture of such databases varies, from relational to NoSQL approaches [20, 58]. The recommended query language to use is SPARQL [114]. It provides a range of operations over published RDF graphs, accessible via SPARQL endpoints: from read queries [116] to write operations [115, 113].

2.1.4.2 SPARQL Queries

In this section we will mathematically define core components of *SPARQL*, briefly discuss the basis of functioning for this type of queries, explain how they are evaluated, given the semantics of *graph patterns*, and cover their syntax.

Definition 1 (**RDF dataset**). Let I, B, L be disjoint sets of IRIs, blank nodes, and literals, respectively. RDF data is a set of subject, predicate and object triples $t = (s, p, o)$, where $s \in I \cup B$, $p \in I$ and $o \in I \cup B \cup L$.

Let V be a set of nodes formed by subjects and objects and E be a set of edges, labeled by the corresponding predicate. A set of RDF triples can be represented as a directed, edge-labeled, graph $G = (V, E)$. An RDF dataset is set of *named* graphs and the *default* unnamed graph. We will define such dataset as $D = \{G, \gamma\}$, where γ is a function which assigns a named graph $\gamma(i)$ to each i in a finite set $dom(\gamma) \in I \cup B$ [72, 77].

Definition 2 (**SPARQL Query**). Let T = {SELECT, ASK, CONSTRUCT, DESCRIBE}, a SPARQL query can be defined as a tuple $q = (t_q, d, m, P_q)$ where $t_q \in T$ is the *query type*, $d \in I \cup B$ is an optional term identifying the RDF graph to be queried, m is a set of *solution modifiers* and P_q is a *graph pattern*.

Definition 3 (**Graph pattern**). Let $V = \{?v_0, ..., ?v_n\}$, where $n \in \mathbb{Z}^+$, be a set of variables, disjoint from I, B, and L. We will denote the set of variables in P as $vars(P)$. A triple pattern is a tuple $tp = (s', p', o')$ such that $s' \in I \cup V$, $p' \in I \cup V$ and $o' \in I \cup L \cup V$. A set of triple patterns is usually referred as a Basic Graph Pattern (BGP). A *property path pattern* pp can be inductively defined as an element of $(I \cup B \cup V) \times pp \times (I \cup B \cup L \cup V)$. Finally, a *graph pattern* is an expression specified by the following grammar:

$$\begin{aligned} P ::= \; & tp \mid pp \mid q \mid P_1 \,.\, P_2 \mid P \text{ FILTER } R \mid P_1 \text{ UNION } P_2 \mid P_1 \text{ OPTIONAL } P_2 \mid \\ & P_1 \text{ MINUS } P_2 \mid \text{Graph } i \; P \mid \text{Graph } v? \; P \end{aligned} \tag{2.1}$$

Here tp is a triple pattern, pp is a *property path pattern*, q is a subquery, $i \in I$, $?v \in V$ and R is a SPARQL filter constraint[3]. To remove ambiguities parentheses can be added to a pattern. Finally, we'll define the set of variables that occur in a pattern P as $vars(P)$ [72].

[3] For a detailed explanation of the syntax and semantics, we recommend the reader to consult section 17 of [116]

How do SPARQL queries work? An isomorphism of two graphs is an adjacency preserving bijection between their vertex sets [56].

The basis of SPARQL queries is finding matches with *graph patterns* in the RDF dataset [49, 20]. This problem is equivalent to the *subgraph isomorphism problem*, a generalization of the graph isomorphism problem: given two graphs G_1 and G_2, we try to find if G_1 contains a subgraph that is isomorphic to G_2.

In the context of a SPARQL query q, G_1 is a published RDF graph identified by d and G_2 corresponds to a *graph pattern* P_q. From this match we obtain a set of *bindings*: a multiset of mappings between $vars(P)$ and data within G_d. The final result of a query will also depend on t_q, the *query type*, and m, the specified *solution modifiers* (i.e. aggregation, grouping, sorting, duplicate removal).

Definition 4 (Semantics of SPARQL graph patterns). A *mapping* μ is a partial function $\mu : V \to I \cup B \cup L$ that assigns values to a finite set of variables. The domain of μ, denoted by $dom(\mu)$, is the subset of V where μ is defined.

Two *mappings* μ_1 and μ_2 are compatible, denoted by $\mu_1 \sim \mu_2$, if $\mu_1(?v) = \mu_2(?v)$ for all common variables $?v \in dom(\mu_1) \cap dom(\mu_2)$.

Abusing notation, we define $t = \mu(tp)$ as the triple obtained after replacing the variables in the *triple pattern* tp according to μ [51, 72]. Finally, let $\mu \vDash R$ represent a mapping that satisfies a filter condition R.

We define the *join*, *union*, *difference*, and *left outer join* operations on two sets of mappings Ω_1 and Ω_2 as follows:

$$\begin{aligned}
\Omega_1 \bowtie \Omega_2 &= \{\mu_1 \cup \mu_2 \mid \mu_1 \in \Omega_1,\ \mu_2 \in \Omega_2,\ \mu_1 \sim \mu_2\} \\
\Omega_1 \cup \Omega_2 &= \{\mu \mid \mu \in \Omega_1 \vee \mu \in \Omega_2\} \\
\Omega_1 - \Omega_2 &= \{\mu_1 \in \Omega_1 \mid \forall \mu_2 \in \Omega_2,\ \mu_1 \nsim \mu_2\} \\
\Omega_1 \leftouterjoin \Omega_2 &= (\Omega_1 \bowtie \Omega_2) \cup (\Omega_1 - \Omega_2)
\end{aligned} \tag{2.2}$$

The semantic evaluation of P over a dataset D is a set of mappings $[\![P]\!]_D$. Inductively, we define it as:

$$\begin{aligned}
&[\![tp]\!]_D = \{\mu \mid dom(\mu) = vars(tp) \wedge \mu(tp) \in G_D\} \\
&[\![pp]\!]_D = \{\mu \mid dom(\mu) = vars(pp) \wedge \mu(pp) \in G_D\} \\
&[\![q]\!]_D = \{\mu \mid dom(\mu) = vars(P_q) \wedge \mu(P_q) \in G_D\} \\
&[\![P_1 \,.\, P_2]\!]_D = [\![P_1]\!]_D \bowtie [\![P_2]\!]_D \\
&[\![P_1 \text{ UNION } P_2]\!]_D = [\![P_1]\!]_D \cup [\![P_2]\!]_D \\
&[\![P_1 \text{ OPTIONAL } P_2]\!]_D = [\![P_1]\!]_D \leftouterjoin [\![P_2]\!]_D \\
&[\![P_1 \text{ MINUS } P_2]\!]_D = [\![P_1]\!]_D - [\![P_2]\!]_D \\
&[\![P_1 \text{ FILTER } R]\!]_D = \{\mu \in [\![P]\!]_D \mid \mu \vDash R\} \\
&[\![\text{Graph } i\ P]\!]_D = \{\mu \in [\![P]\!]_{(\gamma(i),\gamma)} \mid i \in dom(\gamma)\} \\
&[\![\text{Graph } ?v\ P]\!]_D = \bigcup_{i \in dom(\gamma)} \{?v \to i\} \bowtie [\![P]\!]_{(\gamma(i),\gamma)}
\end{aligned} \tag{2.3}$$

Syntax SPARQL queries have a similar structure to the Structured Query Language (SQL) queries: we define the operation to execute using a keyword, followed by the projection of variables to be shown in the result, the keyword *WHERE*, the *graph pattern*,

enclosed in brackets, and, optionally, some modifiers of the result (i.e. *ORDER BY* to sort results and *DISTINCT* to remove duplicates). Additionally, we may also specify the RDF graph to consider for the query evaluation with the keywords *FROM* or *FROM NAMED*. Furthermore, the SPARQL syntax is very similar to TURTLE, so we can specify prefixes for namespaces. The basic *SPARQL 1.1 Query Language* [116] provides the following query types:

- *SELECT*: used to extract values from a RDF graph. The result is a set of bindings;
- *ASK*: returns a boolean, true if there was a match between the *graph pattern* and the RDF graph, false otherwise;
- *CONSTRUCT*: we specify two *graph patterns*, one to fetch *bindings* and another to use as a template for constructing new triples, from the values in the *bindings*. The final result is the set of newly generated triples.
- *DESCRIBE*: results depend on implementation [72], but usually returns a set of RDF triples describing the matching patterns.

In the following listings 2.3, 2.4 and 2.5 we show a few query examples.

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>

# Operation and variables
SELECT ?opt_course ?man_course ?popular_course
# Query Pattern
WHERE
{
    ?opt_course rdf:type fct:Optional .
    ?man_course rdf:type fct:Mandatory
}
```

Listing 2.3: SPARQL query with a conjuntion pattern

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>

# Operation and variables
SELECT DISTINCT ?popular_course
# Query Pattern
WHERE
{
    ?popular_course fct:hasStudentEnrolled ?num_students .
    FILTER (?num_students > 60)
}
# Optional Modifiers
LIMIT 20
```

Listing 2.4: SPARQL query with filters and modifiers

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix fct: <http://www.my-custom-domain/fct.ttl>
@prefix nova: <http://www.my-custom-domain/nova.ttl>

# Operation and variables
SELECT DISTINCT ?optional_course
# Query Pattern
WHERE
{
    {?optional_course rdf:type fct:Optional}
    UNION
    {?optional_course rdf:type nova:Elective}
}
```

Listing 2.5: SPARQL query with a disjunction pattern

Besides read operations, *SPARQL Update* [115], provides additional options for inserting, publishing and deleting data in triplestores:

- *INSERT/DELETE*: these operations add and delete, respectively, newly generated triples, based on a template, into a destination graph (if it does not exist it should be created). Like in a *CONSTRUCT* query we specify two *graph patterns*, one to fetch bindings and another to use as a template for constructing triples. We can join both operations, in this situation we specify three *graph patterns*, two to use as templates to generate the triples to be inserted or deleted and a third one to fetch bindings.
- *INSERT/DELETE DATA*: similar to their pattern-based variants, but in these operations we specify concrete data inline.
- *LOAD*: used to read the contents of a document that represents a graph and saves it into a destination graph in a triplestore.
- *CLEAR*: this operation removes all the triples in the specified graphs.

2.1.4.3 Preserving reasoning

In a *knowledge base* we can distinguish two sets of statements: the *TBox* and the *ABox*. The *TBox* set contains terminology data (e.g. ontologies) which we can define using RDFS or OWL2. The *ABox* set refers to assertions.

SPARQL was not designed for vocabularies like RDFS or OWL2. By default, a SPARQL engine solely considers explicit data, as if it were at the RDF layer. Query evaluations interpret all data as assertions, completely ignoring the reasoning capabilities of RDFS or OWL2. This results in answers that might be incomplete, not containing inferred statements (only derivable from the *ABox* set, using the *TBox* data).

Two popular techniques for preserving reasoning capabilities are *materialization* and *query rewriting*.

Materialization In this approach, we derive all the implicit information, from the original data, and persist it. This solution insures fast query evaluation that requires no inference, but suffers from high storage costs. Additionally, write operations are more complex, due to the problem of *truth maintenance*. After each update, we need to maintain the materialized data in a consistent state.

Insertions are fairly simple, we just need to compute new implicit triples from the data already saved in the system.

Deletions, however, are more complex. We need to compute the set of triples to be removed and assure that no triple in it is entailed without the removed assertions. We usually find three types of implementations for this operation: to always recompute the materialization after a deletion; removing the inferred triples and recomputing the closure of the remaining; maintaining a data-structure with the state of deletions. Neither of these options is perfect, the first two are not efficient for databases with high volume of write operations and the last one incurs in increased costs (associated with maintaining an additional structure).

Query Rewriting This technique consists in deriving a complete answer set at query time. Before evaluating a query, the system modifies it according to the statements in the *TBox*. Query evaluation is slower: rewriting the queries requires communication with a reasoner and the reformulated queries tend to be larger.

Contrary to the first solution, there is no problem with *truth maintenance*. Writes operations can be processed without any additional computations.

2.1.5 Industry solutions

2.1.5.1 Development Software & Tools

To developed and maintain ontologies, professionals use advanced semantic editors. *Protégé* [60] is a free, open-source, and the usual choice. It provides a rich ontology editing environment with full support for OWL2, available in a desktop version or on the web [94]. Many of its functionalities are based on plugins: automatic verification of errors and consistency in the design using reasoners like *Pellet* [84]; debugging tools; query answering; collaborative development [95]; graphic visualization of data and knowledge. Additionally, for developers, many Application Programming Interface (API)s [67] or full framework solutions, such as *Apache Jena* [91], are available for building *Semantic Web* applications.

2.1.5.2 Triplestores

To store RDF data, popular solutions are: *MarkLogic* [54], *Virtuoso* [66], *Amazon Neptune* [4], *GraphDB* [65], *Apache Jena - TDB* [92]. While *Apache Jena - TDB* is primarily a RDF triplestore, the rest are *multi-model* database systems.

To test their capability to handle SPARQL queries, the Berlin SPARQL Benchmark (BSBM) is a standard benchmark [16]. The Lehigh University Benchmark (LUBM) is another benchmark, which is tailored for OWL2 data [37]. It provides synthetic data generation for a single realistic ontology.

T[2]

2) *TODO:* Add queries in annex, explain evaluation metrics in experimental evaluation chapter

2.2 Secure Computations

In Section 1.1, we have contextualized the motivations behind the need for secure computation when sharing ontologies over cloud or other distributed networks.

In this setting we consider the *honest-but-curious*, or *passive*, adversary model. As such an adversary will follow the protocol properly but keep a record of all intermediate computations [35] in an attempt to learn private information while observing views of the protocol execution [27]. Security concerns should take into account the operation

history, data stored in the server and also the memory access pattern, as the adversary will have access to all of this information.

The *Encryption at Rest* approach encrypts all data stored in the untrusted server. To perform computations we need to download the full dataset, decrypt it, and only then can we execute operations on the data. This is too inflexible and computationally expensive.

ORAM [36] fits well this adversary model, as it allows for the concealment of memory access patterns by shuffling and continuously re-encrypting data on memory, but it is still impractical.

When used exclusively, Homomorphic Encryption [3, 31] schemes are not efficient enough to be feasible in real-world applications [10]. However, they are applied in conjunction with other security schemes, such as in some Searchable Encryption schemes [10, 15]. For this reason, in the following Sections we will discuss related work in the fields of HE, in Section 2.2.1, and SE, in Section 2.2.3.

2.2.1 Homomorphic Encryption

2.2.1.1 Overview

The term homomorphism, in security, was used for the first time by Rivest et al. in 1978 [78]. Homomorphic Encryption is a cryptographic method based on the concept of algebra homomorphism, where both decryption and encryption are seen as homomorphisms of the relations between the plaintext and the ciphertext [3, 31]. With HE, computations done over encrypted data, are preserved after decryption of the ciphertext.

Definition 5 (**Homomorphic Encryption**). An encryption scheme is homomorphic over operation $\bullet_1$ if it satisfies the following equation, where E is the encryption algorithm, $\bullet_1$ and $\bullet_2$ are arbitrary operations and M is the set of all possible messages [3, 32]:

$$E(m_1) \bullet_2 E(m_2) = E(m_1 \bullet_1 m_2), \forall m_1, m_2 \in M \tag{2.4}$$

2.2.1.2 Types of HE schemes

Some HE schemes preserve additive properties [68, 23, 22], others maintain multiplicative properties [79, 26], and some conserve both additive and multiplicative properties [34]. The HE schemes that preserve both properties allow for any arbitrary operation, because boolean circuits can be represented using only *XOR* (addition) and *AND* (multiplication) gates [3].

HE is classified in four main types:

- *Fully Homomorphic Encryption*: which supports any number of operations with no restriction on the number of operations;
- *Somewhat Homomorphic Encryption*: which permits some types of operations, a limited number of times;
- *Partially Homomorphic Encryption*: that allows for one type of operation with no restriction on the number of times;
- *Leveled-Fully Homomorphic Encryption* [11]: which supports any type of operations for a limited, and expected, number of L times (referred to as depth-L).

Definition 6 (**Additive Homomorphic Encryption**). An encryption scheme is as an additive homomorphic scheme if Equation 2.4 holds for the + operator. In this thesis, we will uniquely consider schemes that respect this condition, if $\bullet_1$ is an addition and $\bullet_2$ is a multiplication:

$$E(m_1) * E(m_2) = E(m_1 + m_2) \tag{2.5}$$

Multiplication with constants Multiplication by a constant $k > 0$ can be performed on an encrypted value, by raising it to the power of k. This is equivalent to performing $k-1$ multiplications:

$$E(m)^k = \underbrace{E(m) * ... * E(m)}_{k \text{ times}} = E(\underbrace{m + ... + m}_{k \text{ times}}) = E(k * m) \tag{2.6}$$

Considering that the plaintext space is $M \in \mathbb{Z}_n$ and that messages are small in comparison to the modulus n (so that reductions modulo n never occur), we can represent a negative constant c as $n-c$. Multiplication by a negative value is achieved by raising the encrypted value to the power of $n-c$ (an overflow of module n will occur)[32]:

$$\begin{aligned} E(m)^{(n-c)} &= E((n-c) * m \bmod n) \\ &= E(n * m - c * m \bmod n) \\ &= E(-c * m) \end{aligned} \tag{2.7}$$

Subtraction Given two messages m_1 and m_2, subtraction is computed by:

$$\begin{aligned} E(m_1 - m_2) &= E(m_1 + (-1) * m_2) \\ &= E(m_1) * E((-1) * m_2) \\ &= E(m_1) * E(m_2)^{-1} \end{aligned} \tag{2.8}$$

2.2.2 Property-preserving Encryption

2.2.2.1 Overview

Homomorphic Encryption allows arbitrary computations to be executed on encrypted data while not leaking any information about the original plaintext. For operations such as equality joins, where we require equality checks, we do need to preserve (leak) certain properties from the original plaintexts. This type of encryption is known as property-preserving encryption.

2.2.2.2 Deterministic Encryption

Deterministic encryption is type of property-preserving encryption scheme. Contrary to a probabilistic scheme, it will always generate the same ciphertext for each plaintext. This allows equality checks to be performed on the encrypted data. Naturally, its security guaranties are weakened. Deterministic ciphertexts are vulnerable to statistical analysis attacks.

2.2.2.3 Order-Preserving Encryption

Order-Preserving Encryption (OPE) schemes preserve the order guaranties of the plaintext data. Naturally, the order relations between the ciphertext are leaked. Some implementations can leak as much as half of the data bits of the ciphertexts [75].

2.2.3 Searchable Encryption

2.2.3.1 Overview

Searchable Encryption uses classical encryption schemes to perform encrypted search queries. While the first approach implemented it via a searchable ciphertext [85], most modern schemes do it using encrypted searchable indexes. Searchable Encryption can be divided in SSE, when it is based on symmetric key cryptography, and PEKS, when public key algorithms are used. We will focus on index-based SE schemes, more specifically in index-based SSE ones, due to their higher efficiency compared to PEKS approaches.

2.2.3.2 General Index-Based Model

A general model, for the index-based SE schemes was defined in [10]. Given a database $DB = (M_1, ..., M_n)$ consisting in a set of n plaintext documents M, some data items (e.g, keywords in the documents) $W = (w_1, ..., w_m)$ are extracted from the documents and encrypted using a key K, using an encryption algorithm Enc. The documents may be encrypted too, generating a collection of ciphertexts C, usually with a different key K'. A client generates an encrypted index I, for the database DB, using a *BuildIndex* function. The client stores the following information in an *honest-but-curious* untrusted server:

$$\begin{aligned} I &= \text{BuildIndex}_K(DB = (M_1, ..., M_n), W = (w_1, ..., w_m)) \\ C &= Enc_{K'}(DB) \end{aligned} \tag{2.9}$$

To perform search queries, the client generates a trapdoor $T = Trapdoor_K(f(w_j))$, for a predicate f a keyword w_j. With trapdoor T, the server performs the search over I, using a $Search$ algorithm, by checking for an encrypted index $i = Search(I, T)$. If there is a keyword w that satisfies the predicate f, it returns to the client the corresponding ciphertexts associated with i or $\perp$ if no match was found. In figure 2.5 we demonstrate the interaction between client and server.

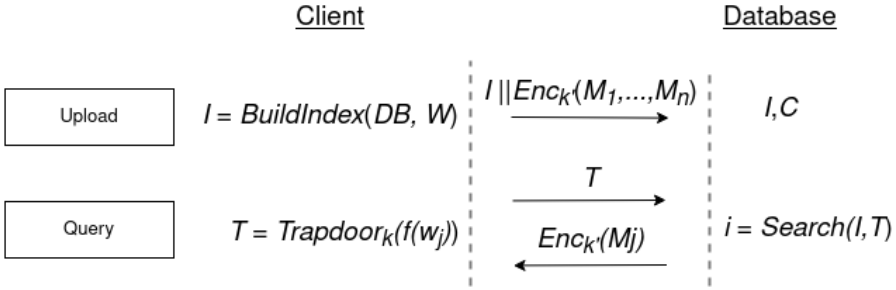

Figure 2.5: Interaction between client and server.

2.2.3.3 Client/Server Model

In SE schemes, one or various clients, the *writers* store encrypted data in a server. To issue a query request to the server, one or multiple clients, the *readers*, generate trapdoors.

There are four possible SE scheme architectures: Single Writer/Single Reader (S/S); Single Writer/Multiple Reader (S/M); Multiple Writer/Single Reader (M/S); Multiple Writer/-Multiple Reader (M/M).

The S/S architecture is usually used for *data outsourcing* and M/S, S/M and M/M employed in *data sharing* scenarios [10]. Symmetric key schemes are typically S/S, while public key schemes M/S (multiple users can write using a *public key*, but only the owner of the *private* key can generate trapdoors). Extension of the single reader schemes to multiple reader schemes can be done with *key sharing* or with the usage of *proxy re-encryption*.

In proxy re-encryption cryptography, a proxy is used to re-encrypt a ciphertext, encrypted using key K, and generates a new ciphertext, that is decrypted using key K', where $K \neq K'$ and the decrypted plaintext remains unchanged. Proxy re-encryption is usually employed with public key cryptography [8, 41, 24], but there are some schemes that support the symmetric key setting [90, 80, 71].

2.2.3.4 Information Leakage

Almost all SE techniques have privacy issues because they leak some information [10]. This leakage can be *index* information, the *search pattern*, or *access pattern* during execution:

- *Index Information*: refers to the information about the index keywords. It may include the number of keywords per document or database, the total of documents, document similarity, their length and metadata;
- *Search Pattern*: includes the information about keywords, and predicates, that is inferred by correlating equal results from different searches. The search pattern can be targeted using statistical analysis;
- *Access Pattern*: includes the information that may be derived from the query results, based on the memory accesses needed to retrieve the documents. (e.g. one can infer whether a query is more restrictive than another, based on the total of documents retrieved; or if some files are more accessed than others).

2.2.4 Symmetric Searchable Encryption

2.2.4.1 Overview

SSE is a type of SE based on symmetric key cryptography. We will start by specifying the security definitions used to evaluate such schemes and then describe a few reference schemes.

2.2.4.2 Security Definitions

Indistinguishability Against Chosen Plaintext Attacks (IND-CPA) is a property commonly used to evaluate encryption schemes, particularly in the context of public-key or symmetric-key cryptography. Informally, a scheme is IND-CPA secure if an adversary A, given two arbitrary plaintext messages and their respective ciphertexts, cannot distinguish which ciphertext corresponds to each message with a probability higher than that of random chance. This probability upholds even if A can adaptively query the encryption oracle.

This definition is not appropriate for SE schemes where the leakage of information originates from trapdoors and queries.

In 2006, Curtmola et al. [21] revisited the security definitions for SSE schemes, contributing with two new definitions, which are still the standard in use: Indistinguishability Against Non-Adaptive Chosen Keyword Attack (IND-CKA1) and Indistinguishability Against Adaptive Chosen Keyword Attack (IND-CKA2).

Both require that only the outcome of search patterns may be leaked. Additionally, they guarantee that trapdoors do not leak information about the keywords, apart from what can be inferred from the search and access pattern. Their main difference is whether the adversary can produce an adaptive or non-adaptive attack.

- *IND-CKA1*: guarantees security if the client generates all queries at once;
- *IND-CKA2*: guarantees security even if the client chooses the queries based on previously obtained trapdoors and search outcomes.

2.2.4.3 SSE Schemes

Song et al. In 2000, the first practical SSE scheme was proposed [86]. The general basis of their method is to encrypt each word of a document and embed hash values with a specific format in the ciphertext. During search, the server executes a *sequential scan* of the document, extracts the hashes and checks for matches on their values.

First, the plaintext is divided into *fixed-size* blocks w_i and each one is encrypted with a *deterministic*[4] encryption algorithm Enc, using key k. The ciphertexts $X_i = Enc_k(w_i)$ are split into two parts $X_i = \langle l_i, r_i \rangle$. Using a *stream cipher*[5] as a Pseudo-Random Function (PRF), a pseudo-random string of bytes $S_i = PRF_{k'}(i)$ is generated from the index position using key $k' \neq k$. Then, a hash $Y_i = \langle S_i, F_{k_i}(S_i) \rangle$ of this string of bytes is computed using a keyed hash function F_{k_i}, where the key is the output of a pseudo-random function $k_i = PRF_{k''}(l_i)$ that takes l_i as an input, with key $k''' \neq k''$. Finally, we compute the *XOR* of X_i with S_i to generate the final ciphertexts $C_i = X_i \oplus Y_i$.

To search, we generate a trapdoor $T = \langle X, K_s \rangle$, where $X = E_k(w) = \langle L, R \rangle$ is the encryption of keyword w and $K_s = PRF_{k''}(L)$. A server searches for the keyword in a document by iterating over all ciphertexts C_i, computing $C_i \oplus X$ and checking if the output is of the form $\langle S, PRF_{K_s}(S) \rangle$ for some S.

A security analysis of the scheme reveals that it leaks keyword position in the document. It is also susceptible to statistical attacks (e.g. after various searches, the words in the document can be inferred).

Curtmola et al. In 2006, two new schemes, for non-adaptive and adaptive adversaries, where proposed [21]. Each one is, respectively, at least IND-CKA1 and IND-CKA2 secure. The idea behind these was to use an *inverted index*, which is an index per distinct word in the database, not per document.

Non-Adaptive Scheme Considering a database with n documents, we define two data structures: an array A, where each position of the array contains a node $n_{i,j}$ which belongs to the linked list L_i associated to the keyword w_i; a look-up table T to identify the first node of a linked list associated with a keyword.

To build A, we start by constructing the nodes. These nodes are encrypted, stored in random order (using permutations) and augmented with information do find and decrypt the next node in the list. The augmentation step happens before the encryption

[4] Deterministic encryption leaks message equality.

[5] For information on stream ciphers we recommend reading chapter 8.4 of [70].

of the node. Each node $n_{i,j} = \langle id_m, k_{next}, p_{next} \| \varnothing \rangle$, includes information about the id of document m ($0 \leq m \leq n$) which contains the keyword, a randomly generated key used to encrypt the next node and a pointer to the next node in the list, or $\varnothing$ if it is the last one. The randomized disposition of the nodes, hides the length of the lists.

The look-up table I associates each keyword w_i with the node $n_{i,0} = \langle k_{i,1}, p_{i,1} \rangle \oplus PRF_k(w_i)$ stored in a random position using a Pseudo-Random Permutation (PRP), $PRP_{k'}(w_i)$, where $k \neq k'$. This node contains the encryption key and the pointer to the first node of the linked list L_i.

To search, we generate trapdoor $T_w = \langle PRF_k(w), PRP_{k'}(w) \rangle$. The server uses the trapdoor to find the correct node $n_{i,0} = T[PRP_{k'}(w)]$ and then retrieve all the identifiers of the documents that contain the keyword.

Adaptive Scheme Considering a database with n documents, we now define only a look-up table I, but instead of generating the index for a single keyword w, we generate it for the family of the keyword. The family of a keyword is defined as a set of labels $F_w = \{w | str(j) : 1 \leq j \leq t\}$ where j is an integer converted to string, using str, t is the number of identifiers of documents containing w, and the label is the result of concatenating w with j. The look-up table associates each label with the identifier of a document containing the w, in a random position, using $PRP_{k'}(w_i | str(j))$. To hide differences in the number of labels per keyword, extra entries are padded.

We generate the trapdoor $T_w = (t_1, \cdots, tn) = (PRP_k(w \mid str(1)), \cdots, PRP_k(w \mid str(n)))$ to search. The server uses the trapdoor values to find valid index positions in I and outputs the set of identifiers.

Generalization to the Dynamic Setting The previous schemes where considered static, as they do not allow modifications on the database after setting up the scheme. In 2012, Kamara et al. [44] proposed a *dynamic* extension to the *non-adaptive* scheme in [21]. They used a *deletion* array to track the positions on the search array that needed to be modified, after an update. They encrypted the array pointers, using homomorphic encryption, to support modifications on their values without decryption.

Generalization to Structured Data In 2010, Chase and Kamara [15] proposed a new adaptively secure scheme based on the *non-adaptive* scheme in [21]. This new scheme supports arbitrarily-structured data. Structured Symmetric Encryption is based on generating an inverted index in the form of a padded and permuted dictionary. They associated data items with metadata, allowing information about structure to be preserved if needed.

Generalization to Graph-Structured Data After STE, additional research has been conducted to support operations on graph-structured data.

Some authors extended STE to support queries on conceptual graphs [73], but they need to compute all possible query answers *a priori*.

Others, proposed a SSE scheme that supports *subgraph isomorphism queries* [12]. In their approach, they extract features from the graphs and generate the encrypted-indexes from the extracted features. This scheme was planned for generic graphs.

Finally, in [88] they combine an index-based SSE with a bucketization algorithm. By doing so, queries do not leak information about the structure of the graph. Although the authors state that their approach works with any kind of query, they only focused on neighbor and adjacency ones.

2.3 Secure Semantic Data Solutions

One of the simplest works in the field focuses solely on protecting individuals in OWL2 ontologies [62], uniquely encrypting their values and leaving the rest of the data in plaintext. This approach aimed at retaining functionality but leaks too much information. The following solutions try to achieve more robust security guaranties.

2.3.1 Self-Enforcing Access Control for Encrypted RDF

This work researched the implementation of access control in RDF datasets [28] through the use of functional encryption [9].

Encryption Scheme In the proposed encryption scheme, functions correspond to the computation of inner-products over $\mathbb{Z}_N$. The set of possible ciphertext attributes of length n is $\sum = \mathbb{Z}_N^n$ and the class of decryption key functions is $F = \{f_{\overrightarrow{x}} \mid \overrightarrow{x} \in \mathbb{Z}_N^n\}$. An attribute vector $\overrightarrow{y} \in F$ is associated with each ciphertext. A decryption key is a vector $\overrightarrow{x}$ such that $f_{\overrightarrow{x}}(y) = 1 \;\; iff \;\; \sum_{i=1}^{n} y_i x_i = 0$

To encrypt an RDF triple $t_i = (s_i, p_i, o_i)$ we functionally encrypt a random seed m_{t_i}, with an attribute vector $\overrightarrow{y}_{t_i} = (y_{s_i}, y'_{s_i}, y_{p_i}, y'_{p_i}, y_{o_i}, y'_{o_i})$ such that $\overrightarrow{y}_l = -r \cdot \sigma(l),\ y'_l := r,\ l \in \{s, p, o\}$ with σ being a function that maps a triple's subject, predicate and object to a value in $\mathbb{Z}_N$, and r a random value also in $\mathbb{Z}_N$

Advanced Encryption Standard (AES)[6] encryption is used to encrypt the plaintext triple using a key derivable from the generated seed m_{t_i}. The final ciphertext $c_{t_i} = \left\langle AES(t_i), FE(m_{t_i}) \right\rangle$ corresponds to both the AES ciphertext and the ciphertext of the seed.

The decryption keys are generated taking into account the possible triple patterns $tp = (s, p, o)$ combinations. For each triple pattern we define $\overrightarrow{x}_{tp} = (x_s, x'_s, x_p, x'_p, x_o, x'_o)$ such that $x_l := 1$, $x'_l := \sigma(l)$ if l is bound and $x_l := 0$, $x'_l := 0$ otherwise.

Finally, to decrypt a triple $t = (s_t, p_t, o_t)$, given a triple pattern $tp = (s_{tp}, p_{tp}, o_{tp})$ we compute the inner product $\overrightarrow{y}_t \cdot \overrightarrow{x}_{tp} = y_{s_t} x_{s_{tp}} + y'_{s_t} x'_{s_{tp}} + y_{p_t} x_{p_{tp}} + y'_{p_t} x'_{p_{tp}} + y_{o_t} x_{o_{tp}} + y'_{o_t} x'_{o_{tp}}$

Only if the result of the inner product is 0 can we decrypt the seed, derive the correct AES key and decrypt the triple.

Data Structure The data is stored in a key-value structure where the keys are unique integer IDs and the values correspond to the encrypted triples. The IDs are indexed using two strategies.

The first strategy is a *3-Index* approach. We compute secure hashes $h(s)$, $h(p)$ and $h(o)$ for each s, p, o (unbound values correspond to $h(?)$). Then, we generate three index tables *SPO*, *POS* and *OSP* where we store all triple pattern combinations and associate each value to the corresponding ID.

The second strategy is *vertical partitioning*. Instead of indexing the permutations of subjects, predicates and objects. We create one table per predicate, indexing the subject, object pairs that are related via that predicate. Each ID is indexed by a ("so" | "os", s, o).

3) *TODO:* Add image with both index schemes

T[3]

2.3.2 CryptGraphDB

CryptGraphDB [2] is an adaptation of the *CryptDB* [76, 75, 74] system to graph-structured data. It is capable of answering cypher [63] queries over conceptual graphs. We will start

[6] For information on AES we recommend reading chapter 6 of [70].

by describing the *CryptDB* architecture, security guaranties, query execution pipeline and encryption schemes. We will then explain the adaptations employed by *CryptGraphDB*.

System Architecture *CryptDB* is a practical solution for processing SQL queries over encrypted relational datasets. This solution leverages onion-encryption to support various primitive operations necessary to answer a SQL query (equality checks, order comparisons, aggregates and joins).

The system is composed by an application server, a proxy and an unmodified Database Management Systems (DBMS). The proxy intercepts all queries, encrypts and decrypts all data, and changes query operators while preserving the semantics of the original query.

T[4]

4) *TODO:* Add image of Cryptdb system

Security Threats *CryptDB* addresses two security threats. The first threat, consists in a passive attacker (e.g. database administrator), which only observes confidential data in the DBMS server. Against this attacker *CryptDB* provides a few security guaranties: sensitive data is always encrypted; the DBMS server cannot generate results for queries whose primitives do not belong in the subset of supported operations; information leakage depends on the types of computations that will be executed over the data - if no relational predicate filtering is needed, nothing more than the data's length is leaked; on equality checks, it reveals the histogram of encrypted values in the respective columns; in order checks, the system reveals the order between the encrypted values in the column.

The second threat is an attacker that gains control over the proxy, application server and DBMS server. Against this adversary, *CryptDB* provides confidentiality of data belonging to users not logged in during the period of the compromise (until it has been fixed). The adversary can, however, delete data stored in the database.

Besides confidentiality, *CryptDB* does not ensure integrity, freshness, or completeness of results.

Query Execution Pipeline For executing a query issued by the application server, a sequence of steps are necessary: i) the proxy intercepts and rewrites the query, anonymizing the table and column names and encrypting each constant present with the adequate encryption scheme for the desired operation, using a master key; ii) the proxy checks whether it should provide keys to the DBMS server, so that adjustments can be done on the encryption layers of the targeted data, if the check is positive it issues an UPDATE query that triggers a custom function to adjust the encryption of the selected columns; iii) the encrypted query is forwarded to the DBMS server and is executed using plain SQL; iv) the encrypted results are sent to the proxy, which are then decrypted and forwarded to the application server.

Adjustable Encryption Schemes To avoid unnecessary leakage of information data is encrypted with different layers, like an onion. When operations need encryption schemes with lower security guaranties, the data encryption layers are pealed. For each layer, per onion, per column, the proxy uses the same encryption key, derived from a master key, MK. This reduces the leakage of relations between columns and avoids the complexity of having different keys for each row in a column. For table t, column c, onion o, and encryption layer l, the proxy would use the encryption key $K_{t,c,o,l} = PRP_{MK}(t, c, o, l)$.

We now present the different encryption schemes supported by *CryptDB*:

- *Random (RND)*: A probabilistic encryption scheme that provides the most security but does not allow any computations on the ciphertexts.

- *Deterministic (DET)* A deterministic encryption scheme which allows equality checks to be executed over ciphertexts.

- *Order-Preserving (OPE)* An encryption scheme that preserves order between the ciphertexts in a column. This allows the server to execute range queries.

- *Homomorphic encryption (HOM)* The Paillier cryptosystem [68] is used to probabilistically encrypt numeric values. This scheme allows additive operations to be executed with ciphertexts.

- *Join (equi-JOIN)* In *CryptDB* columns are encrypted with a different encryption keys. Essentially, joins can not be performed between columns until the encryption keys of the columns coincide. To support join operations between different columns a new primitive is introduced JOIN-ADJ. It can be interpreted as a *keyed random hash* with the property that the hashes can be adjusted to modify their key without access to the plaintext, using an adjustment key $\Delta K = k_{c_1}/k_{c_2}$. The adjustment of the columns is done only on one of the two (the first column in lexicographic order). The information leakage is similar to that of the DET layer.

- *OPE-JOIN CryptDB* does not provide a construct to dynamically modify the keys of columns encrypted with the OPE scheme. To support range joins, these columns have to be specified before, so that they are encrypted under the same key.

- *Word search (SEARCH)* This encryption scheme allows queries with the LIKE keyword. In *CryptDB* they used the scheme by Song et al. [85]. The information leakage is similar to that of the RND layer.

The following images provide, respectively, an overview of the different onions types in *CryptDB* and an example a query execution where layer adjustments are needed.

T[5] T[6]

5) *TODO:* Add query flow image

6) *TODO:* Add onion encryption image

CryptGraphDB constructs upon the approach of *CryptDB*, adapting the solution to be able to answer *cypher* queries [63]. The proxy is implemented to handle *cypher* queries instead of SQL queries, but keeps the same functionality. Furthermore, as the *cypher* language does not require the join operator, *CryptGraphDB* only maintains the RND, DET, OPE, HOM and SEARCH encryption schemes.

2.3.3 GOOSE

GOOSE [17] is a secure graph outsourcing framework capable of answering UCRPQ queries. The solution relies on secure multiparty computation techniques, encryption schemes and a standard SPARQL engine.

GOOSE was designed as a distributed system with 5 participants: a *data owner* (DO), who owns the outsourced graph G; a *user* (U), who submits graph queries against G and receives query answers; a *query translator* (QT); *answer translator* (AT); and a *SPARQL engine* (SE).

T[7]

7) *TODO:* Add protocol image

Preliminaries Let ENC_{A_B} or ENC_{B_A} denote AES-CBC encryption using a shared key between participants A and B.

Let the outsourced RDF graph be $G = (N, E)$ where N is a set of nodes and $E \in N \times \Sigma \times N$ is a set of directed edges between the nodes, labeled using the alphabet Σ. Let $\sigma_N : N \rightarrow \{0, \cdots, |N|-1\}$ and $\sigma_\Sigma : \Sigma \rightarrow \{0, \cdots, |\Sigma|-1\}$ be two random bijections, respectively, for the edges and nodes, and let σ^{-1} represent the inverse of σ. The hidden set of edges from E is defined as $\hat{E} = \{(\sigma_N(s), \sigma_\Sigma(p), \sigma_N(o)) \mid (s, p, o) \in E\}$.

Consider $\Sigma^+ = \{a, a^- \mid a \in \Sigma\}$, a^- denotes the inverse of the edge label a. Let $V = \{?v_0, ..., ?v_n\}$, with $n \in \mathbb{Z}^+$, be a set of variables. A *query rule* is an expression of the form $head \leftarrow body$ defined as $(\overline{?v}) \leftarrow (?v_0, r_0, ?v_1), \cdots, (?v_{n+1}, r_n, ?v_{n+2})$, where, for every $i \in \mathbb{Z}_0^+$, r_i is a regular expression over Σ^+ using $\{\cdot, +, *\}$ and $\overline{?v}$ is a vector composed by a subset of the variables from the *body*. The length of this vector is the *arity* of the rule.

A UCRPQ query q is finite and non-empty set of *query rules* with the same *arity*. The answer of a query q evaluated against a graph G, using standard SPARQL semantics, is denoted as $ans(G, q)$.

Graph outsourcing To outsource a graph, the *data owner* generates σ_V, σ_Σ and $\hat{E}$. It then sends $\text{ENC}_{DO\_QT}(\sigma_\Sigma)$ to the *query translator*, $\text{ENC}_{DO\_AT}(\sigma_V)$ to the *answer translator* and $\text{ENC}_{DO\_SE}(\hat{E})$ to the SPARQL engine.

Query Evaluation In GOOSE, query evaluation is a four steps process:

1. First, the *user* generates a query q and sends $\text{ENC}_{U\_QT}(q)$ to the *query translator*.

2. The *query translator* generates $\hat{q}$ by applying the σ_Σ function to all predicates in q. $\text{ENC}_{QT\_SE}(\hat{q})$ is sent to the *SPARQL engine*.

3. The *SPARQL engine* evaluates $\hat{q}$ over $\hat{G} = (\bigcup_{(\hat{s},\hat{p},\hat{o}) \in \hat{E}} \{\hat{s}, \hat{o}\}, \hat{E})$ obtaining the answer $ans(\hat{G}, \hat{q})$. It then sends $\text{ENC}_{SE\_AT}(ans(\hat{G}, \hat{q}))$ to the *answer translator*.

4. The *answer translator* deobfuscates $ans(\hat{G}, \hat{q})$ using ${\sigma_V}^{-1}$ and generates the final answer $ans(G, q)$. It then sends $\text{ENC}_{AT\_U}(ans(G, q))$ to the *user*.

Security Model GOOSE is designed to be secure against *honest-but-curious* adversaries. It is assumed that data and computations are outsourced to external servers. The solution is expected to have the following security properties, which have been formally proven by the authors:

1. No cloud node can learn G.

2. Let q be a query submitted by a user and $ans(G, q)$ the answer of the query on the graph G. No cloud node can learn simultaneously q and $ans(G, q)$.

3. An external observer cannot learn G, any query q nor answer $ans(G, q)$ by analyzing the network messages.

2.3.4 Discussion

Current solutions limitations In this section we covered a few solutions for secure outsourcing of semantic data.

The *CryptGraphDB* [2] was not designed to support the *SPARQL* query language and suffers from the same security leakage problems inherent to *CryptDB*. Some authors argue that the usage of deterministic encryption to support join operations leaks too much information (e.g. frequency information about the encrypted items) and is vulnerable to *leakage-abuse* attacks [43, 38, 82].

The solution from J. D. Fernández et al. [28] uses hashes as indexes and this leaks the structure of the RDF graph. Furthermore, it only supports search operations over simple *triple patterns*.

The GOOSE [17] solution can answer SPARQL queries with complex *graph patterns*, but the data obfuscation methods used are deterministic and still leak the topology of the RDF graphs and frequency information of the encrypted keywords.

Furthermore, none of these solutions offer reasoning capabilities, and it is not clear whether they support write operations besides a single initial upload of the data.

Orthogonal security research Like the previous solutions, the main focus of our thesis will be on data *confidentiality*. Nonetheless, we will briefly mention the research that is being done in other security fields regarding RDF graphs.

One of those fields is ensuring *integrity* and *authenticity* of RDF data. Various signage schemes [13, 96, 45, 46, 47, 89] and even one service-oriented solution [53] have been proposed.

The other field is *access control*. Most solutions try to embed the policies within the data, using annotations. This allows for high-granularity in the *access control*, which can be inferred, but requires conflict resolution [42, 1, 30, 69, 18, 48, 19].

2.4 Summary

In Section 2.1, we described the layered architecture of the *Semantic Web* and discussed the standard languages for describing its *data model* and *ontologies*. More specifically, we covered the semantics and syntax of RDF and explained the primitives of RDFS.

Additionally, we specified the structure of OWL2, focusing on the mechanism it provides to write ontologies. We also discussed the restriction imposed by *OWL DL* to remain decidable.

Furthermore, we explained SPARQL, providing mathematical definitions, defining its semantics, describing the syntax, enumerating a few examples of queries and reasoning preservation techniques.

We finished Section 2.1 by listing a few of the tools, software, storage solutions and evaluation benchmarks used in this field of research.

In Section 2.2, we discussed various solutions to execute secure computations in untrusted servers. We started by describing the *honest-but-curious*, or *passive*, adversary model and explained why *Encryption at Rest* and ORAM are not practical approaches.

We proceeded to cover Homomorphic Encryption schemes, informally mentioning the mathematical basis of HE and enumerating the type of schemes available, and property-preserving encryption schemes.

In Section 2.2.3, we explained the difference between SSE and PEKS. We then described the general model for index-based SE, as well as the *Client/Server* architecture of

SE systems. We continued by discussing how to extend S/S architectures, covered the types of privacy leakage.

In Section 2.2.4 we specified the standard security definitions in use, and gave an in depth explanation of reference SSE schemes. We finished the SSE Section by showing recent generalizations, focusing on dynamic solutions and schemes that support structured data.

In Section 2.3, we mentioned related research on the topic of secure semantic data sharing, discussing the trade-off of each approach. Additionally, we briefly covered parallel work in the fields of data *integrity*, *authenticity* and *access control*.

3

SEnTri: A Searchable Encrypted Triplestore

In this chapter we will present the main contribution of our research. In Section 3.1, we introduce SEnTri and define the system and adversary model. In Section 3.2 we formally define the proposed solution, specify its enforcement of security guaranties and present the SPARQL query evaluation pipeline: triple pattern encryption schemes, high level execution pipelines and protocols, reasoning preservation techniques and list the supported query types.

3.1 Solution Models

3.1.1 Overview

The current *state-of-the-art* solutions solely focus on the problem of secure RDF data outsourcing, leaving the problem of dynamic updates out of scope. Furthermore, these systems do not have reasoning capabilities, suffer from leakage problems and, frequently, solely provide search over simple *triple patterns*.

Considering all these limitations we developed SEnTri, a solution for secure and cooperative development of semantic knowledge bases.

SEnTri possesses role-based *access control* and can execute secure read and write operations. T[8] It supports most SPARQL operations and retains reasoning capabilities while evaluating queries.

8) *TODO:* Do not forget to mention verification of authenticity and data integrity as future work

SEnTri is designed for a multi-user setting. It is purely a software solution, not requiring any special hardware. It remains *transparent* (working with encrypted ontologies does not affect a user's workflow in any significant way) and outsources storage and heavy computations to external servers.

3.1.2 System Model

Overview In SEnTri, authenticated *users* are able to create *encrypted triplestores*. Within a triplestore they are able to upload, modify and query semantic data. Triplestores can be shared between users with varying degrees of access.

SEnTri follows the usual SE schemes architecture. It is a *client/server* application which outsources data storage and heavy computations to third parties. It follows the M/M model with *key sharing* between trusted users. Figure T[9] depicts a high level overview of the system.

9) *TODO:* Add system model figure

System Architecture SEnTri is a multi-party system comprised of 5 components:

- *Client*: a public facing component where users can authenticate themselves and create, upload, share, request or revoke access to and query triplestores.
- *Triplestore*: a storage component where all encrypted data is saved;
- *Secure Computation Proxy*: this component executes secure query plans and set operations between encrypted *bindings*;
- *Identity and Access Management (IAM) Provider*: a component that handles authentication and validates access policies.
- *Vault*: a component to store confidential data required by the cryptographic protocols (e.g. cryptographic keys);

Each individual component is a web server that exposes a REST API. All communication channels between components are secure and reliable. They follow the HTTPS [40] protocol and TLS [93] data encryption with mutual authentication. Figure T[10] depicts a high level architecture of the solution.

10) *TODO:* Add system architecture figure

Role-Based Access Control In *SEnTri* roles grant different permissions to the users, which restrict the type of operations they can execute within the system. Users can have the *ADMIN*, *PRIVILEGED* and *BASIC* roles. For ease of communication from now onwards, we will refer to a user with each role, respectively, as *admin*, *privileged* and *basic* user.

Basic users can only query existing triplestores, if they have been given access to do so. They can request *read* or *write* access to existing triplestores.

Privileged users have all the permissions of a *basic* user. Additionally, they are able to create new triplestores, grant or revoke read/write access to their triplestores, transfer ownership to other users and process access requests targeting triplestores they own.

Admin users have all the permissions of a *privileged* user but can also manage users, any triplestore and process role change requests.

M/M setting In SEnTri access to triplestores is enforced through the use of *ephemeral access tokens*. These tokens are only obtainable by authenticated users that belong to the respective triplestore's read and write Access Control List (ACL).

These tokens ensure that the sharing of *secrets*, needed for the cryptographic protocols, and that the write/read operations are only done by users that have the required permissions to do so.

Correctness of concurrent write and read operations is achieved with a *lock-based* approach when issuing access tokens. *Read access tokens* can be issued at anytime, but only one *write access token*, per triplestore, can exist at any given moment. T[11]

11) *TODO:* Add example of access token emission & validation (IAM Provider, Vault, Triplestore)

3.1.3 Security Model

We will consider an *honest-but-curious* adversary. This adversary will try to eavesdrop communication between servers and, if it has access to the external servers where the semantic data and execution logs are stored, try to infer or decrypt sensitive information.

Following the architecture of SEnTri, this means that the *Triplestore* and *Proxy* components will be the targets of attacks. The other components and communication channels

are trusted, reliable and secure. We will also assume that authenticated users are trusted and non-malicious. Such users can read sensitive information and security parameters of the triplestores they have created or have been given access to.

When targeting these components, the attackers will try to decrypt data and infer sensitive information from the leakage of the protocols in use.

Additionally, we will not consider the case where attacks involve *revoked users*. [1]

3.2 Solution Definition

3.2.1 Overview

The functioning of a query evaluation system can be described as a pipeline of sequential operations. Firstly, if the system supports authentication, there is an authentication step. An authenticated user will be able to submit a series of queries while their session is active. A submitted query is parsed, and given the specified heuristics, the system generates a series of query execution plans, sequences of logical operations (e.g. data retrieval, update or execution of an equijoin). After a query plan is chosen and executed, the final results are sent to the user, with the applied solution modifiers. The following figure depicts this pipeline T[12] .

12) *TODO:* Add figure with standard query evaluation pipeline

SEnTri follows a similar logic for query evaluation with a few modifications to enhance security. The system only works with encrypted data, which is encrypted in a manner that allows for search and set operations to be executed. Additionally, before each step of the pipeline, access checks are performed against the users' ephemeral access tokens, to ensure that only the right actors are performing actions within the system. Figure shows the modified pipeline T[13] .

13) *TODO:* Add figure with SEnTri query evaluation pipeline

In SEnTri we employ *SSE* techniques during query evaluation: we generate trapdoors to search encrypted subjects, predicates and objects present in *triple patterns*. With these, we build *binding tables*, which are used for the evaluation of complex *graph patterns*

We have implemented SEnTri using three different approaches:

- *SEnTri V0*: Uses the same query evaluation techniques as the other two, but operates over plaintext data.
- *SEnTri V1*: In this implementation, data encryption retains elements from *CryptDB* [76, 75, 74]. Subjects, predicates and objects are encrypted in two layers. The outer layer is probabilistically encrypted, whereas the second is deterministically encrypted. When equality checks are required, we peel the outer layer and compare the values.
- *SEnTri V2*: In this last implementation, we aim to reduce the leakage produced during evaluation of queries, by employing *additive blinding* techniques[2] and using additive homomorphic encryption schemes. The value of a subject, predicate or object is mapped to an *equality tags*. These values are homomorphically encrypted and support additive blinding. Trapdoors index the *equality tags* and equality tags

[1] The role-based access controls prevent these users from interacting (via client) with triplestores they do not have permissions to. Nevertheless, we have not implemented a key rotation, or re-encryption protocol for data. As such, an adversary collaborating with these users, or an attack from such users, will always succeed in the decryption the data of all the triplestores they used to have access to. They can still possess the cryptographic secrets used by the protocols, which makes it trivial to decrypt leaked information.

[2] *Additive blinding* is a technique where we sum a random value to the ciphertext.

reference the probabilistically encrypted subjects, predicates and objects. The *binding tables* are built with the *blinded equality tags*. When doing equality checks, we decrypt the *blinded equality tags*. These remain deterministic, but different for every query evaluation.

In the remainder of this section we will: 1) begin by defining the core data elements and cryptographic tools; 2) specify the SPARQL query evaluation protocol; 3) define the triple pattern SSE schemes, and the adaptations done to SPARQL query evaluation protocol in order for it to work with encrypted data; 3) explain how we can expand the SPARQL query evaluation protocols to the M/M setting; 5) define the various implementations of SEnTri; 5) provide a mapping between the protocols and the system components; 6) define the security properties of SEnTri and analize the security and performance trade-offs of the various implementations.

3.2.2 Core elements

Triplestores A RDF dataset within SEnTri is represented as a single named triplestore. A triplestore $T = (triplestoreID,\ nodes,\ acp)$ is a key value store, where $triplestoreID$ is a unique string that identifies the dataset, $nodes = \{(idx_0, node_0), ..., (idx_n, node_n)\}$ is a map of predicate, subject or object *nodes*, $node \in I \cup B \cup L$, and each node is uniquely identified by its pattern index, idx, which is a hash of the *triple pattern*. The acp is the triplestore's Access Control Policy (ACP).

This data structure is a simplified version of the one defined in section 2.1.4, where we only represent the *default* graph.

Triplestore Access Control Policy A triplestore's ACP $acp = (r,\ w,\ acrl)$, is a tuple where r and w are both collections of usernames and $acrl$ is a collection of *access change requests*. An access change request $accessRequest = (requestID,\ username,\ write)$ is a tuple where $requestID$ is a Universally Unique Identifier (UUID) (version 4) [50], $username$ identifies the user that this request affects and $write$ is a boolean, that specifies if the user desires write access (true) or read access (false).

Users A user $u = (username,\ password,\ owned,\ role)$ is a tuple, where $username$ and $password$ are unique strings, $owned = [triplestoreID_0, ..., triplestoreID_n]$ is a collection of user's owned triplestores identifiers and $role \in$ {BASIC, PRIVILEGED, ADMIN}.

Role Change Request A role change request $roleRequest = (requestID, username, role)$ is a tuple where $requestID$ is a UUID (version 4), $username$ identifies the user that this request affects and $role \in$ {BASIC, PRIVILEGED, ADMIN} specifies the desired new role.

Sessions A user's session $s = (sessionID,\ username,\ ttl)$ is a tuple, where $sessionID$ is a UUID (version 4), $username$ belongs to a valid registered $user$ and $ttl \in N$ specifies the maximum time (in seconds) that a session can remain active, before expiring.

Locks A lock is $l = (lockID,\ resourceID,\ ttl)$ is a tuple, where $lockID$ is a UUID (version 4), $resourceID$ is either a valid $username$ or $triplestoreID$ and $ttl \in N$ specifies the maximum time (in seconds) that a lock can remain valid, before expiring.

Ephemeral Access tokens An ephemeral access token $at = (tokenID,\ ttl,\ username,\ sessionID,\ triplestoreID,\ lockID)$ is a tuple, where $tokenID$ is a UUID (version 4), $ttl \in N$ specifies the remaining time (in seconds) that the token will stay active, $username$ is the identifier of the user who holds the token, $sessionID$ is the identifier of the token holder's active session, at the time of its creation, $triplestoreID$ is the identifier of the triplestore whose access is being requested and $lockID$ can either be $\perp$ (for *read* access requests) or the identifier of the current lock, against the targeted triplestore (for *write* access requests).

SPARQL queries The definition of a SPARQL query within SEnTri is equal to that provided in section 2.1.4.

Query Execution Plans A query execution plan is a tuple $plan = (jobs,\ execOrder)$, where $jobs = \{(jobID_0,\ job_0),...,(jobID_n,\ job_n)\}$ is a map that maps a query job to their respective identifiers, which are UUIDs (version 4), and $execOrder$ is a sequence of job identifiers.

Query Jobs A query job is an internal operation that is executed during the evaluation of a SPARQL query. Query jobs can be of three types: *basic*, *unary* and *binary* jobs. Unary and binary jobs represent operations that are executed against the *bindings* of, respectively, one and two other jobs, whereas basic jobs are independent of other jobs' results. The execution of a job will always generate *bindings*, even if it is an empty collection. These *bindings* will be associated to the respective $jobID$.

There exist two basic jobs, job_{SEARCH} and job_{EMPTY}.

The search job is a tuple $job_{SEARCH} = (jobID,\ varPattern,\ s,\ p,\ o)$, where $varPattern \in$ {S, P, O, SP, SO, PO, SPO} and s, p, o are $\perp$ or have the value of, respectively a subject, predicate or object of the *triple pattern* being searched.

The empty result job is a tuple $job_{EMPTY} = (jobID,\ vars)$, where $vars = [var_0,...,var_n]$ is a collection of variables.

There exist four unary jobs, which represent the operations that apply result modifiers. These jobs are tuples of type $job_I = (jobID,\ jobID_{prev})$, where $I \in$ {PROJECT, ORDERBY, SLICE, DISTINCT} and dictates which solution modifier will be applied to the *bindings* associated with $jobID_{prev}$.

Finally, there are four binary jobs, which represent set operations between *binding*. These are tuples of type $job_{II} = (jobID,\ jobID_l,\ jobID_r)$, where $II \in$ {JOIN, UNION, OPTIONAL, MINUS} and specifies which set operation will be executed between the *bindings* of $jobID_l$ and $jobID_r$.

***Bindings* tables** The output of a query job is a *bindings* table. This data structure is a tuple $bindingsTable = (bindings,\ patterns)$ where $bindings = \{(var_0,\ bindingsMap_0),...,(var_n,\ bindingsMap_n)\}$ and $patterns = \{(var_0,\ patternsMap_0),...,(var_n,\ patternsMap_n)\}$ are maps that map query variables, respectively, to *bindings* and *triple patterns* maps.

A *bindings* map, $bindingsMap = \{(binding_0,\ [pattternID_{0,0},...,patternID_{0,k}]),...,(binding_n,\ [pattternID_{0,0},...,patternID_{n,k}])\}$, is a map where *bindings* are mapped to a list of *triple pattern* identifiers, which are UUIDs (version 4).

A *triple patterns* map, $patternsMap = \{(pattternID_0,\ binding_0),...,(patternID_n,\ binding_n)\}$, is a map where *triple pattern* identifiers are mapped to *bindings*.

SPARQL Query Result A SPARQL query result is a tuple $sparqlResult = (bindings, distinct, ordered, sliced, sortConditions, limit, offset)$ where $bindings = [pattern_0,..., pattern_n]$ is a collection of *triple patterns*, $ordered$, $distinct$ and $sliced$ are boolean values, $limit$ and $offset$ are integers and $sortConditions = [(var_0, dir_0),...,(var_n, dir_n)]$ is a collection of variables and their respective sorting direction, which can either be 1 or −1. A positive dir represents an *ascending* sorting whereas a negative dir represents a *descending* sorting condition.

Let $n \in \mathbb{Z}^+$ be the number of variables in a SPARQL query and $k \in \mathbb{Z}_0^+$ be the number of triple patterns found after the evaluation of a query. A triple pattern $pattern_i = \{(var_0, binding_{i,0}),...,(var_n, binding_{i,n})\}$ is a map where each variable is mapped to a $binding$. A $binding$ can be a plaintext value or $\perp$ for the particular case of OPTIONAL queries.

SPARQL Final Query Result A SPARQL query result $sparqlResult$ is the result of evaluating a $WHERE$ clause. The final output of a query evaluation must respect the standards present in [**W3c2013**, 115]. This final query result, $sparqlFinalResult$, will vary depending on the query type:

- *SELECT*: it will be a query result in JSON format;
- *ASK*: it will be a boolean;
- *CONSTRUCT*: it will be a set of triples;
- *DESCRIBE*: it will be a frequency table of the query variables;
- *SPARQL UPDATE*: it will be either SUCCESS, or $\perp$ in case of failure;

3.2.2.1 SEnTri V1 and V2 specific

We will denote $E(m)$ as the encryption of m, $O(m)$ as the obfuscation of m, mapping it to a random value.

Encrypted Triplestores An encrypted triplestore $E(T) = (triplestoreID, E(nodes), acp)$ is a key value store, where $triplestoreID$ is a unique string that identifies the dataset, $E(nodes)$ is an encrypted map of predicate, subject or object *nodes* and the acp is the triplestore's ACP. $E(nodes)$ will be defined in Subsection 3.2.5

Secret Maps A secret map $smap = (triplestoreID, secrets)$ is a tuple where $triplestoreID$ is the identifier of an existing triplestore, $secrets = \{(key_0, secret_0),...,(key_n, secret_n)\}$ is a map and key_i is a unique string that identifies $secret_i$, the serialized value of a secret parameter.

Secure Query Jobs The only difference between jobs in SEnTri V1 and V2, to jobs in SEnTri V0, is that they operate over obfuscated variables and output encrypted *bindings* tables.

Only the *secure search job* is structurally different. It is a tuple $job_{SEC\_SEARCH} = (jobID, vars, searches)$, where $vars = [O(var_0),...,O(var_n)]$ is a collection of obfuscated variables. During the query preparation phase, $searches = \{(O(var_0), keyword_0),...,(O(var_n), keyword_n)\}$ is a map that maps variables to the keywords to be searched, whereas, in the query evaluation phase, $searches = \{(O(var_0), searchID_0),...,(O(var_n), searchID_n)\}$

is a map that maps variables to the identifiers of search results. These identifiers are UUIDs (version 4).

Randomized Query Execution Plans A randomized query execution plan is a tuple $plan_{rnd} = (jobs, rndExecOrder)$, where $jobs = \{(jobID_0, job_0), ..., (jobID_n, job_n)\}$ is a map that maps a query job to their respective identifiers, which are UUIDs (version 4), and $rndExecOrder$ is a randomly chosen sequence of job identifiers, from the subset of valid execution sequences.

Randomized Prepared Query Execution Plans After the query preparation phase, the *searches* maps, in the secure search jobs, have been updated to map obfuscated variables to search identifiers. A plan where all search jobs have been updated like this is said to be *prepared*.

Encrypted *Bindings* tables Similar to a *bindings* table, an encrypted *bindings* table is a tuple $E(BindingsTable) = (E(bindings), E(patterns))$ where $E(bindings) = \{(O(var_0), E(bindingsMap_0)), ..., (O(var_n), E(bindingsMap_n))\}$ and $E(patterns) = \{(O(var_0), E(patternsMap_0)), ..., (O(var_0), E(patternsMap_n))\}$ are maps that map *obfuscated* query variables, respectively, to encrypted *bindings* and *triple patterns* maps.

An encrypted *bindings* map, $E(bindingsMap) = \{(E(binding_0), [pattternID_{0,0}, ..., patternID_{0,k}]), ..., (E(binding_n), [pattternID_{0,0}, ..., patternID_{n,k}])\}$, is a map where encrypted *bindings* are mapped to a list of *triple pattern* identifiers, which are UUIDs (version 4).

An encrypted *triple patterns* map, $E(patternsMap) = \{(pattternID_0, E(binding_0)), ..., (patternID_n, E(binding_n))\}$, is a map where *triple pattern* identifiers are mapped to encrypted *bindings*.

We will define $E(binding)$ in Subsection 3.2.5 for both SEnTri V1 and SEnTri V2.

Encrypted SPARQL Query Result An encrypted SPARQL query result is a tuple $E(sparqlResult) = (E(bindings), distinct, ordered, sliced, sortConditions, limit, offset)$ where $E(bindings) = [E(pattern_0), ..., E(pattern_n)]$ is a collection of encrypted *triple patterns*, $ordered$, $distinct$ and $sliced$ are boolean values, $limit$ and $offset$ are integers and $sortConditions = [(var_0, dir_0), ..., (var_n, dir_n)]$ is a collection of variables and their respective sorting direction, which can either be 1 or -1. A positive dir represents an *ascending* sorting whereas a negative dir represents a *descending* sorting condition.

Let $n \in \mathbb{Z}^+$ be the number of variables (obfuscated) in a SPARQL query and $k \in \mathbb{Z}_0^+$ be the number of encrypted triple patterns found after the evaluation of a query. An encrypted triple pattern $E(pattern_i) = \{(O(var_0), E(binding_{i,0})), ..., (O(var_n), E(binding_{i,n}))\}$ is a map where each obfuscated variable is mapped to an encrypted $binding$. An encrypted $binding$ can either have a value or be $\perp$ for the particular case of OPTIONAL queries.

3.2.3 Cryptographic Tools

3.2.3.1 Notation

For the remainder of the document, the function $E(K, m, iv)$ denotes the encryption of message m, using the initialization vector iv and key K. The notation $E(data)$ will represent uniquely that $data$ is encrypted. The function $D(K, ct)$ denotes the decryption of ciphertext ct, using key K. The keys used as input will identify the encryption scheme

(e.g. K_A implies the usage of the symmetric encryption scheme A and $K_{B_{priv}}$ implies the usage of public key encryption scheme B).

3.2.3.2 DGK Encryption

SEnTri V2 can be implemented with any additive homomorphic encryption scheme, but we have chosen to use the Damgård, Geisler and Krøigaard (DGK) encryption scheme [23, 22, 32, 52]. Compared to other schemes (e.g. Paillier [68]) it produces smaller ciphertexts, and the reduced plaintext and ciphertext spaces result in large performance gains. It also permits us to employ more efficient equality checks, which is a very common operation when evaluating SPARQL queries.

Definition 7 (**DGK Encryption**)**.** Let k, t, and l be public parameters. Parameter k determines the size of a Rivest-Shamir-Adleman (RSA) modulus, l is the bit length of the messages to encrypt and t should be large enough so that exhaustive search is infeasible.

Key Generation Let p, q be two primes, generated such that $n = pq$ is a k-bit RSA modulus and that the u, v_p and v_q also exist and possess the following properties:

1. u is the smallest prime greater than $l + 2$
2. u is a divisor of both $p - 1$ and $q - 1$
3. v_p is t-bit prime numbers that divides $p - 1$
4. v_q is t-bit prime numbers that divides $q - 1$

Choose random values $g, h \in \mathbb{Z}_n^*$, so that:

1. the multiplicative order of h is $v_p v_q$
2. g is an element of order $u v_p v_q \in \mathbb{Z}_n^*$

The message space is $M = \mathbb{Z}_u$ and the ciphertext space is $C = \{g^m h^r \bmod n \mid m \in \mathbb{Z}_u,\ r \in \mathbb{Z}_{v_p v_q}\} \subseteq \mathbb{Z}_n^*$. The public and private keys are, respectively, $K_{DGK_{pub}} = (n,\ g,\ h,\ u)$ and $K_{DGK_{priv}} = (p,\ q,\ v_p,\ v_q)$.

Probabilistic Encryption An encryption ct of m is calculated as:

$$ct = g^m h^r \bmod n \tag{3.1}$$

The value $r \in \mathbb{Z}_{v_p v_q}$ is only possible to calculate if we have access to private key. Knowing the public key, we instead determine the value of r as a random $2.5t$-bit integer, so that h^r is indistinguishable from a uniformly random element from the subgroup generated by h.

Deterministic Encryption To compute a deterministic encryption of m we remove the randomization part, $h^r \bmod n$, from the calculations:

$$ct = g^m \bmod n \tag{3.2}$$

Re-encryption To re-encrypt a probabilistic encryption of m we sample a new random $2.5t$-bit integer r_2 and compute:

$$\begin{aligned} ct_2 &= ct \cdot h^{r_2} \bmod n \\ &= g^m h^r h^{r_2} \bmod n \\ &= g^m h^{r+r_2} \bmod n \end{aligned} \tag{3.3}$$

Decryption The decryption of a ciphertext ct is done by calculating:

$$\begin{aligned} c^{v_p v_q} \bmod n &= (g^m h^r)^{v_p v_q} \bmod n \\ &= (g^m)^{v_p v_q} (h^r)^{v_p v_q} \bmod n \\ &= (g^m)^{v_p v_q} * 1 \bmod n \qquad \text{because order of h is } v_p v_q \\ &= (g^{v_p v_q})^m \bmod n \end{aligned} \tag{3.4}$$

Due to the value of u being small, it is feasible to search for the calculated value in a look-up table with tuples of type $((g^{v_p v_q})^i \bmod n,\ i)$, for $0 < i < u$. Even without the look-up table, one can quickly determine if ct is an encryption of 0, because $D(E(0)) = 1$:

$$c^{v_p v_q} \bmod n = (g^{v_p v_q})^m \bmod n \overset{\text{m}=0}{=} (g^{v_p v_q})^0 \bmod n = 1 \bmod n = 1 \tag{3.5}$$

So far we have presented the decryption equations using all p, q, v_p, v_q values, but we can utilize a faster decryption method. This method only requires the computation of modulo p or q, because m is uniquely determined from $c^{v_p} \bmod p$ or $c^{v_q} \bmod q$. For the remaining of the document we will only be considering this decryption method.

Homomorphic property The DGK cryptosystem is homomorphic with respect to the addition operation (+). For $ct_1, ct_2 \in C$, there exists $m_1, m_2 \in M$ and $r_1, r_2 \in \mathbb{Z}_{v_p v_q}$, where $ct_1 = g^{m_1} h^{r_1} \bmod n$ and $ct_2 = g^{m_2} h^{r_2} \bmod n$ and Equation 2.5 holds:

$$\begin{aligned} E(m_1) * E(m_2) &= (g^{m_1} h^{r_1} \bmod n) * (g^{m_2} h^{r_2} \bmod n) \\ &= (g^{m_1} h^{r_1} * g^{m_2} h^{r_2}) \bmod n \\ &= g^{(m_1+m_2)} h^{(r_1+r_2)} \bmod n \\ &= E(m_1 + m_2) \end{aligned} \tag{3.6}$$

Additional operations Multiplication of an encrypted value by a positive constant k is supported, just like in Equation 2.6, by raising it to the power of k.

Multiplication by a negative constant c is done similarly to what is specified in Equation 2.7, but instead of considering a plaintext space $\mathbb{Z}_n$, we consider $M = \mathbb{Z}_u$. As such, this operation requires raising the encrypted value to the power of $u - c$:

$$E(m)^{(u-c)} = E(-c * m) \tag{3.7}$$

Given Equations 2.8 and 3.7, subtraction of two ciphertexts is achieved by computing:

$$E(m_1 - m_2) = E(m_1 + (-1) * m_2) = E(m_1) + E(m_2)^{(u-1)} \tag{3.8}$$

To perform an equality check, between two plaintext messages, m_1, m_2, we compute the difference of their respective ciphertexts, ct_1, ct_2, and check if the result is 1 ($m_1 = m_2 \Leftrightarrow m_1 - m_2 = 0$):

$$D(E(m_1) - E(m_2)) = D(E(0)) = 1 \tag{3.9}$$

3.2.4 Evaluation of SPARQL queries

Now that we have defined all the core data structures and the cryptographic tools, we will proceed to explain how we evaluate SPARQL queries. We will start by explaining the protocol for doing so over plaintext data, implemented in SEnTri V0. For simplicity, we will not take into consideration role based access control and permissions, as it will not alter in any way the protocol logic.

3.2.4.1 Storage of triple patterns

The first step when designing a solution that supports SPARQL, is to define how we will store *triple patterns*. The storage strategy must preserve semantics and ensure fast access during query evaluation.

To achieve this, when we index data, our goal is to preserve the relationships between subjects, predicates and objects (which might appear in queries' graph patterns). In the remainder of the document we will use *node* as a common term to denominate *subjects*, *predicates* and *objects*.

In SEnTri V0, we index a triple pattern, $t = (s, p, o)$ based on all of its possible variations within a SPARQL query. For example, to answer a query where we need to search for $(?v, p, o)$ (the subject is a variable) we need an index $p, o \leftarrow [s_0, ..., s_n]$. With this index, all subjects are referenced by the predicate and object pairs in the triplestore. Applying this strategy to other possible patterns, we can index all triple pattern variations with the following 6 indexes:

$$\begin{array}{ll}
(p,o) \leftarrow [s_0,...,s_n] & \text{answers } (?v,p,o) \\
(s,o) \leftarrow [p_0,...,p_n] & \text{answers } (s,?v,o) \\
(s,p) \leftarrow [o_0,...,o_n] & \text{answers } (?v,p,o) \\
s \leftarrow [(p,o)_0,...,(p,o)_n] & \text{answers } (s,?v_1,?v_2) \\
p \leftarrow [(s,o)_0,...,(s,o)_n] & \text{answers } (?v_1,p,?v_2) \\
o \leftarrow [(p,s)_0,...,(p,s)_n] & \text{answers } (?v_1,?v_2,o)
\end{array} \tag{3.10}$$

We do not index patterns with 3 variables, as a join of two complementary indexes (e.g. s with (p, o)) can answer this kind of queries. Nonetheless, because the encryption schemes of SEnTri V1 and V2 do not support this kind of triple pattern, we will not support this type of queries in V0 either.

For patterns with 2 variables, the index consists in the concatenation of the pattern type (e.g, "PO" for indexes referencing predicate and object pairs) with the known value (e.g. a subject). This index will reference a set of node pairs, whose values we concatenate together.

For patterns with a single variable, the index is the concatenation of the pattern type (e.g. "S" for indexes referencing subjects) with the concatenation of the other two known values (e.g. predicate and object). This index references a set of individual nodes (e.g. subjects).

In Listing 3.1, we define the logic for SEnTri V0 storage engine. We will assume that this engine executes write operations *atomically*: it either executes all operations, or none.

```
State:
    triplestores ← {}
    separator_I ← fetchConfig(separator_I) # A configurable parameter, must provide a value
        ↪ that does not belong to the charset used for nodes
    separator_II ← fetchConfig(separator_II) # Must be different than separator_I, and follow
        ↪ the same restrictions
    separator_III ← fetchConfig(separator_III) # Must be different than separator_I and separator_II
        ↪ , and follow the same restrictions

function save(triplestoreID, triples):
    foreach t in triples:
        saveTriple(t)
    return "SUCCESS"

function delete(triplestoreID):
    triplestores ← triplestores / { triplestores[triplestoreID] }
    return "SUCCESS"

function delete(triplestoreID, triples):
    foreach t in triples:
        deleteTriple(t)
    return "SUCCESS"

function saveTriple(triple = (s,p,o)):
    parsed_s ← parseNode(s)
    parsed_p ← parseNode(p)
    parsed_o ← parseNode(o)
    keyword_s ← parseKeyword(s)
    keyword_p ← parseKeyword(p)
    keyword_o ← parseKeyword(o)
    save(triplestoreID, "PO", keyword_s, parsed_p, parsed_o)
    save(triplestoreID, "SO", keyword_p, parsed_s, parsed_o)
    save(triplestoreID, "SP", keyword_o, parsed_s, parsed_p)
    save(triplestoreID, "S", keyword_p, keyword_o, parsed_s)
    save(triplestoreID, "P", keyword_s, keyword_o, parsed_p)
    save(triplestoreID, "O", keyword_s, keyword_p, parsed_o)

function deleteTriple(triple = (s,p,o)):
    parsed_s ← parseNode(s)
    parsed_p ← parseNode(p)
    parsed_o ← parseNode(o)
    keyword_s ← parseKeyword(s)
    keyword_p ← parseKeyword(p)
    keyword_o ← parseKeyword(o)
    delete(triplestoreID, "PO", keyword_s, parsed_p, parsed_o)
    delete(triplestoreID, "SO", keyword_p, parsed_s, parsed_o)
    delete(triplestoreID, "SP", keyword_o, parsed_s, parsed_p)
    delete(triplestoreID, "S", keyword_p, keyword_o, parsed_s)
    delete(triplestoreID, "P", keyword_s, keyword_o, parsed_p)
    delete(triplestoreID, "O", keyword_s, keyword_p, parsed_o)

function save(triplestoreID, varPattern, keyword, node_1, node_2):
    index ← generateIndex(varPattern, keyword)
    triplestores[triplestoreID][index] ← triplestores[triplestoreID][index] ∪ { node_1 ||
        ↪ separator_III || node_2 }

```

```
53  function save(triplestoreID, varPattern, keyword1, keyword2, node):
54      index ← generateIndex(varPattern, keyword1, keyword2)
55      triplestores[triplestoreID][index] ← triplestores[triplestoreID][index] ∪ { node }
56
57  function delete(triplestoreID, varPattern, keyword, node1, node2):
58      index ← generateIndex(varPattern, keyword)
59      triplestores[triplestoreID][index] ← triplestores[triplestoreID][index] / { node1 ||
            ↪ separatorIII || node2 }
60
61  function delete(triplestoreID, varPattern, keyword1, keyword2, node):
62      index ← generateIndex(varPattern, keyword1, keyword2)
63      triplestores[triplestoreID][index] ← triplestores[index] / { node }
64
65  function generateIndex(varPattern, keyword):
66      return separatorII || varPattern || separatorII || keyword
67
68  function generateIndex(varPattern, keyword1, keyword2):
69      return separatorII || varPattern || separatorII || node1 || separatorII || node2
70
71  function search(triplestoreID, varPattern, s, p, o):
72      switch(varPattern):
73          case "S": return fetchBindings(triplestoreID, generateIndex("S", p, o), s)
74          case "P": return fetchBindings(triplestoreID, generateIndex("P", s, o), p)
75          case "O": return fetchBindings(triplestoreID, generateIndex("P", s, p), o)
76          case "PO": return fetchBindings(triplestoreID, generateIndex("PO", s), p, o)
77          case "SO": return fetchBindings(triplestoreID, generateIndex("SO", p), s, o)
78          case "SP": return fetchBindings(triplestoreID, generateIndex("SP", o), s, p)
79
80  function fetchBindings(triplestoreID, index, var):
81      table ← createBindingsTable()
82      table.setVariables({var})
83      foreach node in triplestores[triplestoreID][index]:
84          table.add(generateID(), var, node)
85      return table
86
87  function fetchBindings(triplestoreID, index, var1, var2):
88      table ← createBindingsTable()
89      table.setVariables({var1, var2})
90      foreach nodePair in triplestores[triplestoreID][index]:
91          patternID ← generateID()
92          nodes = nodePair.split(separatorIII)
93          table.add(patternID, var1, nodes[0])
94          table.add(patternID, var2, nodes[1])
95      return table
96
97  function parseNode(n):
98      if n∈I:
99          return "S" || separatorI || n.value // IRI
100     elif n∈L:
101         return "L" || separatorI || n.value || separatorI || n.datatype
102     elif n∈B:
103         return "B" || separatorI || n.value // identifier of the blank node
104     return ⊥
105
106 function parseKeyword(n):
107     if n∈I:
108         return "S" || separatorI || n.value
109     elif n∈L:
110         return "L" || separatorI || n.value || separatorI || n.datatype
111     elif n∈B:
```

```
        return "BLANK"
    return ⊥

function generateNode(parsedValue):
    (prefix, value, datatype) ← parsedValue.split(separator_I);
    if prefix == "BLANK":
        return createBlankNode(value)
    elif prefix == "S":
        return createIRINode(value);
    else
        return createLiteralNode(value, datatype);
```

Listing 3.1: Storage Engine for SEnTri V0

Notice that a search operation always produces a *bindings table*. This data structure was designed to support the operations that need to be executed between intermediate collections of *bindings*, during query evaluation. In Listing 3.2, we show the underlying logic of a *bindings table*.

```
State:
    bindings ← {}
    patterns ← {}

function setVariables(vars):
    foreach var in vars:
        bindings[var] ← {}
        patterns[var] ← {}

function add(patternID, var, binding):
    addBinding(binding, var, patternID)
    addPattern(binding, var, patternID)

function addPattern(binding, var, patternID):
    varPatternIDs ← patterns[var]
    if varPatternIDs == ⊥:
        varPatternIDs ← {}
        varPatternIDs[patternID] ← binding
        patterns[var] ← varPatternIDs
    else:
        varPatternIDs[patternID] ← binding

function addBinding(binding, var, patternID):
    varBindings = binding[var]
    if varBindings == ⊥:
        varBindings ← {}
        savePatternID(varBindings, binding, patternID)
        binding[var] ← varBindings
    else
        varBindingPatternIDs = varBindings[binding]
        if varBindingPatternIDs == ⊥:
            savePatternID(varBindings, binding, patternID)
        else
            varBindingPatternIDs ← varBindingPatternIDs ∪ { patternID }

function savePatternID(varBindings, binding, patternID):
    varBindingPatternIDs ← {}
    varBindingPatternIDs ← varBindingPatternIDs ∪ { patternID }
    varBindings[binding] ← varBindingPatternIDs

```

```
41  function getVariables():
42      return bindings.getKeys()
43
44  function getBindings(var):
45      return bindings[var]
46
47  function getBindings():
48      return bindings
49
50  function getPatternIDs(var):
51      return patterns[var]
52
53  function getPatternIDs():
54      return patterns
55
56  function getPatterns():
57      patterns ← createLinkedList()
58      vars ← getVariables()
59      patternIDs ← {}
60      foreach var in vars:
61          patternIDs ← patternIDs ∪ patterns[var].getKeys()
62      foreach patternID in patternIDs:
63          pattern ← [#vars]
64          i ← 0
65          foreach var in vars:
66              binding ← patterns[var][patternID]
67              pattern[i] ← binding
68              if binding == ⊥:
69                  i++
70          if i < #vars:
71              patterns ← patterns ∪ { pattern }
72      return patterns
73
74  function project(vars):
75      bindings.retain(vars)
76      patterns.retain(vars)
77
78  function join(other):
79      vars_mutual ← copy(getVariables())
80      vars_mutual.retain(other.getVariables())
81      return join(vars_mutual, this, other, "INNER")
82
83  function leftOuterJoin(other):
84      vars_mutual ← copy(getVariables())
85      vars_mutual.retain(other.getVariables())
86      return join(vars_mutual, this, other, "LEFT")
87
88  function union(other):
89      vars_l ← getVariables()
90      vars_r ← other.getVariables()
91      vars ← copy(vars_l)
92      vars ← vars ∪ vars_r
93      table_union ← createBindingsTable()
94      table_union.setVariables(vars)
95      copyAllBindings(vars_l, this, table_union)
96      copyAllBindings(vars_r, other, table_union)
97      return table_union
98
99  function minus(other):
100     vars_mutual ← copy(getVariables())
```

```
    vars_mutual.retain(other.getVariables())
    diff ← {}
    table_minus ← createBindingsTable()
    table_minus.setVariables(vars)
    foreach var : vars_mutual:
        bindings_l ← getBindings(var)
        bindings_r ← other.getBindings(var)
        foreach (binding, patternIDs) in bindings_l:
            if bindings_r[binding] == ⊥:
                diff ← diff ∪ patternIDs
    varBindings_RND ← getBindings(sample(vars_mutual))
    foreach (binding, patternIDs) in varBindings_RND:
        foreach patternID in patternIDs:
            if patternID ∈ diff:
                copyBindings(patternID, patternID, vars_mutual, this, table_minus)
    return table_minus

function copyBindings(newPatternID, oldPatternID, vars, source, target)
    foreach var in vars
        binding ← source.getPatternIDs(var)[oldPatternID]
        if binding ≠ ⊥:
            target.add(newPatternID, var, binding)

function copyAllBindings(vars, source, target)
    foreach var in vars:
        varBindings ← source.getBindings(var)
        foreach (binding, patternIDs) in varBindings:
            foreach patternID in patternIDs:
                target.add(patternID, var, binding)

function join(vars_mutual, table_l, table_r, type)
    vars_l ← copy(table_l.getVariables())
    vars_r ← copy(table_r.getVariables())
    vars ← vars_l ∪ vars_r
    vars_l ← vars_l / vars_r
    vars_r ← vars_r / vars_l
    table_join ← createBindingsTable()
    table_join.setVariables(vars)
    var ← sample(vars_mutual)
    bindings_l = table_l.getBindings(var)
    bindings_r = table_r.getBindings(var)
    foreach (binding, patternIDs_l) in bindings_l:
        patternIDs_r = bindings_r[binding]
        if patternIDs_r ≠ ⊥ or type == "LEFT":
            joinPatterns(vars_mutual, table_l, vars_l, patternIDs_l, table_r, vars_r, patternIDs_r,
                ↪ table_join, type)
    return table_join

function joinPatterns(vars_mutual, table_l, vars_l, patternIDs_l,
                      table_r, vars_r, patternIDs_r, table_join, type)
    foreach patternID_l in patternIDs_l:
        match ← false
        if patternIDs_r ≠ ⊥:
            foreach patternID_r in patternIDs_r:
                if equalPatterns(vars_mutual, table_l, patternID_l, table_r, patternID_r):
                    match ← true
                    patternID = generateID()
                    copyBindings(patternID, patternID_l, vars_mutual, table_l, table_join)
                    copyBindings(patternID, patternID_l, vars_l, table_l, table_join)
                    copyBindings(patternID, patternID_r, vars_r, table_r, table_join)
```

```
160         if match == false  and type == "LEFT":
161             copyBindings(patternID_l, patternID_l, vars_mutual, table_l, table_join)
162             copyBindings(patternID_l, patternID_l, vars_l, table_l, table_join)
163
164 function equalPatterns(vars_mutual, table_l, patternID_l, table_r, patternID_r)
165     foreach var in vars_mutual:
166         binding_l = table_l.getPatternIDs(var)[patternID_l]
167         binding_r = table_r.getPatternIDs(var)[patternID_r]
168         if binding_l == ⊥ and binding_r ≠ ⊥
169             return false
170         elif binding_l ≠ ⊥ and binding_r == ⊥
171             return false
172         elif binding_l ≠ ⊥ and binding_l ≠ binding_r
173             return false
174     return true
```

Listing 3.2: Bindings Table in SEnTri V0

3.2.4.2 SPARQL Query Evaluation Protocol

Having defined how data is stored, we will now explain the protocol for evaluating SPARQL queries.

Definition 8 (**SPARQL Query Evaluation Protocol**)**.** Let SPARQL-EVAL be a *SPARQL* query evaluation protocol. It is a tuple SPARQL-EVAL = (*PlanQuery*, *EvaluateQuery*, *GenerateFinalResult*) of 3 algorithms:

- {*plan*, *construct*, *upload*, *delete*, ⊥} ← **PlanQuery**(**query**): takes as input a SPARQL query. If the query is well-formed, and all required operations are supported, outputs a query execution plan and three collections of triple patterns used while constructing the final query results: the *construct*, *upload* and *delete* collections. In case of any error, it outputs ⊥.

- {*sparqlResult*, ⊥} ← **EvaluateQuery**(**plan**, **triplestoreID**): takes as input a query execution plan and the identifier of the triplestore to be queried. In case of success, outputs the query results, ⊥ otherwise.

- {*sparqlFinalResult*, ⊥} ← **GenerateFinalResult**(**triplestoreID**, **queryType**, **variables**, **sparqlResult**, **construct**, **upload**, **delete**): takes as input a triplestore identifier, the query type and variables, the *sparqlResult* obtained from the evaluation of a WHERE clause, and three collections of triple patterns. The *construct* template will be used for generating the final result of a CONSTRUCT query, whereas the *upload* and *delete* templates will be used to compute the triples to, respectively, upload and delete in the specified triplestore. In case of success outputs a *sparqlFinalResult*, ⊥ otherwise.

SPARQL-EVAL is a distributed, two-party protocol: the query plan generation and final result generation are executed by a *client* component; whereas the query evaluation and data updates are executed by a *triplestore* component.

Query Plan Generation In SEnTri we created two abstractions to separate the logic for parsing a query from the logic for generating a query execution plan. Respectively, we denominate them as the *SPARQL query engine* and *query planner*. The query engine will

parse the query into processable operations. These operations will be consumed by the query planner, which will create a query execution plan.

The parsing of a SPARQL query is out of the scope of this thesis. We've based our solution in *Apache Jena* [91]. We will utilize the solution provided by the API to parse a query and decompose it into a collection of *operations* (their equivalent abstraction to our *query jobs*). We will not formally define these data structures, but we will assume that their implementation is correct and sufficient to generate valid query jobs. All of this logic is scoped to the query engine, and it is specified in Listing 3.3.

```
State:
    JENA_algebraGenerator ← JENA.createAlgebraGenerator(JENA_ARQ.getContext()) # the algebra
        ↪ generator compiles a query into operations

function getQueryPlan(planner, queryString):
    try:
        query = JENA_QueryFactory.create(queryString) # the query factory parses a string and
            ↪ generates a query
        planner.setQueryType(query.getType())
        if planner.getQueryType() == "CONSTRUCT":
            planner.setConstructTemplate(query.getConstructTemplate().getTriples())
        return planner.generatePlan(JENA_algebraGenerator.compile(query))
    catch QueryParseException: # the query factory throws an exception when parsing a
        ↪ string for an update query
        updateQuery = JENA_UpdateFactory.create(queryString) # the update factory parses a
            ↪ string and generates an update query
        return planner.generatePlan(updateQuery.getOperations().get(0), JENA_UpdateFactory)
```

Listing 3.3: SEnTri SPARQL Query Engine

One can argue that the collection of *Apache Jena* operations, created after parsing and compiling a query, are equivalent to a query execution plan: executing them by their precedence order, will generate a valid query evaluation. In Chapter 4, we will explain why these are not compatible with our solution, and why we have decided to implement our custom query planner. The functioning of a simplified SEnTri V0 query planner (without any reasoning capabilities) is shown in Listing 3.4.

```
State:
    operations_parsed ← {}
    upload_template ← {}
    delete_template ← {}
    construct_template ← {}
    queryType ← :bot
    plan ← (jobs = {}, execOrder = createLinkedList(), vars = {})
    variables ← {}

function getQueryType():
    return queryType

function setQueryType(type):
    queryType ← type

function setConstructTemplate(triples)
    construct_template ← triples

function getConstructTemplate:
    return construct_template
```

```
21
22 function getUploadTemplate:
23     return upload_template
24
25 function getDeleteTemplate:
26     return delete_template
27
28 function pushJob(job):
29     jobID ← job.getID()
30     plan.jobs[jobID] ← job
31     plan.execOrder.addLast(jobID)
32
33 function generatePlan(op)
34     JENA.OpWalker.walk(op, this) # the operation walker will call visit while there are
           ↪ operations which have not been visited, using a visitor (the query planner is
           ↪ a JENA_Visitor)
35     if #plan.vars == 0:
36         plan.vars ← variables
37     return plan
38
39 function generatePlan(op, JENA_algebraGenerator):
40     if op is UpdateDataInsert:
41         visitUpdate(op)
42     elif op is UpdateDataDelete:
43         visitUpdate(op)
44     elif op is UpdateModify:
45         visitUpdate(op, JENA_algebraGenerator)
46     elif op is UpdateDeleteWhere:
47         visitUpdate(op)
48     else:
49         throw NotImplementedException()
50     if #plan.vars == 0:
51         plan.vars ← variables
52     return plan
53
54 function visit(op)
55     if op is OpBGP:
56         generateSearchJobs(op)
57     elif op is OpTriple:
58         generateSearchJobs(op.asBGP())
59     elif op is OpJoin:
60         generateJoinJob(op)
61     elif op is OpLeftJoin:
62         generateOptionalJob(op)
63     elif op is OpMinus:
64         generateMinusJob(op)
65     elif op is OpUnion:
66         generateUnionJob(op)
67     elif op is OpModifier:
68         if op is OpDistinctReduced:
69             generateDistinctJob(op)
70         elif op is OpOrder:
71             generateOrderByJob(op)
72         elif op is OpProject:
73             generateProjectJob(op)
74         elif op is OpSlice:
75             generateSliceJob(op)
76         else
77             throw NotImplementedException()
78     else:
```

```
79          throw NotImplementedException()
80
81  function visitUpdate(op_UpdateDataInsert)
82      setQueryType("INSERT_DATA")
83      foreach quad = (graph, triple) in op_UpdateDataInsert.getQuads(): # we only support a single
            ↪ default graph, hence quads can be converted to triples
84          upload_template ← upload_template ∪ { triple }
85
86  function visitUpdate(op_UpdateDataDelete)
87      setQueryType("DELETE_DATA")
88      foreach quad = (graph, triple) in op_UpdateDataDelete.getQuads():
89          delete_template ← delete_template ∪ { triple }
90
91  function visitUpdate(op_UpdateDeleteWhere):
92      setQueryType("MODIFY")
93      bgp ← {}
94      foreach quad = (graph, triple) in op_UpdateDeleteWhere.getQuads():
95          bgp ← bgp ∪ {triple}
96          delete_template ← delete_template ∪ { triple }
97      JENA.OpWalker.walk(JENA.createOpBGP(bgp), this)
98
99  function visitUpdate(op_UpdateModify, JENA_algebraGenerator)
100     setQueryType("MODIFY")
101     if op_UpdateModify.hasInsertClause():
102         foreach quad = (graph, triple) in op_UpdateDataInsert.getInsertQuads():
103             upload_template ← upload_template ∪ { triple }
104     if op_UpdateModify.hasDeleteClause():
105         foreach quad = (graph, triple) in op_UpdateDeleteWhere.getDeleteQuads():
106             delete_template ← delete_template ∪ { triple }
107         JENA.OpWalker.walk(JENA_algebraGenerator.compile(op_UpdateModify.getWherePattern()), this)
108
109 function generateSearchJobs(op_OpBGP):
110     patterns = op_OpBGP.getPattern()
111     jobIDs = {}
112     jobs = {}
113     jobVars = {}
114     foreach t = (s,p,o) in patterns:
115         jobID = pushSearch(s, p, o, jobs, jobIDs)
116         job = jobs[jobID]
117         vars_job ← extractVars(job)
118         variables ← variables ∪ vars_job
119         jobVars[jobID] ← vars_job
120     if #patterns > 1:
121         generateRandomJoinPipeline(op_OpBGP, jobVars)
122     else:
123         operations_parsed.put(op_OpBGP, jobID)
124
125 function pushSearch(s, p, o, jobs, jobIDs):
126     searchPattern ← parsePattern(s, p, o)
127     jobID = jobIDs.get(searchPattern)
128     if jobID == :bot:
129         jobID ← generateID()
130         job_SEARCH ← createSearchJob(jobID, extractVariablesPattern(s, p, o), s, p, o)
131         jobs[jobID] ← job_SEARCH
132         jobIDs[searchPattern] ← jobID
133         pushJob(job_SEARCH)
134     return jobID
135
136 function generateRandomJoinPipeline(op_OpBGP, jobVars)
```

```
137     (graph, vars, jobs) ← generateAdjacencyMatrix(jobVars)
138     path ← createArrayList(numJobs)
139     sampled ← {}
140     stop ← false
141     numJobs ← #jobVars.getKeys()
142     while #sampled < numJobs and stop == false:
143         next ← sample(jobs / sampled)
144         sampled ← sampled ∪ { next }
145         bfsSearch(next, graph, path, numJobs)
146         if #path == numJobs:
147             stop ← true
148     if #path == numJobs:
149         joins = createLinkedList()
150         for i in range(0, #path):
151             jobID_1 = path.get(i)
152             jobID_2 = path.get(i + 1)
153             joinID = generateID()
154             path.get(i + 1) ← joinID
155             joins.add(createJoinJob(joinID, jobID_1, jobID_2))
156         foreach job_JOIN in join:
157             pushJob(job_JOIN)
158         operations_parsed[op_BGP] ← joins.getLast().getID()
159     else :
160         jobID = generateID()
161         pushJob(createEmptyResultJob(jobID, vars))
162         operations_parsed[op_BGP] ← jobID
163
164 function generateAdjacencyMatrix(jobVars)
165     graph ← {}
166     jobs ← {}
167     vars ← {}
168     foreach jobID_1 in jobVars.getKeys():
169         jobs ← jobs ∪ { jobID_1 }
170         v_1 ← jobVars[jobID_1]
171         vars ← vars ∪ v_1
172         edges ← {}
173         foreach jobID_2 in jobVars.getKeys():
174             v_2 ← jobVars[jobID_2]
175             if jobID_1 ≠ jobID_2
176                 foreach var \; in \; v_1:
177                     if v ∈ v_2:
178                         edges ← edges ∪ { JobID_2 }
179                         break
180         graph[jobID_1] ← edges
181     return (graph, jobs, vars)
182
183
184 function bfsSearch(root, graph, path, depth)
185     queue = createQueue()
186     path.add(root)
187     queue.push(root)
188     while #path < depth:
189         next ← queue.poll()
190         foreach neighbour in graph[next]:
191             if neighbour ∉ path:
192                 path.add(neighbour)
193                 queue.push(neighbour)
194         if #queue == 0:
195             return
196
```

```
function getJobID(op)
    jobID = operations_parsed[op]
    if jobID == ⊥:
        throw QueryBuildException()
    return jobID

function generateProjectJob(op_OpProject)
    jobID ← generateID()
    variables ← op.getVars()
    plan.vars ← variables
    pushJob(createProjectJob(jobID, getJobID(op.getSubOp()), variables))
    operations_parsed[op] ← jobID

function generateDistinctJob(op_OpDistinctReduced)
    jobID ← generateID()
    pushJob(createDistinctJob(jobID, getJobID(op.getSubOp())))
    operations_parsed[op] ← jobID

function generateOrderByJob(op_OpOrder)
    jobID ← generateID()
    pushJob(createOrderByJob(jobID, getJobID(op.getSubOp()), op.getConditions()))
    operations_parsed[op] ← jobID

function generateSliceJob(op_OpSlice)
    jobID ← generateID()
    pushJob(createSliceJob(jobID,getJobID(op.getSubOp()), op.getStart(), op.getLength()))
    operations_parsed[op] ← jobID

function generateJoinJob(op_OpJoin)
    jobID ← generateID()
    pushJob(createJoinJob(jobID, getJobID(op.getLeft()), getJobID(op.getRight())))
    operations_parsed[op] ← jobID

function generateOptionalJob(op_OpLeftJoin)
    jobID ← generateID()
    pushJob(new OptionalJob(jobID, getJobID(op.getLeft()), getJobID(op.getRight())))
    operations_parsed[op] ← jobID

function generateMinusJob(op_OpMinus)
    jobID ← generateID()
    pushJob(createMinusJob(jobID, getJobID(op.getLeft()), getJobID(op.getRight())))
    operations_parsed[op] ← jobID

function generateUnionJob(op_OpUnion)
    jobID ← generateID()
    pushJob(createUnionJob(jobID, getJobID(op.getLeft()), getJobID(op.getRight())))
    operations_parsed[op] ← jobID
```

Listing 3.4: SEnTri V0 SPARQL Query Planner (without reasoning capabilities)

Notice that the query planner generates a query execution plan where equi-joins are randomly ordered. The basis for this decision is to not incur in any bias, when evaluating performance, by always generating a deterministic execution order. Like the parsing of a *SPARQL* query, optimization of its execution plan is out of the scope of this thesis, hence this approach.

We generate a random ordering of the equi-joins, by randomly sampling one of the valid execution orders. BGPs are the only elements from a query that generate equi-joins. When we have visited all BGP operations, we create an auxiliary graph where compatible

equi-joins are neighbors. Two joins are *compatible* if they have, at least, 1 common variable. We generate the execution order of the equi-joins, by exploring the graph and computing a path that contains each job_{JOIN} once. To generate the random ordering, we randomly sample the origin of the path. After generating the execution order we add the query jobs to the execution plan.

Query Evaluation Following the dispatcher pattern, we have created two other abstractions in SEnTri: the *query execution*, and the *query worker*. A query execution dispatches work, and a query worker processes pieces of work.

In the specific case of our query evaluation protocol, the query execution will process the query execution plan and dispatch the next available query job, to the worker. The worker has access to the stored data and will execute the assigned query job. The currently SEnTri only supports sequential dispatching of jobs to a single worker.

The dispatching of jobs continuous until there are no more jobs pending execution. After the last job is executed, the query result is generated and stays available to be retrieved as output.

Listings 3.5 and 3.6 specify, respectively, the functioning of the query execution and query worker for SEnTri V0.

```
1  State:
2      pending ← {}
3      finished ← {}
4      results ← {}
5      current ← ⊥
6      sparqlResult ← ⊥
7
8  function exec(plan = (jobs, execOrder), worker):
9      pending ← execOrder
10     while not isFinished():
11         current = execOrder.peek()
12         results[current] ← delegateJob(worker, jobs[current])
13         finished ← finished ∪ { pending.poll() };
14     sparqlResult ← worker.generateSPARQLResult(results[current]);
15
16 function delegateJob(worker, job):
17     res ← results[job.jobID]
18     if res == ⊥:
19         if job is UNARY:
20             return worker.exec(job, results[job.jobID_prev])
21         elif job is BINARY:
22             return worker.exec(job, results[job.jobID_l], results[job.jobID_r])
23         else:
24             return worker.exec(job);
25     else
26         return res;
27
28 function getFinishedJobs():
29     return finished
30
31 function getPendingJobs():
32     return pending
33
34 function getCurrentJob():
35     return current
36
```

```
function isFinished():
    return #pending > 0

function isFinished(jobID):
    return results[jobID] ≠ ⊥

function getSPARQLResult():
    return sparqlResult
```

Listing 3.5: SEnTri V0 Query Execution

```
State:
    triplestoreID ← ⊥
    storageEngine ← ⊥
    sparqlResult ← ⊥

function create(target, storage):
    triplestoreID ← target
    storageEngine ← storage
    sparqlResult ← ({}, false, false, false, ⊥, ⊥)

function exec(job):
    if job is SEARCH:
        return execSearch(job)
    elif job is EMPTY:
        return createBindingsTable(job.vars)
    throw JobInstanceException()

function exec(job, prevJobResults):
    if job is PROJECT:
        return execProject(job, prevJobResults)
    elif job is ORDERBY:
        return execOrderBy(job, prevJobResults)
    elif job is DISTINCT:
        return execDistinct(prevJobResults)
    elif job is SLICE:
        return execSlice(job, prevJobResults)
    throw JobInstanceException()

function exec(job, left, right):
    if job is JOIN:
        return execJoin(left, right)
    elif job is UNION:
        return execUnion(left, right)
    elif job is OPTIONAL:
        return execOptional(left, right)
    elif job is MINUS:
        return execMinus(left, right)
    throw JobInstanceException()

private execSearch(job_SEARCH = (jobID, varPattern, s, p, o)):
    if varPattern ≠ "SPO"
        return storageEngine.search(triplestoreID, varPattern, s, p, o)
    else
        throw NotImplementedException()

function execProject(job_PROJECT = (jobID, vars), prevJobResults):
    prevJobResults.project(vars)
    return prevJobResults
```

```
49
50  function execOrderBy(job_ORDERBY = (jobID, jobID_prev, sortConditions), prevJobResults):
51      sparqlResult.ordered ← true
52      sparqlResult.sortConditions ← sortConditions
53      return prevJobResults
54
55  function execSlice(job_SLICE = (jobID, jobID_prev, offset, length), prevJobResults):
56      sparqlResult.sliced ← true
57      sparqlResult.length ← length
58      sparqlResult.offset ← offset
59      return prevJobResults
60
61  function execDistinct(prevJobResults):
62      sparqlResult.distinct ← true
63      return prevJobResults
64
65  function execJoin(left, right):
66      return left.join(right)
67
68  function execUnion(left, right):
69      return left.union(right)
70
71  function execOptional(left, right):
72      return left.leftOuterJoin(right)
73
74  function execMinus(left, right):
75      return left.minus(right)
76
77  function generateResults(jobResults):
78      bindings ← ⊥
79      if sparqlResult.ordered == true:
80          if sparqlResult.distinct == true:
81              bindings ← generateBindings(createTreeSet(sparqlResult.sortConditions), jobResults
                    ↪ )
82          else:
83              bindings ← generateBindings(createLinkedList(), jobResults)
84              sort(bindings, sortConditions)
85      else:
86          if sparqlResult.distinct == true:
87              bindings = generateBindings(createHashSet(), jobResults)
88          else
89              bindings = generateBindings(createLinkedList(), jobResults)
90      if sparqlResult.sliced == true:
91          bindings = slice(bindings, sparqlResult.offset, sparqlResult.length)
92      sparqlResult.bindigns ← bindings
93      return sparqlResult.bindigns
94
95  function generateBindings(collector, jobResults):
96      vars = jobResults.getVariables()
97      values ← {}
98      foreach patterns in jobResults.getPatterns():
99          binding = {}
100         i = 0
101         foreach parsedValue in patterns:
102             if parsedValue ≠ ⊥:
103                 binding[vars[i]] ← storageEngine.generateNode(parsedValue)
104             i++
105         collector.add(binding)
106     return collector
```

Listing 3.6: SEnTri V0 Query Worker

Generating Final Result & Applying modifications After the evaluation of a query's WHERE clause, or the generation of the *delete* and *update* templates, there is still one final phase to execute. During this phase we generate the correct final query answer, which matches the query type, and update the triplestore data, upon the presence of an update query. Listings 3.7 and 3.8 present, respectively, the logic of the SPARQL-EVAL *client* and *triplestore* components. These two components will serve as the base reference for the fully functional SEnTri components.

```
State:
    queryEngine ← createSPARQLQueryEngine()
    planner ← ⊥

function executeSPARQLQuery(triplestoreID, query):
    try:
        (planner, plan) ← planQuery(query)
        queryType ← planner.getQueryType()
        if queryType ∈ { SELECT, CONSTRUCT, ASK, DESCRIBE, MODIFY,
            ↪ DELETE_WHERE }:
            sparqlResult ← Triplestore.evaluateQuery(triplestoreID, plan)
        return generateFinalResult(triplestoreID, queryType, sparqlResult, construct, upload,
            ↪ delete)
    catch Exception:
        return ⊥

# SPARQL-EVAL.planQuery(query)
function planQuery(query):
    planner ← createSPARQLQueryPlanner()
    return (planner, queryEngine.getQueryPlan(planner, query))

# SPARQL-EVAL.generateFinalResult(triplestoreID, queryType, sparqlResult, construct, upload,
    ↪ delete):
function generateFinalResult(triplestoreID, queryType, variables, sparqlResult,
    ↪ construct_template, upload_template, delete_template)
    if queryType == "SELECT":
        return generateSELECTResults(variables, sparqlResult)
    elif queryType == "CONSTRUCT":
        return generateCONSTRUCTResults(construct_template, sparqlResult.bindings)
    elif queryType == "ASK":
        return generateASKResults(sparqlResult.bindings)
    elif queryType == "DESCRIBE":
        return generateDESCRIBEResults(variables, sparqlResult)
    else:
        return applyModifications(triplestoreID, sparqlResult.bindings, upload_template,
            ↪ delete_template)

function applyModifications(triplestoreID, bindings, upload_template, delete_template):
    triples_upload ← {}
    triples_delete ← {}
    if queryType == "MODIFY":
        triples_upload ← generateTriplesFromBindings(upload_template, bindings) #foreach binding,
            ↪ map values to variables and output the set of generated triples
    triples_delete ← generateTriplesFromBindings(delete_template, bindings) #foreach binding, map
        ↪ values to variables and output the set of generated triples
    else if (queryType == INSERT_DATA)
        triples_upload ← upload_template
    else if (queryType == DELETE_DATA)
        triples_delete ← delete_template
    try:
```

```
        Triplestore.deleteTriples(triplestoreID, triples_delete)
        Triplestore.saveTriples(triplestoreID, triple_upload)
    catch Exception:
        return ⊥
    return "SUCCESS"

function generateASKResults(bindings):
    return #bindings > 0

function generateCONSTRUCTResults(construct_template, bindings):
    return generateGraphFromBindings(construct_template, bindings) #foreach binding, map
        ↪ values to variables and output a graph with all the generated triples

function generateSELECTResults(variables, bindings):
    return generateJSONQueryResult(variables, bindings) # map bindings collection to a well-
        ↪ formed JSON, as per the official specification

private Response generateDESCRIBEResults(variables, bindings) :
    frequencies ← {}
    foreach var in variables:
        frequencies[var] ← 0
    foreach binding in bindings:
        foreach (var, node) in binding.getKeys():
            frequencies[var] ← frequencies[var] + 1
    return generateJSONFrequencyTable(frequencies) # format the collection of variable
        ↪ frequencies as JSON
```

Listing 3.7: SPARQL-EVAL client

```
State:
    storageEngine ← createStorageEngine()

# SPARQL-EVAL.evaluateQuery(triplestoreID, plan):
function evaluateQuery(triplestoreID, plan):
    worker ← createQueryWorker(triplestoreID, storageEngine)
    execution ← createQueryExecution()
    execution.exec(plan, worker)
    return execution.getSPARQLResult()

function saveTriples(triplestoreID, triples):
    return storageEngine.save(triplestoreID, triples)

function deleteTriples(triplestoreID, triples):
    return storageEngine.delete(triplestoreID, triples)
```

Listing 3.8: SPARQL-EVAL triplestore

3.2.4.3 SPARQL Query Evaluation Protocol with Reasoning

We will now explain how we modify the SPARQL-EVAL protocol, by applying *query rewriting* techniques, during the generation of query execution plans, to retain reasoning capabilities.

Definition 9 (SPARQL Query Evaluation Protocol with Reasoning). Let SPARQL-EVAL-II be a *SPARQL* query evaluation protocol that supports reasoning, via the application of query rewriting techniques.

It is a tuple SPARQL-EVAL-II = (*PlanQuery*, SPARQL-EVAL.*EvaluateQuery*, SPARQL-EVAL.*GenerateFinalResult*) of 3 algorithms:

- $\{plan, construct, upload, delete, \perp\} \leftarrow$ **PlanQuery**(**query**, **triplestoreID**, **inference**, **expansionDepth**, **transitivityDepth**): takes as input a SPARQL query, a triplestore identifier, a boolean which indicates if the query needs to be rewritten and two integer parameters that control the rewriting depth. The *expansionDepth* specifies the number of rounds and the *transitivityDepth* specifies the number of transitivity relationships to considered. If the query is well-formed, and all required operations are supported, outputs a query execution plan and three collections of triple patterns used while constructing the final query results: the *construct*, *upload* and *delete* collections. In case of any error, it outputs $\perp$.

SPARQL-EVAL-II is a distributed, two-party protocol: the query plan generation and final result generation are executed by a *client* component; whereas the query evaluation and data updates are executed by a *triplestore* component.

Storage of Schema triples We will refer to the subset of triples containing nodes that belong to the Tbox as *schema triples*. Storage of such triples will not require indexation by the pattern type. Furthermore, in SEnTri we will only support a single set of schema triples per triplestore, which can either be uploaded or deleted, but not modified. In Listing 3.9 we specify the modifications and extra operations of SEnTri V0 storage engine that are required to support the storage of such triples.

```
State:
    keyword_schema ← generateRandomString()

function save(triplestoreID, triples, schema):
    if schema == true:
        delete(triplestoreID, schema)
        foreach t in triples:
            saveSchemaTriple(t)
    else:
        foreach t in triples:
            saveTriple(t)
    return "SUCCESS"

function delete(triplestoreID, schema):
    if schema == true:
        triplestores ← triplestores / { triplestores[triplestoreID][schema] }
    else:
        triplestores ← triplestores / { triplestores[triplestoreID] }
    return "SUCCESS"

function saveSchemaTriple(triple = (s, p, o)):
    parsed_s ← parseNode(s)
    parsed_p ← parseNode(p)
    parsed_o ← parseNode(o)
    triple ← parsed_s || separator_III || parsed_p || separator_III || parsed_o
    triplestores[triplestoreID][keyword_schema] ← triplestores[triplestoreID][keyword_schema] ∪ {
        ↪ triple }

function searchSchema(triplestoreID):
    triples_schema ← {}
    foreach t in triplestores[triplestoreID][keyword_schema]:
```

```
        (parsed_s, parsed_p, parsed_o) ← t.split(separator_III)
        triples_schema ← triples_schema ∪ { (generateNode(parsed_s), generateNode(parsed_p),
            ↪ generateNode(parsed_o)) }
    return triples_schema
```

Listing 3.9: Storage of schema triples

Query Rewriting To rewrite the queries we first fetch the schema triples, and then instantiate the triplestore's ontology. In SEnTri, an ontology is an abstraction where we map class and property axioms to nodes.

Using the *Apache Jena* API [91] we create an ontological model. After the generation of the model, we load it with the schema triples. Once the triples are loaded, we iterate over all the classes and map each one to equivalent classes, subclasses, restrictions and intersections they are part of.

In the next step, we iterate over all properties and map each one to their subproperties, equivalent and inverse properties. We also map the properties to two booleans which are true if, respectively, the property is symmetric and transitive.

All of this information is stored in adequate auxiliary data structures which will accessible to the query planner. Listing ... provides the specification of a triplestore's ontology in SEnTri V0.

During the creation of the query execution plan, when visiting a BGP, the query planner consults the ontology and searches for information related with the nodes present in the triple patterns. This information is used to expand the BGP with new patterns. The logic to rewrite queries is scoped to the query planner. All the modifications required are depicted in Listing 3.10.

```
State:
    operations_parsed ← {}
    upload_template ← {}
    delete_template ← {}
    construct_template ← {}
    queryType ← :bot
    plan ← (jobs = {}, execOrder = createLinkedList(), vars = {})
    variables ← {}
    ontology ← ⊥

function setOntology(ontology_new):
    ontology ← ontology_new

function extractVarsOfBinaryJob(jobs, job = (jobID, jobID_l, jobID_r)):
    vars ← {}
    jobs_queue ← createQueue()
    jobs_queue.push(jobs[jobID_l])
    jobs_queue.push(jobs[jobID_r])
    while #jobs_queue > 0:
        job_next ← jobs_queue.poll()
        if job_next is SEARCH:
            vars ← vars ∪ extractVars(searchJob)
        elif job_next = (jobID_next, l, r) is BINARY:
            jobs_queue.push(jobs[l])
            jobs_queue.push(jobs[r])
    return vars

function pushUnion(left, right, jobs, jobIDs):
```

```
    unionPattern ← "union" || left || right
    jobID ← jobIDs[unionPattern]
    if jobID == ⊥:
        job_MINUS ← createMinusJob(generateID(), right, left)
        jobID ← generateID()
        job_UNION ← createUnionJob(jobID, left, job_MINUS.jobID)
        jobs[jobID] ← job_UNION
        jobIDs[jobID] ← unionPattern
        pushJob(job_MINUS)
        pushJob(job_UNION)
    return jobID

private String pushJoin(left, right, jobs, jobIDs):
    joinPattern ← "join" || left || right
    jobID ← jobIDs[joinPattern]
    if jobID == ⊥:
        jobID = generateID()
        job_JOIN = new JoinJob(jobID, left, right)
        jobs[jobID] ← job_JOIN
        jobIDs[joinPattern] ← jobID
        pushJob(job_JOIN)
    return jobID

function generateSearchJobs(op_OpBGP):
    patterns = op_OpBGP.getPattern()
    jobIDs = {}
    jobs = {}
    jobVars = {}
    foreach t=(s,p,o) in patterns:
        jobID = pushSearch(s, p, o, jobs, jobIDs)
        if ontology ≠ ⊥:
            if (p == "rdf:type") and (o is not VARIABLE):
                jobID = expandClass(jobID, s, o, 0, jobs, jobIDs)
            elif p is not VARIABLE:
                jobID = expandProperty(jobID, s, p, o, 0, jobs, jobIDs, true, true)
        job ← jobs[jobID]
        if job is SEARCH:
            vars_job ← extractVars(job)
            variables ← variables ∪ vars_job
            jobVars[jobID] ← vars_job
        elif job is BINARY:
            jobVars[jobID] ← extractVarsOfBinaryJob(jobs, job)
    if #patterns > 1:
        generateRandomJoinPipeline(op_OpBGP, jobVars)
    else:
        operations_parsed.put(op_OpBGP, jobID)

function expandClass(jobID, s, o, depth, jobs, jobsIDs):
    if depth == ontology.getMaximumExpansionDepth():
        return jobID
    subclasses ← ontology.getSubClasses(o)
    equivalentClasses ← ontology.getEquivalentClasses(o)
    restriction ← ontology.getRestriction(o)
    intersectionClasses ← ontology.getIntersectionWhereClassIsOperand(o)
    jobID ← expandClassDisjunction(subclasses, jobID, s, depth, jobs, jobsIDs)
    jobID ← expandClassDisjunction(equivalentClasses, jobID, s, depth, jobs, jobsIDs)
    jobID ← expandRestriction(restriction, jobID, s, depth, jobs, jobsIDs)
    jobID ← expandClassDisjunction(intersectionClasses, jobID, s, depth, jobs, jobsIDs)
    intersection ← ontology.getIntersection(o)
    if #intersection == 0:
```

```
89          jobID ← expandClassConjunction(intersection, jobID, s, depth, jobs, jobsIDs)
90      return jobID
91
92  function expandRestriction(restriction = (type, property, value), jobID, s, depth, jobs, jobsIDs):
        ↪
93      if restriction ≠ ⊥:
94          if type == "owl:hasValue":
95          var ← createVariable(parse(property))
96              left ← pushSearch(var, "rdf:type", value, jobs, jobsIDs)
97              right ← expandProperty(pushSearch(s, property, value, jobs, jobsIDs), s, property
                    ↪ , value, depth + 1, jobs, jobsIDs, true, true)
98              jobID ← pushUnion(right, left, jobs, jobsIDs)
99          elif type ∈ {"owl:isSomeValuesFrom", "owl:allValuesFrom"}:
100             var ← createVariable(parse(value))
101             left ← expandClass(pushSearch(var, "rdf:type", value, jobs, jobsIDs), var, value,
                    ↪  depth + 1, jobs, jobsIDs)
102             right ← expandProperty(pushSearch(s, property, var, jobs, jobsIDs), s, property,
                    ↪ var, depth + 1, jobs, jobsIDs, true, true)
103             jobID ← pushUnion(jobID, pushJoin(right, left, jobs, jobsIDs), jobs, jobsIDs)
104     return jobID
105
106 function expandClassConjunction(classes, jobID, s, depth, jobs, jobsIDs):
107     foreach c in classes:
108         search ← pushSearch(s, "rdf:type", c, jobs, jobsIDs)
109         current = expandClass(search, s, c, depth + 1, jobs, jobsIDs)
110         if next ≠ ⊥:
111             next ← pushJoin(next, current, jobs, jobsIDs)
112         else:
113             next ← current
114     if next == ⊥:
115         return pushUnion(jobID, next, jobs, jobsIDs)
116     return jobID
117
118 function expandClassDisjunction(classes, jobID, s, depth, jobs, jobsIDs):
119     foreach c in classes:
120         search ← pushSearch(s, "rdf:type", c, jobs, jobsIDs)
121         current = expandClass(search, s, c, depth + 1, jobs, jobsIDs)
122         if next ≠ ⊥:
123             next ← pushUnion(next, current, jobs, jobsIDs)
124         else:
125             next ← current
126     if next == ⊥:
127         return pushUnion(jobID, next, jobs, jobsIDs)
128     return jobID
129
130 function expandProperty(jobID, s, p, o, depth, jobs, jobsIDs, symmetric, inverseOf):
131     if depth == ontology.getMaximumExpansionDepth():
132         return jobID
133     if symmetric == true and ontology.isSymmetric(p):
134         jobID ← pushUnion(jobID, pushSearch(o, p, s, jobs, jobsIDs), jobs, jobsIDs)
135         jobID ← expandProperty(jobID, o, p, s, depth + 1, jobs, jobsIDs, false, inverseOf)
136     if ontology.isTransitive(p):
137         var ← createVariable(parse(p) || 0)
138         left ← pushSearch(s, p, var, jobs, jobsIDs)
139         join ← pushJoin(left, pushSearch(var, p, o, jobs, jobsIDs), jobs, jobsIDs)
140         jobID ← pushUnion(jobID, join, jobs, jobsIDs)
141         for i in range(0, ontology.getTransitivityDepth())
142             for j in range(0, i + 1):
143                 if j == i:
144                     var_next = createVariable(parse(p) || j)
```

```
                    left ← pushJoin(left, pushSearch(var, p, var_next, jobs, jobsIDs), jobs,
                        ↪ jobsIDs)
                    var = var_next
                join ← pushJoin(left, pushSearch(var, p, o, jobs, jobsIDs), jobs, jobsIDs)
                jobID ← pushUnion(jobID, join, jobs, jobsIDs)
        subproperties ← ontology.getSubProperties(p)
        equivalentProperties ← ontology.getEquivalentProperties(p)
        jobID ← expandProperties(subproperties, jobID, s, o, depth, jobs, jobsIDs, symmetric,
            ↪ inverseOf)
        jobID ← expandProperties(equivalentProperties, jobID, s, o, depth, jobs, jobsIDs,
            ↪ symmetric, inverseOf)
        if inverseOf == true:
            inverseProperty ← ontology.getInverseOf(p)
            jobID ← expandProperties(inverseProperty, jobID, o, s, depth, jobs, jobsIDs,
                ↪ symmetric, false)
        return jobID

function expandProperties(properties, jobID, s, o, depth, jobs, jobsIDs, symmetric,
    ↪ inverseOf):
        for p in properties:
            jobID ← pushUnion(jobID, pushSearch(s, p, o, jobs, jobsIDs), jobs, jobsIDs)
            jobID ← expandProperty(jobID, s, p, o, depth + 1, jobs, jobsIDs, symmetric,
                ↪ inverseOf)
        return jobID
```

Listing 3.10: Query Plan Generation with Query Rewriting

Optimization and development of a fully functional query rewriting technique is out of the scope of this thesis. The system can not evaluate property paths, functions and nested queries, which limits our available solutions. We focused on trying to support a few expansions related with: subclasses, equivalent classes, restrictions and intersections; subproperties, equivalent, transitive, symmetric and inverse properties. Furthermore, we assume that the data that will be queried is consistent with the ontology provided.

We expand the triple patterns in a leveled approach. We decided to implement this parametrized expansion and transitive depth because it offers control over the expansion of the query, which impacts performance: the expansion may introduce additional operations.

3.2.4.4 Supported Queries

3.2.5 Encryption of triple patterns

When we encrypt *triple patterns* we need to ensure that we do not lose any semantics. Our encryption schemes will mimic the indexing strategy of SEnTri V0.

3.2.5.1 Trapdoor generation

Following the SSE approach, in SEnTri V1 and V2 we do not reference nodes using plaintext indexes. Instead, we generate *trapdoors* by deterministically encrypting the value of the indexes that would reference the node. To enforce the property that every trapdoor must uniquely reference a single node, we deterministically encrypt the index value using an incremental *initialization vector*. This initialization vector is equal to the observed frequency of the index keyword.

Trapdoors generated with $iv = 0$ always reference the latest value of the index keyword frequency.

For each index, the respective trapdoors are encrypted using a unique key. This key is build from a master key, for the context of the index keyword.

Definition 10 (**Trapdoor Generation**). Let $keyword_n$ be the keyword of the index that references node n. Let K_{MASTER} be a master key for a deterministic encryption scheme and $K_{keyword_n}$ be the derived key for the context of $keyword_n$. Finally, let i be the observed frequency for $keyword_n$, and $trapdoor_{n,i}$ be the trapdoor that is built from $keyword_n$, after $i-1$ observations of the term:

$$trapdoor_{n,i} = E(K_{keyword_n}, keyword_n, i) \tag{3.11}$$

The trapdoor that references the frequency value of $keyword_n$ is a special case where $i = 0$:

$$trapdoor_{n,0} = E(K_{keyword_n}, keyword_n, 0) \tag{3.12}$$

3.2.5.2 Triple Pattern SSE Scheme

Requirements SEnTri offers various functionalities, and to support all of them, the encryption schemes must respect a set of requirements:

- *Preserve Equality*: Encrypted nodes must preserve their equality property, which is essential for executing intermediate set operations during query evaluation;
- *Be dynamic*: For write operations, it must provide quick access to nodes from a specific triple pattern that requires deletion;
- *Preserve reasoning* To support the query *rewriting techniques* that we use, it needs to allow quick access to nodes belonging to the *TBox*.

SEnTri V1 and V2 encryption schemes differ uniquely on the method of encryption for the nodes belonging to the *ABox*, but are identical in all other aspects. If we consider the encryption of the *ABox* nodes as an abstract operation, we can isolate the *base encryption scheme* from where we build the other two. Such scheme is specified in Listing 3.11.

Definition 11 (**Triple Pattern SSE Scheme**). Let Π be a triple pattern SSE scheme that supports the evaluation of *SPARQL* queries. It is defined as a tuple $\Pi = (Setup, ReInitialize, Encrypt, Trapdoor, Decrypt, Equals)$ of 8 algorithms:

- $\lambda \leftarrow$ **Setup**():
- {SUCCESS, $\perp$} $\leftarrow$ **ReInitialize**(λ):
- $E(nodes) \leftarrow$ **Encrypt**({$\mathbf{t_0}, \mathbf{t_1}, ..., \mathbf{t_n}$}, **schema**):
- $trapdoor_{n,i} \leftarrow$ **Trapdoor**($\mathbf{keyword_n}$, **i**):
- $E(binding_n) \leftarrow$ **Search**($\mathbf{trapdoor_{n,i}}$):
- $node \leftarrow$ **Decrypt**(**E**(**node**), **schema**):
- {$true, false$} $\leftarrow$ **Equals**(**E**($\mathbf{node_1}$), **E**($\mathbf{node_2}$)):

3.2.5.3 Initial setup & re-initialization of scheme

During initialization, we generate all the encryption keys and security parameters of the scheme. We also create the data structures, where we keep track of the state of keyword frequencies, and derived keys.

3.2.5.4 Schema triple encryption (TBox statements)

...

```

// (K_MASTER, K_RND, iv_DET, keyword_schema) ← Π.Setup()
State:
    K_MASTER ← generateKey()
    K_RND ← generateKey()
    iv_DET ← generateRandomIV()
    keyword_schema ← generateRandomString()
    iv_ZERO ← generateZeroFilledIV()
    encryptedNodes ← {}
    keywordFrequencies ← {}
    derivedKeys ← {}

// E(T) ← Π.ReInitialize(K_MASTER, K_RND, keyword_schema)
function loadState(master, rnd, iv, schema):
    K_MASTER ← master
    K_RND ← rnd
    iv_ZERO ← iv
    keyword_schema ← schema
    iv_ZERO ← newZeroFilledIV()
    encryptedNodes ← {}
    keywordFrequencies ← {}
    derivedKeys ← {}

// E(T) ← Π.Encrypt(T, schema)
function encrypt({t_0, t_1, ... t_n}, schema):
    if (schema == true)
        encryptSchemaTriples({t_0, t_1, ... t_n})
    else
        encryptTriples({t_0, t_1, ... t_n})
    encryptKeywordInfo()

function EncryptSchemaTriples({t_0, t_1, ... t_n}):
    foreach t=(s,p,o) in {t_0, t_1, ... t_n}:
        encryptTBoxNode(parseNode(s))
        encryptTBoxNode(parseNode(p))
        encryptTBoxNode(parseNode(o))

function encryptTBoxNode(node):
    freq ← incrementKeywordFrequency(keyword_schema)
    trapdoor ← E(deriveKey(keyword_schema), keyword_schema, freq)
    encryptedNode[trapdoor] ← E(K_RND, node)


// E(T) ← Π.Decrypt(T, schema)
function decrypt(ct, schema):
    if (schema == true)
        return D(K_RND, ct)
    else
        return decryptNode(ct)
```

```
50
51  function decryptTriples({t_0, t_1, ... t_n}):
52    foreach t=(s,p,o) in {t_0, t_1, ... t_n}:
53          freq ← {}
54          parsed_s ← parseNode(s)
55          parsed_p ← parseNode(p)
56          parsed_o ← parseNode(o)
57          keyword_s ← parseKeyword(s)
58          keyword_p ← parseKeyword(p)
59          keyword_o ← parseKeyword(o)
60          freq ← freq ∪ { encryptNode(parsed_p, "PO" || keyword_s) }
61          freq ← freq ∪ { encryptNode(parsed_o, "PO" || keyword_s) }
62          freq ← freq ∪ { encryptNode(parsed_s, "SO" || keyword_p) }
63          freq ← freq ∪ { encryptNode(parsed_o, "SO" || keyword_p) }
64          freq ← freq ∪ { encryptNode(parsed_s, "SP" || keyword_o) }
65          freq ← freq ∪ { encryptNode(parsed_p, "SP" || keyword_o) }
66          freq ← freq ∪ { encryptNode(parsed_s, "S" || keyword_p || keyword_o) }
67          freq ← freq ∪ { encryptNode(parsed_p, "P" || keyword_s || keyword_o) }
68          freq ← freq ∪ { encryptNode(parsed_o, "O" || keyword_s || keyword_p) }
69          encryptTriple("SPO" || keyword_s || keyword_p || keyword_o, freq)
70
71  function encryptTriple(keyword, frequencies):
72      i ← 0
73      foreach freq in frequencies:
74          trapdoor ← E(deriveKey(keyword), keyword, i)
75          encryptedNode[trapdoor] ← E(K_RND, freq)
76          i++
77
78  function deriveKey(context):
79      key ← derivedKeys[context]
80      if key == ⊥:
81          key ← generateKey(K_MASTER, context)
82          derivedKeys[context] ← key
83      return key
84
85  function incrementKeywordFrequency(keyword):
86      freq ← keywordFrequencies[keyword]
87      if freq == ⊥:
88          freq ← 1
89          keywordFrequencies[keyword] ← freq
90      else:
91          keywordFrequencies[keyword] ← freq + 1
92      return freq
93
94  // trapdoor_{n,i} ← Π.Trapdoor(keyword_n, i)
95  function generateTrapdoor(keyword, i):
96      return E(deriveKey(keyword), keyword, i)
97
98  function generateTrapdoor(keyword):
99      return E(deriveKey(keyword), keyword, iv_ZERO)
100
101 function generateTrapdoorAndIncrementIV(keyword):
102     return E(deriveKey(keyword), keyword, incrementKeywordFrequency(keyword))
103
104 function generateTriplePatternTrapdoors({t_0, t_1, ... t_n}):
105     keywordPatternTrapdoors ← {}
106     foreach t=(s,p,o) in {t_0, t_1, ... t_n}:
107         keywords ← {}
108         i ← 0
109         keyword_s = parseKeyword(s)
```

```
        keyword_p = parseKeyword(p)
        keyword_o = parseKeyword(o)
        keyword_t = "SPO" || keyword_s || keyword_p || keyword_o
        keywords ← keywords ∪ { "PO" || keyword_s }
        keywords ← keywords ∪ { "PO" || keyword_s }
        keywords ← keywords ∪ { "SO" || keyword_p }
        keywords ← keywords ∪ { "SO" || keyword_p }
        keywords ← keywords ∪ { "SP" || keyword_o }
        keywords ← keywords ∪ { "SP" || keyword_o }
        keywords ← keywords ∪ { "S" || keyword_p || keyword_o }
        keywords ← keywords ∪ { "P" || keyword_s || keyword_o }
        keywords ← keywords ∪ { "O" || keyword_s || keyword_p }
        generatePatternTrapdoors(res, keyword_t, keywords)
        foreach keyword in keywords:
            trapdoors ← keywordPatternTrapdoors[keyword]
            if (trapdoors == ⊥):
                trapdoors ← {}
            trapdoors ← trapdoors ∪ { E(deriveKey(keyword_t), keyword_t, i) }
            keywordPatternTrapdoors[keyword] ← trapdoors
            i++
    return keywordPatternTrapdoors

function setKeywordFrequencies(values):
    foreach keyword in values:
        freq ← values[keyword]
        if (freq > 0)
            keywordFrequencies[keyword] ← freq

function getSchemaKeyword():
    return keyword_schema

function getMasterKey():
    return K_MASTER

function getRNDKey():
    return K_RND

function getEncryptedNodes():
    return encryptedNodes

function clearNodes():
    encryptedNodes ← {}

function clearFrequencies():
    keywordFrequencies ← {}
```

Listing 3.11: Base encryption scheme

3.2.5.5 Triple Encryption (ABox statements)

SEnTri V1 In SEnTri V1, an encrypted *ABox* node corresponds to $E(K_{RND}, E(K_{DET}, node, iv_{DET}))$, where iv_{DET} is a randomly generated initialization vector and *node* is either a subject, predicate or object.

```
State:
    K_DET ← generateKey()

function loadState(master, rnd, det, iv, schema):
```

```
    loadState(master, rnd, iv, schema)
    K_DET ← det

function encryptKeywordInfo():
    foreach keyword in keywordFrequencies:
        trapdoor ← E(derivedKeys[keyword], keyword, iv_ZERO)
        encryptedNode[trapdoor] ← E(K_RND, keywordFrequencies[keyword])

function encryptNode(node, keyword):
    freq ← incrementKeywordFrequency(keyword)
    trapdoor ← E(deriveKey(keyword), keyword, freq)
    encryptedNode[trapdoor] ← E(K_RND, E(K_DET, node, iv_DET))
    return freq

function getDETKey():
    return K_DET

function getIvDET():
    return iv_DET
```

Listing 3.12: Triple Pattern Encryption for SEnTri V1

SEnTri V2 In SEnTri V2, an encrypted *ABox* node corresponds to $E(K_{RND}, node)$. Its encrypted *equality tag* corresponds to $E(K_{DGK_{pub}}, eqTag(node))$, where $eqTag : (I \cup L \cup V) \rightarrow \mathbb{Z}^+$ is an injective function that maps the value of a subject, predicate or object to a unique positive integer.

We index the encrypted the node by the *encrypted equality tag*. The equality tag is referenced by the trapdoor that would index the node in SEnTri V1.

```
State:
    (K_DGK_priv, K_DGK_pub) ← generateDGKKeyPair()
    eqTags ← {}
    eq_last ← 0

function loadState(master, rnd, KeyPair_DGK, iv, schema):
    loadState(master, rnd, iv, schema)
    (K_DGK_priv, K_DGK_pub) ← KeyPair_DGK

function encryptKeywordInfo():
    foreach keyword in keywordFrequencies:
        trapdoor ← E(derivedKeys[keyword], keyword, iv_ZERO)
        encryptedNode[trapdoor] ← E(K_RND, keywordFrequencies[keyword])
    foreach node in eqTags:
        trapdoor ← E(derivedKeys[node], node, iv_ZERO)
        encryptedNode[trapdoor] ← E(K_RND, keywordFrequencies[node])


function encryptNode(node, keyword):
    freq ← incrementKeywordFrequency(keyword)
    trapdoor ← E(deriveKey(keyword), keyword, freq)
    enc_eq_tag ← E(K_DGK_pub, getEqTag(node))
    encryptedNode[trapdoor] ← enc_eq_tag
    encryptedNode[enc_eq_tag] ← E(K_RND, node)
    return freq

function getEqTag(node):
    eq_tag ← eqTags[node]
```

```
29      if eq_tag == ⊥:
30          eq_tag ← eq_last + 1
31          eqTags[node] ← eq_tag
32      if (eq_tag > eq_last)
33          eq_last ← eq_tag
34      return eq_tag;
35
36  function setEqTags(value):
37      eqTags ← value
38
39  function getDETKey():
40      return K_DET
41
42  function getIvDET():
43      return iv_DET
44
45  function getLastEqTag():
46      return eq_last
47
48  function getEqKey():
49      return K_DGK_eq = (K_DGK_priv.p, K_DGK_priv.vp, K_DGK_pub.n, K_DGK_pub.u)
50
51  function clearEqTags():
52      eqTags ← {}
```

Listing 3.13: Triple Pattern Encryption for SEnTri V2

3.2.5.6 Encrypted ABox Node equality

3.2.5.7 Supporting Dynamism

...

3.2.5.8 Evaluation of SPARQL queries over encrypted data

...

Definition 12 (**Secure SPARQL Query Evaluation Protocol**). Let SEC-SPARQL-EVAL be a *SPARQL* query evaluation protocol, that works with triplestores encrypted using Π. It is defined as a tuple $\Omega = (Setup, ReInitialize, Encrypt, Trapdoor, Decrypt, Equals)$ of 8 algorithms:

- $plan_{rnd} \leftarrow$ **PlanQuery**(**query**):
- $preparedPlan_{rnd} \leftarrow$ **PrepareQuery**($\mathbf{plan_{rnd}}$, **triplestoreID**):
- $E(sparqlResult) \leftarrow$ **EvaluateQuery**($\mathbf{preparedPlan_{rnd}}$):

Query Preparation Phase

Query Evaluation Phase

Decryption of SPARQL query results

3.2.6 The M/M setting

We've specified the building blocks for secure evaluation of SPARQL queries. We will now explain how we do so in a multi-user setting where shared access to triplestores exists. It all starts with authentication of users, followed by a structured way of sharing triplestores, with different degrees of access, and allowing concurrent execution of queries.

Authentication is validated with *sessions*, management of access to triplestores is realized with the definition of triplestore ACLs and strict *permission rules*. Finally, we rely on a lock-based access strategy, and *ephemeral access tokens*, to ensure that concurrent access, when executing unsafe operations (writes), is done in a controlled and safe manner.

3.2.6.1 Session Validation

Authentication is the process where we verify the identity of a user. We do so, by checking that the provided credentials, during login, match the ones saved within our system. To avoid having to verify this condition in every request sent by the user, we use *sessions*. The session is generated upon a successful login request. A valid session will prove the identity of the user sending the requests. In SEnTri, an authenticated user needs to be valid (has been registered within the system) and hold an active session.

Definition 13 (**Authentication Check**). Let $IsAuth(user)$ be a predicate that evaluates to $true$ if a $user$ is authenticated, $\perp$ otherwise. Given $s_i = (sessionID,\ username_s,\ ttl)$ and $user_u = (username_u,\ password_u,\ role_u,\ owned_u)$ be valid, correctness of the authentication procedure will be upheld by always checking if the user has an active session:

$$IsAuth(user_u) \Leftrightarrow \exists_s(username_u = username_s \ \wedge \ ttl > 0) \tag{3.13}$$

3.2.6.2 Enforcing Access Permissions

Permissions in SEnTri are logical rules which dictate whether a user is allowed to execute a certain operation, or not. These rules are continuously checked during execution.

Definition 14 (**Permission Rules**). Let $HasAccess(user,\ triplestore, type)$ be a predicate that evaluates to true if $user$ has access of a certain $type$, to the specified $triplestore$, $\perp$ otherwise. Let $CanGrantAccess(user,\ type,\ triplestore,\ target)$, and $CanRevokeAccess(user,\ type,\ triplestore,\ target)$ be predicates that evaluate to $true$, respectively, if a $user$ can grant or revoke access, of a certain $type$, to the specified $triplestore$, to the specified $target$ user, $\perp$ otherwise.

Let $CanCreateTriplestore(user,\ triplestore)$, $CanDeleteTriplestore(user, triplestore)$, $CanChangeOwnership(user,\ triplestore,\ target)$ be predicates that evaluate to $true$, respectively, if a $user$ can create, delete, or transfer ownership, to $target$ user, of the specified $triplestore$, $\perp$ otherwise.

Finally, let $CanGrantRole(user,\ role,\ target)$, $CanRevokeRole(user,\ role,\ target)$ be predicates that evaluate to $true$, respectively, if a $user$ can grant, or revoke, the specified $target$ user's $role$, $\perp$ otherwise.

Given $type \in \{\text{READ}, \text{WRITE}, \text{OWNER}\}$, triplestore $T_i = (triplestoreID_i,\ nodes_i,\ acp_i = (r_i, w_i))$, $user_u = (username_u,\ password_u,\ role_u,\ owned_u)$, and $user_t = (username_t,\ role_t,\ password_t,\ owned_t)$ SEnTri can verify, when requested, the following permissions:

$$
\begin{aligned}
&CanGrantAccess(user_u, type, T_i, user_t) \Leftrightarrow IsAuth(user_u) \wedge \exists_{user_t}\, user_u \neq user_t \\
&\quad \wedge (role_u = \text{ADMIN} \vee triplestoreID_i \in owned_u) \\
\\
&CanRevokeAccess(user_u, type, T_i, user_t) \Leftrightarrow IsAuth(user_u) \\
&\quad \wedge \exists_{user_t}\, user_u \neq user_t \wedge (role_u = \text{ADMIN} \vee triplestoreID_i \in owned_u) \\
\\
&HasAccess(user_u, type, T_i) \Leftrightarrow role_u = \text{ADMIN} \vee triplestoreID_i \in owned_u \\
&\quad \vee (type = \text{READ} \wedge username_u \in r_i) \vee (type = \{\text{READ, WRITE}\} \wedge username_u \in w_i) \\
&\quad \vee (type = \text{OWNER} \wedge triplestoreID_i \in owned_u) \\
\\
&CanCreateTriplestore(user_u, T_i) \Leftrightarrow IsAuth(user_u) \\
&\quad \wedge role_u \in \{\text{ADMIN, PRIVILEGED}\} \wedge \nexists_{triplestore}\, triplestoreID = triplestoreID_i \\
\\
&CanDeleteTriplestore(user_u, T_i) \Leftrightarrow IsAuth(user_u) \\
&\quad \wedge role_u = \text{ADMIN} \wedge triplestoreID_i \in owned_u \\
\\
&CanChangeOwnership(user_u, T_i, user_t) \Leftrightarrow IsAuth(user_u) \wedge \exists_{user_t}\, user_u \neq user_t \\
&\quad \wedge (role_u = \text{ADMIN} \vee triplestoreID_i \in owned_u) \wedge role_t \neq \text{BASIC} \\
\\
&CanGrantRole(user_u, role, user_t) \Leftrightarrow IsAuth(user_u) \\
&\quad \wedge role \neq role_t \wedge ((\exists_{user_t}\, user_u \neq user_t \wedge role_u = \text{ADMIN}) \\
&\quad \vee (user_u = user_t \wedge (role_u = \text{ADMIN} \vee role = \text{BASIC}))) \\
\\
&CanRevokeRole(user_u, role, user_t) \Leftrightarrow IsAuth(user_u) \\
&\quad \wedge role \neq role_t \wedge ((\exists_{user_t}\, user_u \neq user_t \wedge role_u = \text{ADMIN}) \vee user_u = user_t)
\end{aligned}
\tag{3.14}
$$

3.2.6.3 Lock-based Concurrent Access

Concurrent operations can target users or triplestores. The correctness of concurrent write and read operations is achieved using a *lock-based* approach. Resources (users and triplestores) can be in a *locked* or *unlocked* state. Write operations can only be requested against *unlocked* resources, which remain locked while the operation is being executed. Consequently, read operations can occur simultaneously, but write operations are sequential, against the same resource.

Locks are enough for requests targeting users. Operations can only be executed by users, against themselves, or by *admin* users, and the origin of the requests is always the *Client* (the execution context is constant).

Operations against triplestores are more complex. The query evaluation pipeline starts in the *Client* component, but subsequent steps are executed in the *Triplestore*, *Vault* and *Secure Computation Proxy*. Furthermore, whenever the execution context changes, we must verify if the author of the request has sufficient permissions.

To fulfill such requirements, every request from the *Client* to other components, contains an *access token*. The other components execute permission checks, by sending this token to the *IAM Provider*. The *IAM Provider* verifies the validity of the token, and if its associated permissions are correct.

Queries that contain write predicates, will require that their target remains *locked*, during their evaluation. As such, an *access token* can be associated with a *lock*. The triplestore will remain in a *locked* state until the *access token* is deleted or the *lock* expires.

Definition 15 (**Concurrent Lock-based Access Checks**). Let $IsLocked(resource, l)$ and $CanLock(triplestore, at)$ be predicates that evaluate to $true$ if, respectively, $resource$ is in a $locked$ state associated with the lock l and $triplestore$ can transition to a $locked$ state, and associate the generated lock with access token at, $\perp$ otherwise.

Let $CanExecuteOperations(triplestore, at, type)$ be a predicate that evaluates to $true$ if the access token at grants sufficient permissions, against the target $triplestore$, to allow operations, of the specified access $type$, to be executed in the current context of execution, $\perp$ otherwise.

Given $type \in \{\text{READ}, \text{WRITE}, \text{OWNER}\}$, triplestore $T_i = (triplestoreID_i, nodes_i, acp_i = (r_i, w_i))$, lock $l_a = (lockID_a, resourceID_a, ttl_a)$, lock $l_b = (lockID_b, resourceID_b, ttl_b)$, access token $at_k = (tokenID_k, username_k, ttl_k, sessionID_k, triplestoreID_k, lockID_k)$, user $user_u = (username_u, password_u, role_u, owned_u)$ and session $s_j = (session_{ID_j}, username_j, ttl_j)$, let the resources $resource_a$ and $resource_b$ be identified by, respectively, $resourceID_a$ and $resourceID_b$. Concurrent access correctness is achieved by checking, when required, if a resource is locked, can be locked and if the access token is associated with a valid authenticated user, with enough permissions to execute the desired operation:

$$
\begin{aligned}
&IsLocked(resource_a, l_a) \Rightarrow \exists_{l_a} \exists_{resource_a} (ttl_a > 0) \\
&\quad \wedge \nexists_{l_b} (resourceID_b = resourceID_a \wedge lockID_a \neq lockID_b \wedge ttl_b > 0) \\
&\quad \wedge \nexists_{l_b} (resourceID_b \neq resourceID_a \wedge lockID_a = lockID_b \wedge ttl_b > 0) \\
\\
&CanLock(T_i, at_k) \Leftrightarrow \nexists_l (IsLocked(T_i, l)) \wedge triplestoreID_i = triplestoreID_k \\
&\quad \wedge username_k = username_u \wedge username_j = username_u \wedge ttl_j > 0 \\
&\quad \wedge ttl_k > 0 \wedge HasAccess(user_u, type, T_i) \wedge type \in \{\text{WRITE, OWNER}\} \\
\\
&CanExecuteOperations(T_i, at_k, type) \Leftrightarrow triplestoreID_i = triplestoreID_k \\
&\quad \wedge username_k = username_u \wedge username_j = username_u \wedge ttl_j > 0 \\
&\quad \wedge ttl_k > 0 \wedge HasAccess(user_u, type, T_i) \wedge (type = \text{READ} \\
&\quad \vee (type \in \{\text{WRITE, OWNER}\} \wedge IsLocked(T_i, l_a) \wedge lockID_a = lockID_k))
\end{aligned}
\tag{3.15}
$$

3.2.7 Definition of SEnTri

We will now formally define all versions of SEnTri.

Definition 16 (**SEnTri V0**). SEnTri V0 is a collection of 3 protocols (IAM, TRIPLESTORE, CLIENT) where:

- IAM is a tuple of 24 polynomial time algorithms. These algorithms are responsible for management of user data, roles and triplestore access policies. They are also responsible for session and permissions enforcement:

- $\{\text{SUCCESS}, \perp\} \leftarrow$ **init**(): initializes the protocol by registering the default admin user. Outputs SUCCESS if the user is created and registered, $\perp$ otherwise.
- $\{cookie, \perp\} \leftarrow$ **auth**(**username**, **password**): takes as input a pair of user credentials. In case of valid credentials, creates a new session and outputs a *cookie*, $\perp$ otherwise.
- $\{\text{SUCCESS}, \perp\} \leftarrow$ **registerUser**(**username**, **password**): takes as input a pair of user credentials. Outputs SUCCESS if a new user is registered, $\perp$ otherwise.
- $\{\text{SUCCESS}, \perp\} \leftarrow$ **deleteUser**(**cookie**, **username**) takes as input a cookie and the username of the user to delete. Outputs SUCCESS if the user is successfully deleted, $\perp$ otherwise.
- $\{\text{SUCCESS}, \perp\} \leftarrow$ **issueGrantRoleRequest**(**cookie**, **username**, **issuer**, **role**): takes as input a cookie, the usernames of the target and issuer of the role changing request and the desired role. Outputs SUCCESS if the request is successfully persisted, $\perp$ otherwise.
- $[roleRequest_0, ..., roleRequest_n] \leftarrow$ **getPendingRoleRequests**(**cookie**, **username**): takes as input a cookie, and the request issuer username. Outputs the collection of pending role changing requests.
- $\{\text{SUCCESS}, \perp\} \leftarrow$ **processRoleRequest**(**cookie**, **username**, **target**, **requestID**, **decision**): takes as input a cookie, the username of the processing request's issuer, the username of the role change target, the request identifier and the issuer's decision. Outputs SUCCESS if the role changing request is successfully processed, according to the provided decision, $\perp$ otherwise.
- $\{\text{SUCCESS}, \perp\} \leftarrow$ **createTriplestoreAccessPolicy**(**cookie**, **issuer**, **triplestoreID**): takes as input a cookie, the username of the request's issuer and a triplestore identifier. Outputs SUCCESS if the triplestore access policy is sucessfully generated, $\perp$ otherwise.
- $[triplestoreID_0, ..., triplestoreID_n] \leftarrow$ **listTriplestores**(**cookie**, **issuer**, **owns**, **read**, **write**): takes as input a cookie, the username of the request's issuer and three boolean filter parameters. Outputs a collection of triplestore identifiers: if *owns* is *true*, returns identifiers of *owned* triplestores; if *read* is *true*, returns identifiers of triplestores that the user has *read* access; if $write$ is *true*, returns identifiers of triplestores that the user has *write* access;
- $\{msg, \perp\} \leftarrow$ **listUsersWithAccess**(**cookie**,**triplestoreID**, **write**, **at**): takes as input a cookie, a triplestore identifier, a boolean filter parameter and an access token. If the access token has sufficient permissions, outputs a message with the specified triplestore's *owner* username and the collection of *read* and *write* access (if the $write$ parameter is *true*) users' usernames, $\perp$ otherwise.
- $\{\text{SUCCESS}, \perp\} \leftarrow$ **changeTriplestoreOwner**(**cookie**, **triplestoreID**, **target**, **at**): takes as input a cookie, a triplestore identifier, a username and an access token. Outputs SUCCESS if the ownership of the specified triplestore is successfully transfered to the *target* user, $\perp$ otherwise.
- $\{\text{SUCCESS}, \perp\} \leftarrow$ **deleteTriplestoreAccessPolicy**(**cookie**, **triplestoreID**, **at**): takes as input a cookie a triplestore identifier and an access token. Outputs SUCCESS if the specified triplestore access policy is successfully deleted, $\perp$ otherwise.

- {SUCCESS, $\perp$} $\leftarrow$ **grantAccess**(**cookie**, **triplestoreID**, **target**, **write**, **at**): takes as input a cookie, a triplestore identifier, a username, a boolean parameter and an access token. Outputs SUCCESS if the *target* user is successfully granted *read*, or *write* (if *write* parameter is *true*), access against the specified triplestore, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **revokeAccess**(**cookie**, **triplestoreID**, **target**, **write**, **at**): takes as input a cookie, a triplestore identifier, a username, a boolean parameter and an access token. Outputs SUCCESS if the *target* user *read*, or *write* (if *write* parameter is *true*), access against the specified triplestore is successfully revoked, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **issueAccessRequest**(**cookie**, **triplestoreID**, **target**, **write**): takes as input a cookie, a triplestore identifier, a username, a boolean parameter and an access token. Outputs SUCCESS if an access change request, for the *target* user, granting *read* or *write* (if the *write* parameter is *true*) permissions against the specified triplestore is sucessfully issued, $\perp$ otherwise.
- {[$accessRequest_0$,...,$accessRequest_n$], $\perp$} $\leftarrow$ **getPendingAccessRequests**(**cookie**, **triplestoreID**, **at**): takes as input a cookie, a triplestore indentifier and an access token. Outputs the collection of the triplestore's pending access change requests, if the specified access token has sufficient permissions, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **processAccessRequest**(**cookie**, **triplestoreID**, **requestID**, **decision**, **at**): takes as input a cookie, a triplestore identifier, an access change request identifier, a boolean parameter and an access token. Outputs SUCCESS if the specified request is successfully processed according to the provided *decision* parameter, $\perp$ otherwise.
- {*tokenID*, $\perp$} $\leftarrow$ **createAccessToken**(**cookie**, **triplestoreID**, **target**): takes as input a cookie, a triplestore identifier, and a username. Outputs the newly generated token identifier, if the *target* user has sufficient permissions against the specified triplestore, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **deleteAccessToken**(**cookie**, **triplestoreID**, **at**): takes as input a cookie, a triplestore identifier, and an access token. Outputs SUCCESS if the token is successfully deleted, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **checkReadAccess**(**triplestoreID**, **at**): takes as input a triplestore identifier and an access token. Outputs SUCCESS if the token grants *read* access to the specified triplestore, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **checkWriteAccess**(**triplestoreID**, **at**): takes as input a triplestore identifier and an access token. Outputs SUCCESS if the token grants *write* access to the specified triplestore, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **checkOwnerAccess**(**triplestoreID**, **at**): takes as input a triplestore identifier and an access token. Outputs SUCCESS if the token grants *owner* access to the specified triplestore, $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **checkIfActive**(**at**): takes as input a triplestore identifier and an access token. Outputs SUCCESS if the token is active (the associated session or the token has not expired), $\perp$ otherwise.
- {SUCCESS, $\perp$} $\leftarrow$ **acquireTriplestoreLock**(**cookie**, **at**): takes as input a cookie and an access token. Outputs SUCCESS if the token is successfully associated with a newly generated lock against the triplestore affected by the token, $\perp$ otherwise.

  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **releaseTriplestoreLock**(**cookie**, **at**): takes as input a cookie and an access token. Outputs SUCCESS if the lock associated with the token is successfully unlocked, $\perp$ otherwise.

- CLIENT is a tuple of 20 polynomial time algorithms. This protocol centralizes all functionalities of SEnTri and abstracts the communication logic with the other protocols. It is responsible for: 1) session validation; 2) user creation and role management; 3) triplestore creation, deletion, sharing (access management) and querying. We will consider this as the public facing protocol, whereas the others are internal. These 20 algorithms are:

  - $\{cookie, \perp\} \leftarrow$ **auth**(**username**, **password**): takes as input a pair of user credentials, executes IAM.$auth$(*username*, *password*) and outputs the result of the algorithm.
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **registerUser**(**username**, **password**): takes as input a pair of user credentials, executes IAM.$registerUser$(*username*, *password*) and outputs the result of the algorithm.
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **deleteUser**(**cookie**, **username**): takes as input a cookie and the username of the user to delete, executes IAM.$deleteUser$(*cookie*, *username*) and outputs the result of the algorithm.
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **issueUpgradeRequest**(**cookie**, **username**): takes as input a cookie, and a username. Executes IAM.$issueGrantRoleRequest$(*cookie*, *username*, *username*, PRIVILEGED) and outputs the result of the algorithm.
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **issueDowngradeRequest**(**cookie**, **username**): takes as input a cookie, and a username. Executes IAM.$issueGrantRoleRequest$(*cookie*, *username*, *username*, BASIC) and outputs the result of the algorithm.
  - $[roleRequest_0, ..., roleRequest_n] \leftarrow$ **listPendingRoleRequests**(**cookie**, **username**): takes as input a cookie, and the request issuer username. Executes IAM.$getPendingRoleRequests$(*cookie*, *username*) and outputs the result of the algorithm.
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **processRoleRequest**(**cookie**, **username**, **target**, **requestID**, **decision**): takes as input a cookie, the username of the processing request's issuer, the username of the role change target, the request identifier and the issuer's decision. Executes IAM.$processRoleRequest$(*cookie*, *username*, *target*, *requestID*, *decision*) and outputs the result of the algorithm.
  - $\{[triplestoreID_0, ..., triplestoreID_n], \perp\} \leftarrow$ **listTriplestores**(**cookie**, **issuer**, **write**, **read**, **owns**): takes as input a cookie, the username of the request's issuer and three boolean filter parameters. Executes IAM.$listTriplestores$(*cookie*, *issuer*, *write*, *read*, *owns*) and outputs the result of the algorithm, or $\perp$ in case of error.
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **updateTriplestoreOwner**(**cookie**, **triplestoreID**, **issuer**, **target**): takes as input a cookie, a triplestore identifier, a username and an access token. Creates an access token by executing ... . In case of success, tries to lock the triplestore with ... . If it fails to lock the resource, deletes the access token by executing ... . If the lock is successfully acquired, executes ... and retrieves the response of the algorithm. Finally, releases the lock and deletes the access token, with ... followed by ... , and outputs the retrieved response.

  - {SUCCESS, $\perp$} $\leftarrow$ **grantAccess**(**cookie**, **triplestoreID**, **issuer**, **target**, **write**):
  - {SUCCESS, $\perp$} $\leftarrow$ **revokeAccess**(**cookie**, **triplestoreID**, **issuer**, **target**, **write**):
  - {SUCCESS, $\perp$} $\leftarrow$ **issueAccessRequest**(**cookie**, **triplestoreID**, **issuer**, **write**):
  - {SUCCESS, $\perp$} $\leftarrow$ **listPendingAccessRequests**(**cookie**, **triplestoreID**, **issuer**):
  - {SUCCESS, $\perp$} $\leftarrow$ **processAccessRequest**(**cookie**, **triplestoreID**, **issuer**, **requestID**, **decision**):
  - {SUCCESS, $\perp$} $\leftarrow$ **listUsersWithAccess**(**cookie**, **triplestoreID**, **issuer**, **write**):
  - {SUCCESS, $\perp$} $\leftarrow$ **createTriplestore**(**cookie**, **triplestoreID**):
  - {SUCCESS, $\perp$} $\leftarrow$ **uploadTriplestoreContent**(**cookie**, **issuer**, **triplestoreID**, **schema**, **syntax**, **content**):
  - {SUCCESS, $\perp$} $\leftarrow$ **deleteTriplestore**(**cookie**, **issuer**, **triplestoreID**, **inference**):
  - {SUCCESS, $\perp$} $\leftarrow$ **fetchTriplestoreSchema**(**cookie**, **triplestoreID**, **inference**):
  - {SUCCESS, $\perp$} $\leftarrow$ **answerSPARQLQuery**(**cookie**, **triplestoreID**, **issuer**, **query**, **inference**, **transitivityDepth**, **expansionDepth**):

- TRIPLESTORE is a tuple of 5 polynomial time algorithms. These algorithms are responsible for ...:

  - {SUCCESS, $\perp$} $\leftarrow$ **upload**(**triplestoreID**, **nodes**, **at**):
  - {SUCCESS, $\perp$} $\leftarrow$ **delete**(**triplestoreID**, **at**):
  - {SUCCESS, $\perp$} $\leftarrow$ **delete**(**triplestoreID**, **indexes**, **at**):
  - {SUCCESS, $\perp$} $\leftarrow$ **update**(**triplestoreID**, **deletions**, **uploads**, **at**):
  - {$sparqlResult$, $\perp$} $\leftarrow$ **query**(**triplestoreID**, **plan**, **at**):

Definition 17 (**SEnTri V1**)**.** SEnTri V1 extends functionalities of SEnTri V0 to work with encrypted data. It is a collection of 5 protocols (SEnTri-V0.IAM, CLIENT, VAULT, TRIPLESTORE, PROXY) where:

- CLIENT is a tuple of 20 polynomial time algorithms. Like in version V0, this protocol centralizes all functionalities of SEnTri and abstracts the communication logic with the other protocols. It retains the same functionalities, and only differs from version V0, in the algorithms for triplestore management and querying. We will only define these algorithms, and assume the others to be equal as their previous definition:

  - {SUCCESS, $\perp$} $\leftarrow$ **createTriplestore**(**cookie**, **triplestoreID**):
  - {SUCCESS, $\perp$} $\leftarrow$ **uploadTriplestoreContent**(**cookie**, **issuer**, **triplestoreID**, **schema**, **syntax**, **content**):
  - {SUCCESS, $\perp$} $\leftarrow$ **deleteTriplestore**(**cookie**, **issuer**, **triplestoreID**, **inference**):
  - {SUCCESS, $\perp$} $\leftarrow$ **fetchTriplestoreSchema**(**cookie**, **triplestoreID**, **inference**):
  - {SUCCESS, $\perp$} $\leftarrow$ **answerSPARQLQuery**(**cookie**, **triplestoreID**, **issuer**, **query**, **inference**, **transitivityDepth**, **expansionDepth**):

- VAULT is a tuple of 3 polynomial time algorithms. These algorithms are responsible for ...:

  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **create**(**triplestoreID**, **secrets**, **at**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **delete**(**triplestoreID**, **at**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **get**(**triplestoreID**, **at**):

- TRIPLESTORE is a tuple of 6 polynomial time algorithms. These algorithms are responsible for ...:

  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **upload**(**triplestoreID**, **encNodes**, **at**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **delete**(**triplestoreID**, **at**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **delete**(**triplestoreID**, **trapdoors**, **at**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **update**(**triplestoreID**, **deletions**, **uploads**, **at**):
  - $\{encBindings, \perp\} \leftarrow$ **search**(**triplestoreID**, **trapdoors**, **at**):
  - $\{searchID, \perp\} \leftarrow$ **prepareQuery**(**triplestoreID**, **trapdoors**, **at**):

- PROXY is a tuple of 2 polynomial time algorithms. These algorithms are responsible for saving the outputs of TRIPLESTORE.*search*(...) operations, preparing this data for query evaluation, executing SPARQL query plans and outputting valid query results:

  - $\{searchID, \perp\} \leftarrow$ **prepareQuery**(**encBindings**, **at**): takes as input a collection of encrypted bindings and an access token. If the token is valid, and grants sufficient permissions, saves the bindings in memory and associates the data with a newly generated $searchID$, a UUID (version 4). In case of success, outputs the identifier, $\perp$ otherwise.
  - $\{sparqlResult, \perp\} \leftarrow$ **answerSPARQLQuery**(**plan**, $\mathbf{K_{RND}}$, **at**) takes as input a collection of a SPARQL query plan, a secret key and an access token. If the token is valid, and grants sufficient permissions, executes the query plan and generates a query result. In case of success outputs the result, $\perp$ otherwise.

Definition 18 (**SEnTri V2**)**.** SEnTri V2 is a collection of 5 protocols (TRIPLESTORE, SEnTri-V0.IAM, CLIENT, SEnTri-V1.VAULT, PROXY):

- CLIENT is a tuple of 20 polynomial time algorithms. Like in previous versions, the protocol centralizes all functionalities of SEnTri and abstracts the communication logic with the other protocols. It retains the same functionalities, and only differs from version V1, in the algorithms for triplestore management and querying. We will only define these algorithms, and assume the others to be equal as their previous definition:

  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **createTriplestore**(**cookie**, **triplestoreID**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **uploadTriplestoreContent**(**cookie**, **issuer**, **triplestoreID**, **schema**, **syntax**, **content**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **deleteTriplestore**(**cookie**, **issuer**, **triplestoreID**, **inference**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **fetchTriplestoreSchema**(**cookie**, **triplestoreID**, **inference**):
  - $\{\text{SUCCESS}, \perp\} \leftarrow$ **answerSPARQLQuery**(**cookie**, **triplestoreID**, **issuer**, **query**, **inference**, **transitivityDepth**, **expansionDepth**):

- TRIPLESTORE protocol is a tuple with all SEnTri-V1.TRIPLESTORE algorithms, apart from the TRIPLESTORE.*prepareSearch*. In this version the algorithm, we mask the bindings of a search. We will only define this algorithm, and assume the others to be equal as their previous definition:
  - $\{searchID, \perp\} \leftarrow$ **prepareMaskedSearch**(**triplestoreID**, **trapdoors**, **mask**, **at**):
- PROXY is a tuple of 2 polynomial time algorithms. Like in SEnTri V1 it will prepare and evaluate queries. It differs from the previous version, on query evaluation algorithm. We will only define this algorithms, and assume the others to be equal as their previous definition:
  - $\{sparqlResult, \perp\} \leftarrow$ **answerSPARQLQuery**(**plan**, $\mathbf{K_{DGK_{eq}}}$, **at**) takes as input a collection of a SPARQL query plan, a secret key and an access token. If the token is valid, and grants sufficient permissions, executes the query plan and generates a query result. In case of success outputs the result, $\perp$ otherwise.

3.2.8 Component to Protocol Mappings

3.3 Security Analysis

3.3.1 SEnTri Security Properties

In conformity with the assumptions explained in ..., and based on its definition, we expect all implementations of SEnTri to possess the following *security properties*:

1. A user will be able to create triplestores and access tokens, if and only if they have an active session.
2. A user will be able to interact with a triplestore, if and only if they have a valid access token.
3. A user will not be able to query triplestores which they have not been given access to.
4. A user will only be able to execute operations according to its role.
5. An external observer will not be able to learn any useful information from eavesdropping messages sent between components.
6. An attacker will not be able to impersonate any component.
7. An attacker will not be able to tamper with messages sent between components.

In conformity with the lack of data confidentiality, present in SEnTri-V0, the following security properties are exclusive for the other two implementations:

1. An attacker with access to the *Triplestore* component will learn nothing more than the size of a triplestore and its *search* and *access* patterns.
2. Regarding *term equality* during intermediate set operations between *bindings*, an attacker with access to the *Proxy* component will learn nothing more than:
   a) *SEnTri V1*: the equality of all terms, belonging to a series of queries, targeting the same triplestore.

b) *SEnTri V2* the equality of all terms present in the graph patterns of individual queries. The bindings of different queries are indistinguishable.

3.3.2 Security & Performance trade-offs

All implementations come with a trade-off between data confidentiality, query evaluation leakage and performance.

SEnTri V0 only provides access control, trading data confidentiality for performance.

SEnTri V1 provides both data confidentiality and access control, but suffers from the same leakage problems of *CryptDB*, where an adversary can associate the leakage produced by multiple queries targeting the same triplestore. This facilitates the computation of the frequency distribution of all observed terms, which simplifies the execution of *leakage-abuse* attacks [43, 38, 82]. This can be especially problematic during SPARQL queries evaluation, as they tend to require a high volume of equality checks. A situation further aggravated when we apply query rewriting techniques: the modified queries usually require an even higher number of equality joins.

In SEnTri V2 provides both data confidentiality, access control and reduced leakage, but this comes at the cost of an extra round of communication where we have to reverse the blinding applied to the *equality tags* and need to fetch the values being referenced from these. Only after this step, can we decrypt the actual query results.

3.3.3 Leakage Analysis

3.3.3.1 SEnTri V1

3.3.3.2 SEnTri V2

3.3.3.3 Discussion

3.4 Complexity Analysis

3.4.1 Storage Complexity

3.4.1.1 SEnTri V0

3.4.1.2 SEnTri V1

3.4.1.3 SEnTri V2

3.4.2 Query Evaluation Complexity

3.4.2.1 SEnTri V0

3.4.2.2 SEnTri V1

3.4.2.3 SEnTri V2

3.4.3 Discussion

3.5 Summary

4

Implementation

4.1 General Considerations

4.1.1 Apache JENA Limitations

4.1.2 Memory Constraints

4.1.3 Design & Architecture Choices

4.2 Component Specific Implementation Details

4.2.1 Client

4.2.1.1 Exposed API

4.2.1.2 Architecture

4.2.1.3 Implementation Details

Batching

SPARQL Query Parsing

Minimizing number of searches

Plan Generator

4.2.2 Triplestore

4.2.2.1 Exposed API

4.2.2.2 Architecture

4.2.2.3 Implementation Details

Redis (Key Structure & Lua Scripts)

Atomicity

4.2.3 IAM Provider

4.2.3.1 Exposed API

4.2.3.2 Architecture

4.2.3.3 Implementation Details

Redis (Key Structure & Lua Scripts)

4.2.4 Proxy

4.2.4.1 Exposed API

4.2.4.2 Architecture

4.2.4.3 Implementation Details

Redis (Key Structure)

SPARQL Query Worker

SPARQL Execution

Encrypted Bindings' Bucketization (SEnTri V2)

4.2.5 Vault

4.2.5.1 Exposed API

4.2.5.2 Architecture

4.2.5.3 Implementation Details

Redis (Key Structure)

4.2.6 Discussion

4.3 Summary

5

Experimental Evaluation

5.1 Experimental Setup & Methodology

5.1.1 LUBM Benchmark adaptation

5.1.2 Experiments

5.2 Experimental Results

5.2.1 Repository Size & (Up)load Time

5.2.2 Query Response Time

5.2.3 Query Completeness & Soundness

5.2.4 Combined Metric

5.3 Discussion

6

Conclusion & Future Work

Bibliography

[1] F. Abel et al. "Enabling advanced and context-dependent access control in RDF stores". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 4825 LNCS. 2007, pp. 1–14. ISBN: 9783540762973. DOI: 10.1007/978-3-540-76298-0_1 (cit. on p. 30).

[2] N. Aburawi, A. Lisitsa, and F. Coenen. "Querying Encrypted Graph Databases". In: *ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy*. Vol. 2018-January. 2018. DOI: 10.5220/0006660004470451 (cit. on pp. 3, 26, 30).

[3] A. Acar et al. "A survey on homomorphic encryption schemes: Theory and implementation". In: vol. 51. 2018. DOI: 10.1145/3214303 (cit. on pp. 3, 20).

[4] Amazon Web Services, Inc. or its affiliates. *Amazon Neptune*. https://docs.aws.amazon.com/neptune/index.html. Accessed: 2022-2-17 (cit. on p. 19).

[5] G. Antoniou et al. "A Semantic Web Primer (Third Edition)". In: *The MIT Press* (2012), p. 287 (cit. on pp. 7, 8, 11, 12, 14).

[6] N. Ashish. "Semantic-Web Technology: Applications at NASA". In: *Dagstuhl Seminar Proceedings*. 2005 (cit. on p. 2).

[7] T. Berners-Lee, J. Hendler, and O. Lassila. *The semantic web*. Vol. 284. 2001. DOI: 10.1038/scientificamerican0501-34 (cit. on p. 1).

[8] M. Blaze, G. Bleumer, and M. Strauss. "Divertible protocols and atomic proxy cryptography". In: *Advances in Cryptology — EUROCRYPT'98*. Springer Berlin Heidelberg, 1998, pp. 127–144. DOI: 10.1007/BFb0054122 (cit. on p. 23).

[9] D. Boneh, A. Sahai, and B. Waters. "Functional Encryption: Definitions and Challenges". In: *Cryptology ePrint Archive* (2010) (cit. on p. 26).

[10] C. Bösch et al. "A Survey of Provably Secure Searchable Encryption". In: *ACM Comput. Surv.* 47.2 (Aug. 2014), pp. 1–51. ISSN: 0360-0300. DOI: 10.1145/2636328 (cit. on pp. 2, 3, 20, 22, 23).

[11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. "(Leveled) Fully Homomorphic Encryption without Bootstrapping". In: *ACM Trans. Comput. Theory* 6.3 (July 2014), pp. 1–36. ISSN: 1942-3454. DOI: 10.1145/2633600 (cit. on pp. 3, 20).

[12] N. Cao et al. "Privacy-preserving query over encrypted graph-structured data in cloud computing". In: *Proceedings - International Conference on Distributed Computing Systems*. 2011. DOI: 10.1109/ICDCS.2011.84 (cit. on p. 25).

[13] J. J. Carroll. "Signing RDF graphs". In: *Lect. Notes Comput. Sci.* 2870 (2003), pp. 369–384. ISSN: 0302-9743, 1611-3349. DOI: 10.1007/978-3-540-39718-2_24 (cit. on p. 30).

[14] M. Carroll, A. Van Der Merwe, and P. Kotzé. "Secure cloud computing: Benefits, risks and controls". In: *2011 Information Security for South Africa - Proceedings of the ISSA 2011 Conference*. 2011. DOI: 10.1109/ISSA.2011.6027519 (cit. on p. 2).

[15] M. Chase and S. Kamara. "Structured encryption and controlled disclosure". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 6477 LNCS. 2010. DOI: 10.1007/978-3-642-17373-8_33 (cit. on pp. 3, 20, 25).

[16] B. Chris and S. Andreas. *Berlin SPARQL Benchmark*. http://wbsg.informatik.uni-mannheim.de/bizer/berlinsparqlbenchmark/spec/index.html. Accessed: 2022-2-17 (cit. on p. 19).

[17] R. Ciucanu and P. Lafourcade. "GOOSE: A Secure Framework for Graph Outsourcing and SPARQL Evaluation". In: *Data and Applications Security and Privacy XXXIV*. Springer International Publishing, 2020, pp. 347–366. DOI: 10.1007/978-3-030-49669-2_20 (cit. on pp. 4, 28, 30).

[18] L. Costabello, S. Villata, and F. Gandon. "Context-aware access control for RDF graph stores". In: *Frontiers in Artificial Intelligence and Applications*. Vol. 242. IOS Press, 2012, pp. 282–287. ISBN: 9781614990970. DOI: 10.3233/978-1-61499-098-7-282 (cit. on p. 30).

[19] L. Costabello et al. "Access control for HTTP operations on linked data". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 7882 LNCS. Springer Verlag, 2013, pp. 185–199. ISBN: 9783642382871. DOI: 10.1007/978-3-642-38288-8_13 (cit. on p. 30).

[20] O. Curé and G. Blin. *RDF Database Systems: Triples Storage and SPARQL Query Processing*. en. Morgan Kaufmann, Nov. 2014. ISBN: 9780128004708 (cit. on pp. 15, 16).

[21] R. Curtmola et al. "Searchable symmetric encryption: Improved definitions and efficient constructions". In: *J. Comput. Secur.* 19.5 (Nov. 2011), pp. 895–934. ISSN: 0926-227X, 1875-8924. DOI: 10.3233/jcs-2011-0426 (cit. on pp. 24, 25).

[22] I. Damgard, M. Geisler, and M. Kroigard. *A correction to 'efficient and secure comparison for on-line auctions'*. 2009. DOI: 10.1504/ijact.2009.028031 (cit. on pp. 20, 39).

[23] I. Damgård, M. Geisler, and M. Krøigaard. "Efficient and Secure Comparison for On-Line Auctions". In: *Information Security and Privacy*. Springer Berlin Heidelberg, 2007, pp. 416–430. DOI: 10.1007/978-3-540-73458-1_30 (cit. on pp. 20, 39).

[24] C. Dong, G. Russello, and N. Dulay. *Shared and Searchable Encrypted Data for Untrusted Servers*. 2008. DOI: 10.1007/978-3-540-70567-3_10 (cit. on p. 23).

[25] M. Dürst and M. Suignard. *Internationalized Resource Identifiers (IRIs)*. en. Tech. rep. 2005. DOI: 10.17487/RFC3987 (cit. on p. 6).

[26] T. Elgamal. "A public key cryptosystem and a signature scheme based on discrete logarithms". In: *IEEE Trans. Inf. Theory* 31.4 (July 1985), pp. 469–472. ISSN: 0018-9448, 1557-9654. DOI: 10.1109/TIT.1985.1057074 (cit. on p. 20).

[27] D. Evans, V. Kolesnikov, and M. Rosulek. "A Pragmatic Introduction to Secure Multi-Party Computation". In: *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246. ISSN: 2474-1558. DOI: 10.1561/3300000019 (cit. on pp. 2, 19).

[28] J. D. Fernández et al. "Self-Enforcing Access Control for Encrypted RDF". In: *The Semantic Web*. Springer International Publishing, 2017, pp. 607–622. DOI: 10.1007/978-3-319-58068-5_37 (cit. on pp. 4, 26, 30).

[29] B. Ferreira et al. "Boolean Searchable Symmetric Encryption with Filters on Trusted Hardware". In: *IEEE Trans. Dependable Secure Comput.* (2020), pp. 1–1. ISSN: 1941-0018. DOI: 10.1109/TDSC.2020.3012100 (cit. on p. 3).

[30] G. Flouris et al. "Controlling access to RDF graphs". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 6369 LNCS. 2010, pp. 107–117. ISBN: 9783642158766. DOI: 10.1007/978-3-642-15877-3_12 (cit. on p. 30).

[31] C. Fontaine and F. Galand. "A survey of homomorphic encryption for nonspecialists". In: *Eurasip Journal on Information Security* 2007 (2007). ISSN: 2510-523X. DOI: 10.1155/2007/13801 (cit. on pp. 3, 20).

[32] M. Franz. *Secure Computations on Non-Integer Values*. 2011 (cit. on pp. 20, 21, 39).

[33] *Gartner forecasts worldwide public cloud end-user spending to reach nearly $600 billion in 2023*. en. https://www.gartner.com/en/newsroom/press-releases/2022-10-31-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-reach-nearly-600-billion-in-2023. Accessed: 2023-8-21 (cit. on p. 2).

[34] C. Gentry. *A Fully Homomorphic Encryption Scheme*. 2009 (cit. on p. 20).

[35] O. Goldreich. *Foundations of Cryptography: Volume 2, Basic Applications*. en. Cambridge University Press, 2001. ISBN: 9780521830843 (cit. on pp. 2, 19).

[36] O. Goldreich and R. Ostrovsky. "Software protection and simulation on oblivious RAMs". In: *J. ACM* 43.3 (May 1996), pp. 431–473. ISSN: 0004-5411. DOI: 10.1145/233551.233553 (cit. on pp. 3, 20).

[37] Y. Guo, Z. Pan, and J. Heflin. "LUBM: A benchmark for OWL knowledge base systems". In: *Journal of Web Semantics* 3.2 (Oct. 2005), pp. 158–182. ISSN: 1570-8268. DOI: 10.1016/j.websem.2005.06.005 (cit. on p. 19).

[38] F. Hahn, N. Loza, and F. Kerschbaum. "Joins Over Encrypted Data with Fine Granular Security". In: *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. Apr. 2019, pp. 674–685. DOI: 10.1109/ICDE.2019.00066 (cit. on pp. 30, 78).

[39] M. A. Harris et al. "The Gene Ontology (GO) database and informatics resource". en. In: *Nucleic Acids Res.* 32.Database issue (Jan. 2004), pp. D258–61. ISSN: 0305-1048, 1362-4962. DOI: 10.1093/nar/gkh036 (cit. on p. 1).

[40] *HTTP Over TLS*. en. https://datatracker.ietf.org/doc/html/rfc2818. Accessed: 2022-2-18 (cit. on p. 33).

[41] A.-A. Ivan and Y. Dodis. "Proxy Cryptography Revisited". In: *NDSS*. 2003 (cit. on p. 23).

[42] A. Jain and C. Farkas. "Secure resource description framework: An access control model". In: *Proceedings of ACM Symposium on Access Control Models and Technologies, SACMAT*. Vol. 2006. 2006, pp. 121–129. ISBN: 9781595933546 (cit. on p. 30).

[43] C. Jutla and S. Patranabis. "Efficient Searchable Symmetric Encryption for Join Queries". In: *Cryptology ePrint Archive* (2021) (cit. on pp. 30, 78).

[44] S. Kamara, C. Papamanthou, and T. Roeder. "Dynamic searchable symmetric encryption". In: *Proceedings of the 2012 ACM conference on Computer and communications security*. CCS '12. Raleigh, North Carolina, USA: Association for Computing Machinery, Oct. 2012, pp. 965–976. ISBN: 9781450316514. DOI: 10.1145/2382196.2382298 (cit. on p. 25).

[45] A. Kasten and A. Scherp. *Iterative Signing of RDF(S) Graphs, Named Graphs, and OWL Graphs: Formalization and Application*. Tech. rep. Mainz: Universität Koblenz-Landau, Mar. 2013 (cit. on p. 30).

[46] A. Kasten and A. Scherp. "Towards a configurable framework for iterative signing of distributed graph data". In: *CEUR Workshop Proceedings*. Vol. 1121. 2014 (cit. on p. 30).

[47] A. Kasten, A. Scherp, and P. Schauß. "A framework for iterative signing of graph data on the web". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 8465 LNCS. Springer Verlag, 2014, pp. 146–160. ISBN: 9783319074429. DOI: 10.1007/978-3-319-07443-6_11 (cit. on p. 30).

[48] S. Kirrane et al. "Secure manipulation of linked data". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 8218 LNCS. 2013. DOI: 10.1007/978-3-642-41335-3_16 (cit. on p. 30).

[49] G. Ladwig. *Efficient Optimization and Processing of Queries Over Text-rich Graph-structured Data*. en. KIT Scientific Publishing, 2013. ISBN: 9783731500155. DOI: 10.5445/KSP/1000034423 (cit. on p. 16).

[50] P. Leach, M. Mealling, and R. Salz. *A Universally Unique IDentifier (UUID) URN Namespace*. Tech. rep. July 2005. DOI: 10.17487/rfc4122 (cit. on p. 35).

[51] A. Letelier et al. "Static analysis and optimization of semantic web queries". In: *ACM Trans. Database Syst.* 38.4 (Dec. 2013), pp. 1–45. ISSN: 0362-5915. DOI: 10.1145/2500130 (cit. on p. 16).

[52] M. X. Makkes. *Efficient implementation of homomorphic cryptosystems*. 2010 (cit. on p. 39).

[53] A. Maña, G. Pujol, and C. Pandolfo. "A distributed secure ontology for certified services oriented applications". In: *Proceedings - 5th IEEE International Conference on Semantic Computing, ICSC 2011*. 2011. DOI: 10.1109/ICSC.2011.105 (cit. on p. 30).

[54] MarkLogic. *MarkLogic server - multi-model database*. en. https://www.marklogic.com/product/marklogic-database-overview/. Accessed: 2022-2-17. Feb. 2020 (cit. on p. 19).

[55] J. P. McCrae. *The linked open data cloud*. en. https://lod-cloud.net/. Accessed: 2022-2-8 (cit. on p. 2).

[56] B. D. McKay and A. Piperno. "Practical graph isomorphism, II". In: *J. Symbolic Comput.* 60 (Jan. 2014), pp. 94–112. ISSN: 0747-7171. DOI: 10.1016/j.jsc.2013.09.003 (cit. on p. 16).

[57] G. A. Miller. "WordNet: a lexical database for English". In: *Commun. ACM* 38.11 (Nov. 1995), pp. 39–41. ISSN: 0001-0782. DOI: 10.1145/219717.219748 (cit. on p. 1).

[58] G. E. Modoni, M. Sacco, and W. Terkaj. "A survey of RDF store solutions". In: *2014 International Conference on Engineering, Technology and Innovation: Engineering Responsible Innovation in Products and Services, ICE 2014*. 2014. DOI: 10.1109/ICE.2014.6871541 (cit. on p. 15).

[59] *MongoDB Manual: Queryable encryption*. en. https://www.mongodb.com/docs/v7.0/core/queryable-encryption/. Accessed: 2023-8-21 (cit. on p. 2).

[60] M. A. Musen and Protégé Team. "The Protégé Project: A Look Back and a Look Forward". en. In: *AI Matters* 1.4 (June 2015), pp. 4–12. ISSN: 2372-3483. DOI: 10.1145/2757001.2757003 (cit. on pp. 2, 19).

[61] R. Neches et al. "Enabling technology for knowledge sharing". In: *AI Magazine* 12.3 (1991). ISSN: 0738-4602 (cit. on p. 1).

[62] R. Nejjahi, A. Marzouk, and N. Gherabi. "An Enhanced Method for Web Ontologies Encryption". In: *European Journal of Electrical Engineering and Computer Science* 3.4 (July 2019). ISSN: 2506-9853. DOI: 10.24018/ejece.2019.3.4.99 (cit. on p. 26).

[63] Neo4j, Inc. *Cypher Query Language*. en. https://neo4j.com/developer/cypher/. Accessed: 2023-8-24 (cit. on pp. 3, 26, 28).

[64] N. Noy et al. "Industry-scale knowledge graphs: Lessons and challenges". In: *Commun. ACM* 62.8 (2019). ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3331166 (cit. on p. 1).

[65] Ontotext. *GraphDB*. en. https://graphdb.ontotext.com/documentation/. Accessed: 2022-2-17 (cit. on p. 19).

[66] OpenLink Software. *Virtuoso Universal Server*. https://virtuoso.openlinksw.com/. Accessed: 2022-2-17 (cit. on p. 19).

[67] owl.cs. *OWL API repository*. en. https://github.com/owlcs/owlapi. Accessed: 2022-2-17 (cit. on p. 19).

[68] P. Paillier. "Public-Key Cryptosystems Based on Composite Degree Residuosity Classes". In: *Advances in Cryptology — EUROCRYPT '99*. Springer Berlin Heidelberg, 1999, pp. 223–238. DOI: 10.1007/3-540-48910-X_16 (cit. on pp. 20, 28, 39).

[69] V. Papakonstantinou et al. "Access control for RDF graphs using abstract models". In: *Proceedings of ACM Symposium on Access Control Models and Technologies, SACMAT*. 2012. DOI: 10.1145/2295136.2295155 (cit. on p. 30).

[70] W. Pearson. *Cryptography and Network Security: Principles and Practice, Global Edition*. 7th ed. Pearson Education, 2017. ISBN: 9781292158587 (cit. on pp. 3, 24, 26).

[71] Z. Peng, S. Tang, and L. Jiang. *A Symmetric Authenticated Proxy Re-encryption Scheme with Provable Security*. 2017. DOI: 10.1007/978-3-319-68542-7_8 (cit. on p. 23).

[72] F. Picalausa and S. Vansummeren. "What are real SPARQL queries like?" In: *Proceedings of the International Workshop on Semantic Web Information Management*. SWIM '11 Article 7. Athens, Greece: Association for Computing Machinery, June 2011, pp. 1–6. ISBN: 9781450306515. DOI: 10.1145/1999299.1999306 (cit. on pp. 15–17).

[73] G. S. Poh, M. S. Mohamad, and M. R. Z'Aba. "Structured encryption for conceptual graphs". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*. Vol. 7631 LNCS. Springer Verlag, 2012, pp. 105–122. ISBN: 9783642341168. DOI: 10.1007/978-3-642-34117-5_7 (cit. on pp. 3, 25).

[74] R. A. Popa, N. Zeldovich, and H. Balakrishnan. "CryptDB : A Practical Encrypted Relational DBMS". In: *Designfax* (2011). ISSN: 0163-6669 (cit. on pp. 3, 26, 34).

[75] R. A. Popa et al. "CryptDB: Processing queries on an encrypted database". In: *Communications of the ACM*. Vol. 55. 2012. DOI: 10.1145/2330667.2330691 (cit. on pp. 3, 22, 26, 34).

[76] R. A. Popa et al. "CryptDB: Protecting confidentiality with encrypted query processing". In: *SOSP'11 - Proceedings of the 23rd ACM Symposium on Operating Systems Principles*. 2011. DOI: 10.1145/2043556.2043566 (cit. on pp. 3, 26, 34).

[77] *RDF 1.1: On semantics of RDF datasets*. en. https://www.w3.org/TR/rdf11-datasets/. Accessed: 2023-10-14 (cit. on p. 15).

[78] R. L. Rivest, L. Adleman, and M. L. Dertouzos. "On data banks and privacy homomorphisms". In: *Foundations of secure* (1978) (cit. on p. 20).

[79] R. L. Rivest, A. Shamir, and L. Adleman. "A method for obtaining digital signatures and public-key cryptosystems". en. In: *Commun. ACM* 21.2 (Feb. 1978), pp. 120–126. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/359340.359342 (cit. on p. 20).

[80] K. Sakurai, T. Nishide, and A. Syalim. *Improved proxy re-encryption scheme for symmetric key cryptography*. 2017. DOI: 10.1109/iwbis.2017.8275110 (cit. on p. 23).

[81] Security Compass. *The 2021 State of Cloud Adoption*. en. https://resources.securitycompass.com/reports/2021-state-of-cloud-adoption. Accessed: 2022-2-9. Sept. 2021 (cit. on p. 2).

[82] M. Shafieinejad et al. "Equi-Joins Over Encrypted Data for Series of Queries". In: (Mar. 2021). arXiv: 2103.05792 [cs.CR] (cit. on pp. 30, 78).

[83] N. Sioutos et al. "NCI Thesaurus: a semantic model integrating cancer-related clinical and molecular information". en. In: *J. Biomed. Inform.* 40.1 (Feb. 2007), pp. 30–43. ISSN: 1532-0464, 1532-0480. DOI: 10.1016/j.jbi.2006.02.013 (cit. on p. 1).

[84] E. Sirin et al. "Pellet: A practical OWL-DL reasoner". In: *Journal of Web Semantics* 5.2 (June 2007), pp. 51–53. ISSN: 1570-8268. DOI: 10.1016/j.websem.2007.03.004 (cit. on p. 19).

[85] D. X. Song, D. Wagner, and A. Perrig. "Practical techniques for searches on encrypted data". In: *Proceeding 2000 IEEE Symposium on Security and Privacy. S P 2000*. May 2000, pp. 44–55. DOI: 10.1109/SECPRI.2000.848445 (cit. on pp. 3, 22, 28).

[86] D. X. Song, D. Wagner, and A. Perrig. "Practical techniques for searches on encrypted data". In: *Proceeding 2000 IEEE Symposium on Security and Privacy. S P 2000*. May 2000, pp. 44–55. DOI: 10.1109/SECPRI.2000.848445 (cit. on p. 24).

[87] R. Studer, V. R. Benjamins, and D. Fensel. "Knowledge engineering: Principles and methods". In: *Data Knowl. Eng.* 25.1 (Mar. 1998), pp. 161–197. ISSN: 0169-023X. DOI: 10.1016/S0169-023X(97)00056-6 (cit. on p. 10).

[88] G. Suntaxi, A. A. El Ghazi, and K. Böhm. "Secrecy and performance models for query processing on outsourced graph data". In: *Distributed and Parallel Databases* 39.1 (Mar. 2021), pp. 35–77. ISSN: 1573-7578. DOI: 10.1007/s10619-020-07284-0 (cit. on p. 25).

[89] A. Sutton and R. Samavi. "Timestamp-based Integrity Proofs for Linked Data". In: *Proceedings of the International Workshop on Semantic Big Data, SBD 2018 - In conjunction with the 2018 ACM SIGMOD/PODS Conference*. Association for Computing Machinery, Inc, June 2018. ISBN: 9781450357791. DOI: 10.1145/3208352.3208353 (cit. on p. 30).

[90] A. Syalim, T. Nishide, and K. Sakurai. "Realizing Proxy Re-encryption in the Symmetric World". In: *Informatics Engineering and Information Science*. Springer Berlin Heidelberg, 2011, pp. 259–274. DOI: 10.1007/978-3-642-25327-0_23 (cit. on p. 23).

[91] The Apache Software Foundation. *Apache Jena*. en. https://jena.apache.org/. Accessed: 2022-2-17 (cit. on pp. 19, 48, 59).

[92] The Apache Software Foundation. *Apache Jena - TDB*. en. https://jena.apache.org/documentation/tdb/index.html. Accessed: 2022-2-17 (cit. on p. 19).

[93] *The Transport Layer Security (TLS) Protocol Version 1.3*. en. https://datatracker.ietf.org/doc/html/rfc8446. Accessed: 2022-2-18 (cit. on p. 33).

[94] T. Tudorache, J. Vendetti, and N. F. Noy. "Web-Protégé: A lightweight OWL ontology editor for the web". In: *CEUR Workshop Proceedings*. Vol. 432. 2009 (cit. on pp. 2, 19).

[95] T. Tudorache et al. "Supporting collaborative ontology development in Protégé". In: *Lect. Notes Comput. Sci.* 5318 LNCS (2008), pp. 17–32. ISSN: 0302-9743, 1611-3349. DOI: 10.1007/978-3-540-88564-1_2 (cit. on pp. 2, 19).

[96] G. Tummarello et al. "Signing individual fragments of an RDF graph". In: *14th International World Wide Web Conference, WWW2005*. 2005, pp. 1020–1021. ISBN: 9781595930514. DOI: 10.1145/1062745.1062848 (cit. on p. 30).

[97] W3C. *Extensible markup language (XML) 1.0 (fifth edition)*. en. https://www.w3.org/TR/xml/. Accessed: 2022-2-11 (cit. on p. 7).

[98] W3C. *OWL 2 web ontology language direct semantics (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[99] W3C. *OWL 2 web ontology language document overview (second edition)*. en. https://www.w3.org/TR/owl2-overview/. Accessed: 2022-2-11 (cit. on pp. viii, 1, 7, 11).

[100] W3C. *OWL 2 web ontology language Manchester syntax (second edition)*. en. https://www.w3.org/TR/2012/NOTE-owl2-manchester-syntax-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[101] W3C. *OWL 2 web ontology language profiles (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-profiles-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[102] W3C. *OWL 2 web ontology language quick reference guide (second edition)*. en. https://www.w3.org/TR/owl2-quick-reference/. Accessed: 2022-2-14 (cit. on p. 12).

[103] W3C. *OWL 2 web ontology language RDF-based semantics (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-rdf-based-semantics-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[104] W3C. *OWL 2 web ontology language structural specification and functional-style syntax (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-syntax-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[105] W3C. *OWL 2 web ontology language XML serialization (second edition)*. en. https://www.w3.org/TR/2012/REC-owl2-xml-serialization-20121211/. Accessed: 2022-2-14 (cit. on p. 11).

[106] W3C. *RDF 1.1 concepts and abstract syntax*. en. https://www.w3.org/TR/rdf11-concepts/. Accessed: 2022-2-11 (cit. on pp. 1, 7).

[107] W3C. *RDF 1.1 TriG*. en. https://www.w3.org/TR/trig/. Accessed: 2022-2-13 (cit. on p. 8).

[108] W3C. *RDF 1.1 turtle*. en. https://www.w3.org/TR/turtle/. Accessed: 2022-2-13 (cit. on p. 8).

[109] W3C. *RDF 1.1 XML syntax*. en. https://www.w3.org/TR/rdf-syntax-grammar/. Accessed: 2022-2-13 (cit. on p. 8).

[110] W3C. *RDF Schema 1.1*. en. https://www.w3.org/TR/rdf-schema/. Accessed: 2022-2-11 (cit. on pp. 1, 7, 9).

[111] W3C. *RDFa core 1.1 - third edition*. en. https://www.w3.org/TR/rdfa-core/. Accessed: 2022-2-13 (cit. on p. 8).

[112] W3C. *Semantic Web Case Studies and Use Cases*. en. https://www.w3.org/2001/sw/sweo/public/UseCases/. Accessed: 2022-2-8 (cit. on p. 1).

[113] W3C. *SPARQL 1.1 graph store HTTP protocol*. en. https://www.w3.org/TR/sparql11-http-rdf-update/. Accessed: 2022-2-4. Mar. 2013 (cit. on p. 15).

[114] W3C. *SPARQL 1.1 Overview*. https://www.w3.org/TR/sparql11-overview/. Mar. 2013 (cit. on pp. 1, 15).

[115] W3C. *SPARQL 1.1 update*. en. https://www.w3.org/TR/sparql11-update/. Accessed: 2022-2-15 (cit. on pp. 1, 15, 18, 37).

[116] W3C. *SPARQL Query Language for RDF*. https://www.w3.org/TR/rdf-sparql-query/. Jan. 2008 (cit. on pp. 1, 15, 17).

[117] W3C. *SWRL: A Semantic Web Rule Language Combining OWL and RuleML*. https://www.w3.org/Submission/SWRL/. May 2004 (cit. on p. 7).

[118] Wikipedia contributors. *Semantic web stack*. https://en.wikipedia.org/wiki/File:Semantic_web_stack.svg. Accessed: 2022-2-15 (cit. on pp. viii, 6).